

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

COS 102 — Past Questions & Materials

by Renance

Course: COS 102

Institution: Federal University of Petroleum Resources, Effurun

Papers in this pack: 3

Levels: 100

This study pack was compiled and restructured by Renance for COS 102 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app

Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — COS 102

Level 100 • First Semester • Handout

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 02 – The Problem Solving
Approach

These slides are based on textbooks and other materials accompanying this course

6/29/2017 1

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Let us Begin.....

6/29/2017 2

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Definitions

□ Computational Thinking is not to replace problem solving. Rather, problem solving is a part of computational thinking and is held as thus.

□ Problem Solving – is the sequential process of analyzing information related to a given situation and generating appropriate response options. There are 6-steps in problem solving...

- a. Understand the problem
- b. Formulate a model
- c. Develop an Algorithm
- d. Write the program
- e. Test and Debug the Program
- f. Evaluate the solution

□ For Problem Solving, we must consider Input, Process and Output parameters to work out the solution of any task as simple as it may be. For example – calculate the average grade of a student's record

1. Input – get all the grades for the various courses offered by the student
2. Process – add them up and compute average grade
3. Output – show answer to either monitor, printer or a combination of all

3

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 1 - Understand the Problem

□ In trying to understand the problem, you need to know:

- a. What input data or information is required by the system or is available?
- b. What does it represent?
- c. What format is it in?
- d. Is anything missing?
- e. Do I have everything that I need to make this work?
- f. What output information am I trying to produce?
- g. What and how do I want the result to look like...text, picture, graph etc?
- h. What am I going to have to compute?

□ As in our example

1. Our input is bunch of grades. We need to understand the grade format (either as number from 0 to 100, or as letters from A to F). If numbers, the grade may be whole integer or real numbers. We need to understand the format of grades in order to solve this problem.
2. We also consider missing grades. What if we do not have the grade for every student (some were away during the CA or test?). Do we want to include that person or ignore them when computing the average? We need to understand what the output should be formatted as or displayed as (may be as pie chart with average grade etc). Finally, we must understand the kind of processing to be performed on the data. This leads us to...

4

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 2 - Formulate a Model

□ Here, we try to understand the processing part of the problem. Many tasks do break into smaller parts that requires mathematical computations no matter how simple in order to process the data. In our example:

- a. We need to know the model (formular) for computing the average (completing the task)
- b. If there is no such formular, we need to develop one
- c. However the task breaks up into smaller parts, we need to understand it well enough by looking for available information
- d. It is all about figuring out how to make use of available data to compute an answer

Step 3 - Develop an Algorithm

□ Here, we need to have a precise plan of what we want the computer to do. An

algorithm is a set of instructions required for solving a problem

- a. Many complex algorithms are considered randomized or non-deterministic as instructions are not necessarily in order, and may not even have a finite number of sequence.
- b. To develop an algorithm in a way understandable to a person trying to figure out the steps, we use: (1) pseudo-codes (English-Like statements), and (2) flowcharts (picture formats that shows processes and decisions).
- c. Pseudocodes are preferred: (1) tedious to draw flowchart neatly when there are mistakes, (2) pseudocodes fits more easily on a page, (3) they can be written in a way closer to the real program, (4) it takes lesser time to write than drawing a flowchart.

5

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 3 – Develop an Algorithm...

□ Psuedocodes varies on whoever writes it. But, some common control structures (i.e. feats) that appears as we write a pseudocode includes:

□ Sequence – list instructions step-by-step in order and they are often numbered. E.g.

- a. Make sure switch is turned on
- b. Check if lamp is plugged in
- c. Check if bulb is burned out

□ Condition – making a decision. That is, doing one thing or something else depending on outcome of the decision

If lamp is not plugged in

Then plug it in

If bulb is burned out

Then replace bulb

otherwise, buy new bulb

□ Repetition – repeats something a fixed number of times or until a condition occurs
repeat

get new bulb

put it in the lamp

until lamp works or no more bulb left

□ Storage – stores information for use in instruction further down the list

X □ a new bulb

Count □ 8

6

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 3 – Develop an Algorithm...

□ Jumping – being able to go to or jump to a specific step when and where needed

If lamp works

Then goto step 12

□ Some people however will use brackets to indicate what the inner working do as a control structure such as:

If (bulb is burned out) then {

replace bulb }

Otherwise {

Buy new lamp

Or,

Repeat {

get a new light bulb

put it in the lamp }

until (lamp works or no more bulbs left)

Or

Repeat 3times {

Unplug lamp

Plug into different socket }

□ Simple, there are various ways to write pseudocode. Only remember that your algorithm should be

clearly explained with no ambiguity as to the order in which the steps are performed.

□ Also test your algorithm manually by going thru in your head to ensure that you did not forget a step

or special situation. Only when sure that your algorithm is not flawed can you proceed to the next..

7

6/29/2017

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Step 3 – Develop an Algorithm...

□ Suppose we extend the program as in our example to display grades of students – then we have the pseudocode as below

1. Set the sum of grade values to 0.

2. Load all grades $x_1 \dots x_n$ from file

3. Repeat n times {

4. get grades x_1

5. add x_1 to the sum

6. }

7. Compute the average to be sum / n

8. Print the average

□ It is often very wise to test the algorithm by running through it with real numbers and data values. Each time we do this (i.e. test the algorithm) with a fixed set of data input(numeric values), this is known as test case.

□ We can create many test cases as possible. Here are some examples:

□ $N = 5, x_1 = 92, x_2 = 37, x_3 = 43, x_4 = 12, x_5 = 71$result should be 51

□ $N = 3, x_1 = 1, x_2 = 2, x_3 = 3$result should be 2

□ $N = 2, x_1 = 1, x_2 = 1$result should be 1

□ $N = 0$result should be 0

□ Create your own test cases and see how it works.

8

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 4 - Write the Program

□ Writing a program is often termed as coding (so the code or source code is actually the program itself). We have the sample below and notice the codes looks quite similar

□ Pseudocode □ Processing Code

1. Set the sum of grade values to 0. 1. Int sum = 0

2. Load all grades $x_1 \dots x_n$ from file 2. X = eval (input "Enter the numbers ")

3. Repeat n times { 3. For (int i = 0, i < x, i+1)

4. get grades x_1

4. sum = sum + x

5. add x_1 to the sum

5. Int average = sum / x

6. }

6. Print Average

7. Compute the average to be sum / n

8. Print the average

□ However, the processing code is less readable and seems more mathematical. Also, you need to know that the source code will vary depending on the programming language used.

□ Learning a programming language will seem difficult at first; But, it will become easier with much practice. The computer requires the precise instruction in order to understand what you are asking it to accomplish and do. For example, where the appropriate syntax is not adhered to, results in a syntax or compile error. This will cause a bug, and will consequently need to be fixed before the program can be executed.

□ Compiling is the process of converting a source code into instructions that can be understood directly by the computer hardware. The longer your program, the more likely there will be multiple, compile errors. Do not fret over such. It requires patience to fix...

9

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 5 – Test the Program

□ Testing the program is about ensuring that the program runs as should, does exactly what it set out to do and that the solution is correct. When we execute a program, the term is running.

□ Running a program simply tells the computer to evaluate the compiled instruction. When we run a program and all is well, we will see the correct output. It is also very possible that a program runs correctly for some set of data. If the output of your program is incorrect, it is possible that you did not convert your algorithm very well and properly.

□ It is also possible that you did not produce a proper algorithm back in step 3 that handles all situations that could arise.

□ And just may be, you performed some instructions out-of-sequence. Whatever be the case, these result in bugs. Bugs are problem, errors or issues that cause your program not to run correctly or that which stops your program from producing correct and desired results.

□ To easily find bugs effectively, we are advised to test our program with many test cases (called a test suite). It is also a good idea to have others test your program because they may think up situations or input data that you may never have thought of.

□ The process of finding and fixing errors in your code is called debugging and it is often a very time-consuming “chore” when it comes to being a programmer. If you take your time to carefully follow problem solving steps 1 through 3, this should greatly reduce the amount of bugs in your programs and it should make debugging much easier.

10

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Step 6 – Evaluate the solution

□ Once your program produces a result that seems correct, you need to re-consider the original problem and make sure that the answer is formatted into a proper solution to the problem. It is often the case that you realize that your program solution does not solve the problem the way that you wanted it to. You may realize that more steps are involved.

- For example, if the result of your program is a long list of numbers, but your intent was to determine a pattern in the numbers or to identify some feature from the data, then simply producing a list of numbers may not suffice. There may be a need to display the information in a way that helps you visualize or interpret the results with respect to the problem. Perhaps a chart or graph is needed.

- It is also possible that when you examine your results, you realize that you need additional data to fully solve the problem. Or, perhaps you need to adjust the results to solve the problem more efficiently (e.g., your game is too slow). It is important to remember that the

computer will only do what you told it to do. It is up to you to interpret the results in a meaningful way and determine whether or not it solves the original problem. It may be necessary to re-do some of the steps again, perhaps going as far back as step 1 again, if data was missing.

- Try out as many examples as are possible for cases and scenarios where we adopt all the problem solving steps.

11

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Control Abstraction

□ In dealing with algorithms – it is not clear as to how much detail should go into our algorithm. E.g. consider that you are at home on the

couch and you are thirsty. How do you solve the problem? Here is a simple algorithmic solution to your problem:

1. Go to kitchen
2. Open refrigerator
3. Choose a drink
4. Drink it

□ But, we could have come up with a more abstract (i.e., less detailed) algorithm as thus:

1. Get a drink
2. Drink it.

Or, we could have been much more detailed as follows:

1. Get off and Walk to kitchen
2. Open refrigerator
3. If there is a orange juice then {
4. take the juice
5. close refrigerator
6. go to the cupboard
7. open cupboard and take a glass
8. close cupboard
9. pour juice into glass
10. go to refrigerator
11. open refrigerator
12. put carton in refrigerator
13. close refrigerator }
14. otherwise if there is a soda then {
15. take soda and close refrigerator
16. open soda }
17. drink it

□ Each of these algorithmic solutions solves the problem. So then, which one is best ? The answer is not always easy. As a rule of thumb, we want to produce the simplest possible algorithm that is easily understandable. While it is important to provide enough detail to be able to properly describe the problem solution, it is also important not to get hung up on too much detail so that the algorithm becomes cluttered and overly complicated.

12

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Control Abstraction

□ Abstraction tries to reduce or factor out details that are not necessary in order to describe an algorithm. Abstraction is important because it allows us to focus on a few concepts at a time. This allows us to get the “big picture” first in regards to the problem solution.

□ We can then “fill in” the specific details at a later point in time. E.g. in the example, a statement like “get a drink from the refrigerator” would suffice when that step is part of a larger problem such as an algorithm that describes a person’s daily routine around dinner time. However, if we needed to program such a step into a robot, then much more detail would be needed because the robot would require a more precise set of movements in order to carry out such an operation.

□ A low resolution image has less detail and is more abstract than a high-resolution image. Abstraction allows us to lower the resolution of the image so as to save space (i.e., by hiding details) unless they are absolutely necessary.

- Abstraction that allows our algorithm steps to be quite abstract (high level) or more detailed (low level) is known as control abstraction.

- In cases where details are necessary, it is still possible to provide a higher-level algorithm, while allowing the more specific details to be described in a sub-algorithm or subprogram. As a result, we are able to describe an algorithm at multiple layers of abstraction. E.g in our thirst-quenching scenario, we could use the details of our third very specific algorithmic solution but hide the more detailed portions in a sub-algorithm as follows:

13

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Control Abstraction

□ We can take this algorithm and turn it into:

1. Walk to kitchen
2. Open refrigerator

3. Perform subprogram1

4. Close refrigerator

5. Drink it

Subprogram1: GetDrink

1. If there is a orange juice then {

2. take the juice

3. close refrigerator

4. go to the cupboard

5. open cupboard and take a glass

6. close cupboard

7. pour juice into glass

8. go to refrigerator

9. open refrigerator

10. put juice in refrigerator and close refrigerator }

11. otherwise if there is a soda then {

12. take soda

13. open soda }

□ Notice main algorithm is quite simple with much less detail. SubAlgorithm1 is not a stand-alone algorithm in that it cannot solve the problem on its own.

14

6/29/2017

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Control Abstraction

□ AlgorithmX: QuenchThirst

1. Walk to kitchen

2. Perform subalgorithm1

3. Drink it

Subprogram1: GetDrink

1. Open refrigerator

2. If there is a orange juice then {

3. perform subalgorithm2 }

4. otherwise if there is a soda then {

5. take and open soda }

6. Close refrigerator

Subprogram2

1. take the juice

2. close refrigerator

3. go to the cupboard

4. open cupboard and take a glass

5. close cupboard
6. pour juice into glass
7. go to refrigerator
8. open refrigerator
9. put juice in refrigerator

□ Notice subalgorithm1 has also become easier to read and more abstract. We can keep abstracting

as we feel. There are no rules. A subprogram is small enough when it is easily understood.

15

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Algorithm Efficiency

□ When two or more algorithms are put forward, we can sometime seek to consider which of them yield better result at faster time. Being better here depends on what better means.

- Algorithm efficiency is used to describe properties of an algorithm relating to how much of various types of resources it consumes. (Wikipedia). In CS, we are interested in algorithms that are time and space (memory) efficient (though there are also other ways or metrics) for measuring efficiency. For example, an algorithm can be more efficient in terms of time; But may not be more efficient in terms of space that it takes to execute. Another algorithm may be more efficient in terms of detailing in that, depending on the context, it may take longer for a person to guess what it aims to achieve. In this case, the extra time taken to distinguish what it set out to accomplish may result in slower or quicker guess.
- Runtime complexity (or running time) of an algorithm is the time that it takes to complete once it has begun. Space complexity is amount of storage space it requires to run from start to finish. A parameter is a piece of data provided as input to a function or procedure.

In algorithmY1: EfficientSetTableFor8 In algorithmY2: EfficientSetTableFor4

1. walk to kitchen 1. walk to kitchen
2. getGlasses(8) 2. getGlasses()
3. place glasses on table 3. place glasses on table
4. getPlates(8) and place on table 4. getPlates() and place on table
5. getUtensils(8) 5. getUtensils()
6. place knives and forks on table 6. place knives and forks on table
7. go back onto couch 7. go back onto couch

We supplied a number 8 between parenthesis in our function. That is where we supply additional data

(i.e parameter) to the function and it is clear how many settings will be done in Algorithm1 and not in 2.

16

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Algorithm Efficiency....

□ But what will the `getGlass()` function look like? Here are two versions of it
`getGlass():` `getGlasses(n)`

1. Go to the cupboard 1. Go to cupboard
2. Open cupboard 2. Open cupboard
3. Take a glass 3. Repeat n times {
4. Close cupboard 4. take a glass }
5. Close cupboard

□ You notice that the function `getGlass(n)` specifies the parameter for the function and uses it within the function itself. Notice how the parameter is now being used within the function to get the necessary glasses.

□ The value of `n` will vary according to how we call the function. E.g. if we use `getGlasses(8)`, then within the function, `n` will have the value of 8. If we use `getGlasses(4)`, then within the function, `n` will have the value of 4. So, the value for parameter `n` will always be the number that was passed in when the function was called.

□ In more general algorithms, we use `n` as a 'placeholder' or 'label' to tell the algorithm to work for 0-to-`n`. Any letter can be used. So, the statement `getGlasses(n)` indicates "get `n` glasses", where `n` may be any integer number that we want.

□ There is a limit as to the glasses a person carries. Thus, to describe an algorithm in a general way, `n` indicates our algorithm will work from 0 to `n`; Instead of a fixed number. Thus allows us compare two algorithms in regards to their efficiency. We can often compare the number of steps one algorithm requires with another algorithm.

17

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Algorithm Efficiency....

□ For example, consider the two algorithms for setting the table for `n`-people, we have that:
`EfficientSetTable1` `EfficientSetTable2`

1. Walk to kitchen 1. Walk to kitchen
2. repeat n times { 2. `getGlasses(n)`
3. `getGlasses()` 3. place glass on table
4. place glass on table 4. `getPlates(n)`
5. `getPlate()` 5. place plate on table
6. place plate on table 6. `getUtensils(n)`
7. `getUtensils()` 7. place knife and fork on table }
8. place knife and fork on table } 8. go back to couch

9. go back to couch

□ What if we define “efficiency” as number of times we walked to/fro the cupboard and the table? Which is more efficient? Each time thru the loop, Algorithm1 makes 3trips to/fro cupboard and table. With n-times place settings (i.e., n times thru loop), the algorithm takes $n \times 3$ steps. But in Algorithm2, it takes only 3 trips total, regardless of the place settings will be required.

□ Thus, we conclude. If we are setting table for 1 person, either algorithm is good. If setting for 2 people, Algorithm2 is twice more efficient than Algorithm1 since it requires only half travel to/fro cupboard and table. As n gets larger, the difference becomes more significant. Thus, if we are setting for 8-people, Algorithm1 makes 8 times the trip (total of 24) than Algorithm2 that takes only 3 trips.

18

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Algorithm Efficiency....

- Regardless of the number of place settings, Algorithm2 has a fixed cost of 3 (i.e. to/from travels). Since this cost is fixed, we say algorithm2 has constant efficiency in terms of our particular cost metric. But, Algorithm1 is linear because its efficiency grows equally with respect to n.
- Sometimes an algorithm has a constant value times n (e.g., $3n$). Since the 3 is constant (i.e., fixed in our case, because we have exactly 3-kinds of items we are placing), the algorithm1 is also considered to be linear. If we were to vary the items to be n-items (e.g. place 8 items at each of the 8 people’s place settings, or 12 items at each of the 12 person’s place settings), then we end up with an $n \times n$ (or n^2) algorithm which is called quadratic.
- Common algorithm efficiency measures are logarithmic ($\log_2 n$), exponential (nn), cubic (n^3) and others. Here are two graphs comparing various algorithm efficiencies as the value of n grows (graphs shown at two different scales):
- Logarithmic, constant and linear algorithms are insignificant when compared to quadratic, cubic and exponential. Studies show that linear algorithm eventually passes constant algorithm as n gets larger. For very small values of n, the efficiency is negligible. This becomes an issue as n gets larger. Exponential algorithms for example, are usually unreasonable (i.e., useless) in practice except for very small values of n.
- Logarithmic solutions are often preferred since they are significantly more efficient than even linear algorithms. E.g. if n is 1,000,000 then a linear algorithm can take 1,000,000 steps whereas a logarithmic algorithm may take only 20-steps. Sometimes, it is hard to think in terms of an unknown number n since we are used to working with actual concrete numbers.

19

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Finally...On Efficiency

- Assuming 16-players got into a 1-on-1 tennis tournament. If there are no tie-games, how many games is played if each player is eliminated by 1-loss? We can figure this out with a simple table, doing a round-by-round count: and we have total 15 games played

Round Players Remaining No Played Round Players Remaining No Played

1 16 (nobody played yet) 8 1 $n/2$

2 8 (winners from round 1) 4 2 $n/2$ $n/4$

3 4 (winners from round 2) 2 3 ... 2

4 2 (winners from round 3) 1 4 2 1

- The first part was easy to figure. Suppose we have n -players, it will be given by the formula:

$(n/2 + n/4 + n/8 + n/16 + n/32 + \dots + 2 + 1)$. If we have say 128 players, then we have that:

$(128/2 + 128/4 + 128/8 + 128/16 + 128/32 + \dots) = (64 + 32 + 16 + 8 + 4 + 2 + 1) = 127$.

- As we try various values of n , we notice that the games played equals $(n-1)$. It works out evenly for games that are powers of 2 (e.g., $n=2, 4, 8, 16, 32, 64, 128, 256, 510, 1024$, etc).

In fact, the number of rounds played is $\log_2(n)$ (written as $\log(n)$ in computer science).

- Trying to figure out efficiency with an unknown number is not always easy. More importantly, you as a computer scientist will need to understand how to write efficient algorithms that run faster, takes lesser memory or use less resources. There are other courses ahead that will help you investigate algorithmic efficiency much more thoroughly such as Data structures...

20

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 01 – Computational Thinking:
an Introductory Perspective

These slides are based on textbooks and other materials accompanying this course

6/29/2017 1

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Organization

□ Course Lecturer(s)

- Dr. Arnold Adimabua OJUGO
- Lecture Venue/Time: TLH1 / Mondays (12noon – 2pm)
- Consultation Time: Mondays (2 – 3pm)
- David ENEBELI
- Lecture/Laboratory Time: TLH1 / Fridays (12noon – 1pm)
- Consultation Time: Tuesdays (2 – 3pm)

□ One Lecture/Week (2hour and 1hour recitation): Lectures may be recorded and will be available (on demand, if recorded). Also, Lecture slides are available on Piazza and the Unit Website (in time), which will be updated from time to time. Please, check Piazza for announcements and other information. Announcement made on Piazza is assumed that you are aware of it.

□ One Lab/Week (1-hour): It is ideal to have a Laptop/Desktop to accompany you through the course.

□ One Workshop/One Seminar per Semester (1 Hour)

6/29/2017 2

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Text & Other Materials

□ Python Programming (Materials)

- Introduction to Programming using Python by Daniel Y. Liang
- Learning to Program with python by Richard L. Halterman
- Head-First Programming: A Learner's guide to programming using the Python Language by Paul Barry & David Griffiths
- Python Programming by Mark Lutz

□ Videos Lectures (Available, on Demand):

- Lectures on Introduction to Computer Science and Programming in Python by Ana Bell
- Lectures on Introduction to Computer Science and Programming by John Guttag

□ Online Materials:

- Freely download other materials online to help you through this course
- We you are confused, ask questions for guidance on how to navigate this course.
- In the course of the lecture, be free to ask the instructor questions at any point; But, please, be sure the question(s) is interestingly intuitive enough that other students can learn as they participate.

□ Development Environment

- Python 2.5 / 3.3 or above with IDLE which you can download from python.org and install on your home computer (as we already have it installed in the Lab Computers)

6/29/2017 3

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Assessment

- Assessment is based on both: (a) Understanding Fundamental Concepts, (b) Practical Problem Solving and Programming Skills, (c) Presentation and (d) Question formulation / Answer Participation during the Lecture
- Mid-Semester Test(s) and Assignments due (Week 8 = 10%).
- Projects: Project 1 (Week 5 - 10%) and Project 2 (Week 10 = 10%)
- Final Exams...as scheduled by the University is worth (70%). It culminates every content of the course as taught.

Topics

- Understand what a Problem Is?
- Explore Problems through Problem Solving Heuristics and Strategies
- Design Creation and Representation
- Problem Data
- Solution Accuracy
- Define Re-Evaluation and Refinement
- Decompose Complex Problems
- Communicate Results
- Algorithm Efficiency
- Computationally Intensive Problems
- Unsolvable Problems for a Computer
- Computationally Hard Problems

6/29/2017 4

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Timeline To Course Completion

Description Topics in Weeks and Instructional Days

1 2 3 4 5 6 7 8 9 10 11 12 13

Introduction to Computational

Thinking and Strategies

Introduction to Problem Solving
and Data Collection

Introduction to Python

Reinforce Problem Solving steps
and Basic Programming with

Assignment and Accumulator

Graphs, Types of Graph,

Techniques of Resolving Graphs

and How Graphs can be Used to
help Solve Problems

Degree of Freedom, Average
Pathlength and Cluster Coeff
Agent-Based Modeling and SI-
models (SIS and SIR)

Introduce Linear and Binary
Search Algorithms
Sorting and Merging Algorithms
Final Projects

6/29/2017 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Let us Begin.....

6/29/2017 6

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Definitions

□ Computational Thinking – Computers are an incredible device: they

(a) help us extend what we can do with our brains, (b) help us to do things faster, (c) Keep track of a vast amount of data, and (d) they can help us share our ideas with other people.

□ By Definition

a. Computational Thinking is the process of getting computers to help us solve problems.

This is done via 2-means: (a) we think about the steps needed to solve a problem, and
(b) we use technical skills to get the computer working on the problems.

b. It describes the processes and approaches we draw when thinking about problems or systems in such a way that a computer can help.

□ What Computational Thinking Is and Is not!

Computer science is the study of computation, what can be computed and how to compute it. Computational thinking is not about computers – because computers do not think for themselves. It is about looking at a problem in a way that computers can help to solve it.

Computational thinking thus has the following characteristics:

□ Conceptualizing, not programming. Computer science is not programming. Thinking like a computer scientist is more than being able to program. It requires thinking at multiple levels of abstraction

7

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

What Computational thinking is and is not...Contd.

□ Fundamental, not rote skill. A fundamental skill is something you must know to function in a society. Rote means a mechanical routine. Ironically, not until computer science solves AI Grand Challenge of making computers think like humans will thinking be rote

□ Way humans, not computers, think. Computational thinking is how we solve problems; it is not trying to get humans to think like computers. Computers are dull and boring; humans are clever, creative and imaginative. Humans make computers exciting. Equipped with the computing devices, we use our creativeness to tackle problems we would not dare take on prior the computing era & build systems with functionality limited only by our imaginations

□ Completes, complements & combines mathematical & engineering thinking. CS inherently draws on mathematical thinking and modeling. Like all Sciences, its foundations completes mathematics. CS inherently draws on engineering thinking, as we build systems that interact with real world. The constraints of the underlying computing device forces computer scientists to think computationally, not mathematically. Being free to build virtual worlds enables us to engineer systems beyond the real world.

□ Ideas, not artifacts. It is not just the software & hardware artifacts we produce that will be physically present everywhere and touch our lives all the time, it will be the computational concepts we use to approach and solve problems, manage daily lives, and communicate and interact with other people.

□ For everyone, everywhere. Computational thinking will be a reality when it is so integral to human endeavors it disappears as an explicit philosophy

8

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Misconceptions

□ Misconception. Many people equate computer science with computer programming. Some parents see only a narrow range of job opportunities for their children who major in computer science. Many people think the fundamental research in computer science is done and that only the engineering remains. Computational thinking is a grand vision to guide computer science educators, researchers, and practitioners as we act to change society's image of the field. We especially need to reach the pre-college audience, including teachers, parents, and students, sending them two main messages:

- a. Intellectually challenging and engaging scientific problems remain to be understood and solved. The problem domain and solution domain are limited only by our own curiosity and creativity
- b. One can major in computer science and do anything. One can major in English or

mathematics and go on to a multitude of different careers. Ditto computer science. One can major in computer science and go on to a career in medicine, law, business, politics, any type of science or engineering, and even the arts.

□ Professors (and instructors) of CS (computer science) should teach a course called “Thinking Like a Computer Scientist via Problem Solving” to freshmen, making it available to non-majors, not just to computer science majors. We should expose pre-university students to computational methods and models. Rather than bemoan decline of interest in computer science or the decline in funding for research in computer science, we should look to inspire the public’s interest in the intellectual adventure of the field. We’ll thus spread the joy, awe, and power of computer science, aiming to make computational thinking commonplace.

9

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

so...thE Computational thinkEr

□ Computational Thinking involves concepts and approaches that helps us to achieve solutions to problems as we navigate the real world.

□ The Concepts include

- a. Logical Reasoning – involves predicting and analysing
- b. Algorithm – are the steps to achieve solution or having rules for getting things done
- c. Decomposition – breaking down the task into smaller, easily more manageable parts
- d. Patterns – spotting uniqueness and similarities & using unique and similar trends
- e. Abstraction – removing unnecessary details from the model or algorithm
- f. Evaluation – inferences and making judgment

□ Approaches includes

1. Tinkering – experimenting and playing around with plausible cum possible solutions
2. Creating – designing and making draft prototypes of the solution
3. Debugging – finding and fixing errors
4. Persevering – keeping track of solutions and going further
5. Collaborating – working together

10

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Computational Thinking: What we can do with It & its usage

□ Though Computational Thinking describes the sort of thinking that Computer Scientists and Software Developers engage in, plenty of other people think this

way also, and not just in using computers. The thinking processes and approaches that helps with computing are really useful in many other fields of study or domain.

□ For example, the way and how a team of software engineers go about creating a new computer game, video editor, or social networking platform is really not different from how you may work together with others to put on a school play, or organize an educational visit. In each case, you:

- a. Take a complex task and break it down into smaller problem parts
- b. Work out the steps or rules for getting things done
- c. The complexity of the task needs to be managed, typically by focusing on the key details
- d. The way previous projects have been accomplished may help you achieve your results faster

□ Its Adaptation and Adoption in Computer Curriculum

1. Ideals are built using logical reasoning, step-by-step approaches, decomposition, abstraction (and

generalization) etc – all these have wide applications to solving a problem and understanding the system across and beyond the school curriculum

2. Computational thinking is developed in schools beyond the computing curriculum such that as students learn to use these skills in their computing work – they become better in applying them to their work too (especially in writing codes and programs as solution to problems)

11

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Classroom Activity and Ideas

□ Write a recipe for preparing sandwich and Omelet – thinking carefully about each step that needs to be carried out. Share

□ The Concepts include

- a. Write the recipe for preparing Sandwich, Omelet, Stew and Salad
- b. Break the task down into smaller parts
- c. Capture the key elements in these recipes
- d. Share it with your class mates
- e. Spot the differences and similarities (patterns) in them
- f. Source various materials and use decomposition to identify the different components
- g. Create and read a range of recipes
- h. Capture and discuss the key elements (as layers of simplification, i.e. abstraction)
- i. Assemble the materials to a create the end-result
- j. Evaluate and test (to be sure it works)

□ In another Example:

1. You are challenged to work individually and collaboratively
2. Take notes and put them in your briefs

3. Write a report at the terms / semester's end

12

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Logical Reasoning

□ 2 computers set up in same manner, with same instructions and same input – will give same output. Computers do not make things up as they process. They are predictable. Thus, we use logical reasoning to work out exactly what a computer system or program will do. Children quickly form their characters from the things they see us do. These help them to form a mental model of how things work.

a. The process of using existing knowledge of a system to make reliable predictions about its future behaviour is logical reasoning. Logical reasoning is also being able to explain why something is the way it is and why something is not the way it should be.

b. We reason with arguments (i.e. statements that have true or false values). Arguments can be joined together (via disjunction, conjunction & negation operators) to form propositions.

□ Logical Reasoning as Used in Computing

a. As a fundamental concept of how CPUs work – computers perform all operations reducing them to logical operations carried out using electrical signals.

b. Everything a computer does is controlled by logic, in reasoning about program's behaviour

c. Software engineers use logical reasoning in all they do to draw the internal mental model of how a computer hardware (operating system, apps & programming languages) to develop new codes that will work how they intend. They also use logical reasoning when testing & to search for as well as fix errors in their thinking, or when their code testing fails.

13

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Classroom Activity and Ideas

□ Take Cellular network Provision and Telepenetration Project in Nigeria

a. Divide students into groups and give them states in Nigeria or take Delta State

b. Predict how and where will be connected next stating your initial entry connection point and the reasons why your choice.

c. What are the components to be considered when citing these connection in the said place of choice and where cell towers (BTS) should pass tru...

d. Explain why you think this should be the next phase

e. Write the psudocode to show where and which places the road should pass tru

f. Test the program...to see if it works

g. Write a report with a detailed map of the area where you are assigned of the road network and why you feel optimistic about it.

□ Bring a Chess Board

1. Enumerate the rules for the game (pawn, castle, knight, king and queen)
2. Use the logical reasoning to explain why the rules are as they are
3. Have two groups play against each other
4. Let each group pause the game at certain points and predict what is the opponent's next move
5. Write a report of all possible ways to start and end a game

14

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Algorithms

□ is a set of instructions or rules that gets something done. E.g. there are plenty of algorithms (routes) that will accomplish the same goal.

□ How Algorithms are Used in Computing

a. Search engines (Bing, Google, Yahoo etc) use algorithms to put search results in order, so that the results we seek is at the top of the page. Facebook and Instagram feed are results from your friend's status updates and other activity. It uses an algorithm EdgeRank to show recommended things, organized in a manner you are most likely to be most interested in.

b. Given the extent to which algorithms have affected our lives, the recipe thing we did earlier is also

an algorithm. When we solve mathematics problems using known formulas, we are actually creating an algorithm for solving the particular problem using a simple algorithm.

c. It helps us: (a) grasp how easy it is to solve problems, (b) design programs with particular goals in

mind that hinges on our being able to think algorithmically and use logic reasoning to explain the algorithms, and (c) detect/correct errors in programs via practice. Learn to follow the instructions in your code yourself; Rather than just writing them for the computer. Errors and inconsistencies should become apparent

□ Talk to the students about:

1. What makes one algorithm better than the other
2. Write a recipe for omelet
3. Take the guessing game – students can guess a number I am thinking. Students can ask any question as to detect what the number is, and the lecturer can only answer “Yes” or “No”. They can only ask one question per student.

15

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Decomposition

□ Breaks down large projects into smaller, easily managed parts. Helps us to solve complex problems. Its merit is that: (a) makes the process more manageable, (b) more achievable, (c) means a team can work on a project together and at same time – each bringing their own insight, experience and skills to the task

□ How Decomposition is Used in Real World Computing

a. Decomposing is not unique to only computing. It is a standard in engineering, design and project management. Thus, because software development is a complex process, and so being able to break large projects into smaller components is essential. Think of all the different elements that need to be combined to produce a program like facebook...

b. Same is true for hardware, smartphones etc. These are often produce independently by specialists manufacturers and assembled to make the finished product – each under the control of the operating system and application.

□ How is Decomposition used in Curriculum: Students should design a system with particular goal in mind (a system is something with a number of interconnected parts).

□ Classwork: Have students work in groups to build social networking mobile app (using panning, design, algorithm, coding, animation, graphics, sound, debugging and sharing) aimed at collaborative work with development planned over weeks.

□ A Drone project

16

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Abstraction

□ Helps us to decide what details we need to highlight and those we need to ignore in computational thinking. It is about simplifying things, identifying the important ones without worrying too much about its details. It enables us to manage complexity. Example, a school timetable is an abstraction of what happens in a typical week as it captures key info such as who is taught, what course is taught and by which instructor. It leaves out further layers of complexity such as learning objectives and activities planned in any individual lecture.

□ How Decomposition is Used in Real World Computing Curriculum

a. It seeks to ensure that all students can understand and apply the fundamental principles and concepts of computer science including abstraction, logic, algorithms and data representation.

- b. It encourages and engages students' curiosity about how things work, helping them to think about what happens inside any given system etc
- c. Helps them to focus on the key information of a system

□ Classwork

□ Write a simulation game for table tennis to include the ball's motion in 2D and how it bounces off the bat; while ignoring factors like air resistance, spin, and gravity. What details do you need to include, and what can we leave out in programming such a simulation. 17
6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Patterns

□ Seeking a general approach to solve a class of problems is called Generalization.

By identifying patterns that exists in a solution, we can predict, create rules and solve more general problems. E.g. In chess, opening a game in a certain way can help predict how likely the game will end and in who's favor.

□ How Decomposition is Used in Real World

a. Students will become familiar of repeated phrases in a narrative structure or story. Later, they notice a certain way a traditional tale ends or how other genres come to an end. In mathematics, they will undertake investigations in which they spot patterns in solving the problem and deduce generalized results. For example, a student curve for result in a certain course that is well taken is meant to show a bell (Z-score) shape....explain

- b. Helps them to focus on the key information of a system

□ Classwork

□ Students are encouraged to look for simpler/quicker ways to solve a problem, or achieve results.

□ Take the Tower of Hanoi, 3 utility and 3 House problem, 3 cannibals and 3 priests etc

18

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Putting it together via Evaluation

□ Tinkering is willingness to experiment and explore.

a. Some elements of learning a new programming language or exploring a new system look quite similar to a purposeful play or an effective approach to learning.

b. Open source makes it easy to take someone's code, look at how it was written and adapt it as a basis for their own creative coding and expression of their particular project.

c. This course encourage students to play with codes and apps, sharing what they discover about it; Rather than explaining exactly how it works. Also, I would look for ways in which

students can use codes from you, their peers, or online as a starting point for their own programming projects.

□ Creating programs is quite a creative process that involves both originality and making something of value: typically something useful or at least fits the purpose intended.

a. Course encourage students to approach tasks with creative spirit. Seek programming tasks that allow some scope for creative expression rather than merely arriving at the answer.

b. It encourage students to reflect on the quality of the work they produce, critiquing their own and others' projects. The process of always looking for ways to improve on a software project is becoming common practice in software development.

19

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Putting it together via Evaluation...

□ Debugging

a. Due to its complexity, codes often do not work as intended. Taking responsibility for thinking through your algorithms and code, to identify and fix errors is an important part in learning to think and work, like a programmer.

b. It also encourages collaboration and makes us better as we debug each other's code (and proofread each other's work), looking for mistakes and suggesting improvements. Studies have shown that learning from mistake is an effective approach to learn, and the process of debugging each others' code is one way to do this. Keep an eye on the bugs that your see, these can sometimes reveal particular misconceptions that may need to address

□ Persevering – Programming is quite hard and that is a part of its appeal.

a. Writing elegant and effective code is an intellectual challenge requiring an understanding of the ideas of the algorithms being coded, great knowledge about the programming language you are working with, and a willingness to persevere with difficult and often frustrating situation.

b. Growth of the mind suggests that hardwork and a willingness to persevere in the face of difficulties is a key factors in educational outcomes. Students are encouraged to look for strategies they can use when they do encounter difficulties with their programming work (e.g. working out exactly what the problem is, searching for the solution via Google, or simply asking a friend for help).

20

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Putting it together via Evaluation...

□ Collaboration

a. Software is developed by teams of programmers and others working together on a shared project. Look for ways to provide pupils with this experience in computing lessons too.

Collaborative group work has long had a place in primary education, and computing should be no different.

b. 'Pair programming' is an effective development method, with two programmers sharing a screen and a keyboard, working together to write an app. Typically, one programmer acts as the driver, dealing with the detail of the programming; the other takes on a navigator role, looking at the bigger picture. The two programmers regularly swap roles, so both have a grasp of both detail and big picture.

c. Working in a larger group develops can also help us gain a number of additional skills, with each student contributing some of their own particular talents to a shared project. But, we must remember that all students should develop their understanding of each part of the process. So, some must share the role of peer-tutoring and this must be incorporated into such activities.

□ Final Project

a. Each group (and pair) to develop an app...

b. Swap roles within pairs, and within groups

c. Write a report and submit by the end of the Semester...

21

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Final Project

Keep intact the same groups:

□ Develop a mobile App

a. Each group (and pair) to develop a mobile app...

b. Swap roles within pairs and within groups

c. Write a report and submit by the end of the semester

□ Election Projection

a. Take the areas in Page 14 of this slide

b. Select a member of your group whom you intend to field as a candidate to win an election in the year 2019

c. After the topic – Graph, decide how you can spread the fame and popularity of your desired candidate knowing that in areas, there are godfathers, youth unrest etc

d. Using the computational thinking methodology – develop a model that will help you to field and ensure the success of your desired candidate.

e. Write a report detailing your strategy and how you intend to accomplish this.

f. Report is due last week of lecture.

22

6/29/2017

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 05 – Ciphers

These slides are based on textbooks and other materials accompanying this course

6/29/2017 1

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Let us Begin.....

6/29/2017 2

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

cryptography

□ Cryptography is the art and science of codes/cipher, allowing users exchange data in such a way that eavesdroppers cannot understand. It uses data altering schemes allowing only an intended recipient to discover the original text sent. An original text is plaintext. Altered text is a ciphertext. The conversion is called encoding. Its reversion is called decoding.

□ An eavesdropper that tries to undo the change without knowledge of scheme used is a cryptanalyst – and the process of undoing alterations is cracking.

□ Cryptography is a secure means to transfer data over insecure channel like Internet. Elements used in these processes that make it impossible for a cryptanalyst to deduce data is called a cryptosystem. It encrypts data via an encryption key fed into an encryption algorithm. On arrival at its destination, the recipient passes the data via a decryption algorithm using a corresponding decryption key to access to the transmitted data

□ The goals of cryptography is to achieve: data confidentiality, privacy, integrity, authentication and non-repudiation. Thus, it seeks to achieve data security through: (a) security levels implemented by user, (b) functionality of the cryptosystem used, (c) its operations and tools, (d) its performance, (e) its key space size, and lastly, (f) its mode of implementation. Fig of a cryptosystem is as below in fig 1.

Encryption Decryption

Sender Recipient

Algorithm Ciphertext Algorithm Plaintext

Plaintext

Encryption Decryption

6/29/2017 Key Key 3

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Types of Ciphers

□ The importance of the criteria mentioned above is dependent on its application and on the resources available – such that in some cases, security is tradeoff for better performance or memory.

□ When we convert and reconvert data from PT to CT, there are two primitive types of ciphers or codes that can be generated namely: (a) keyed and (b) unkeyed ciphers. These are further stratified into the following

6/29/2017 4

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Substitution Ciphers

□ In a substitution cipher, one character is substituted for another. Thus, it takes a key system E – such that alphabets on the upper divide/line is the encryption key; while, those on the lower part of the divide/line is called the decryption key.

□ For example, we wish to encrypt the word “FORECAST” using the key below

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

R Z B U Q K F C P Y E V L S N G W O X D J I A H T M

□ If we encrypt or encode the word FORECAST, it will yield KNOQBRXD.

□ To decode the words back, we simply reverse the process by taking the “K” on the lower divide/line and finding its corresponding matching letter above to get “F” – which is also the first letter in the word FORECAST....and so on and so forth.

□ Substitution ciphers are easier to crack in English.

a. Certain words in English appear more. Thus, they have the following shows the frequency of some letters are: E = 12.7%, T = 9.1%, A = 8.2%, O = 7.5%, J = 0.2%, Z = 0.1%. The frequency is thus: E, T, A, O, I, N, S, H, R, D, L, U, C, M, F, G, Y, P, B, V, K, X, J, Q and Z.

b. Punctuations and separators can be encoded with alternative symbols

c. User can have codes for letter pairs and common words that helps make the task still more difficult

6/29/2017 5

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Example

- Cryptanalyze the text below and discover the words behind it.
- Generate the key

NOT NUA JMPETZ UTST ZENNELV EL NOT JEGELV SAAW, UMENELV KAS NOTES
 OAZNTZZ, UOA UMZ ZJEVONJX PTJMXTP. NOT PMCVONTS AK NOT KMWEJX UMZ UENO
 NOTW, AL NOT NOTASX NOMN ZOT UACJP QTTY NOT GEZENASZ ADDCYETP PCSELV
 NOT UMEN. NOT DOEJP UMZ YTSOMYZ ZEI XTMSZ AJP, ZLCR LAZTP, RCDQ NAANOTP MLP
 RTZYTNDMDJTP. ZOT WMELNMELTP M PTTY ZEJTLDT MLP NOT NUA JMPETZ YTTSTP
 PACRNKCJXX MN OTS. KELMJXX, ALT AK NOTW WCNNTSTP NA NOT ANOTS,
 "LAN GTSX Y-S-T-N-N-X, E KTMS," DMSTKCJXX ZYTJJELV NOT QTX UASP. UOTSTCYAL NOT
 DOEJP YEYTP CY, "RCN MUKCJ Z-W-M-S-N!"

□ Solution: First, we take sample frequency of the characters in the passage. It is as shown in the table below.

□ To cryptanalyze, we start by guessing that: (1) Letter E and T is represented by either "T" or "N" – as they are two most frequently occurring letters and (2) O is either A or H. With the substitutions, we have the text below:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

25 0 14 10 28 0 3 0 1 23 10 18 25 44 31 24 4 5 22 60 14 7 7 11 13 23

6/29/2017 6

FEDERAL UNIVERSITY OF
 PETROLEUM RESOURCES
 EFFURUN

- Having guessed these

EAT E T TT EE EAT E EAT

THE T E E E TT THE T THE

*NOT NUA JMPETZ UTST ZENNELV EL NOT JEGELV SAAW, UMENELV KAS NOTES

A ET A E T T. EAT ET EAT EA

H TE H T E E . THE TE THE TH

OAZNTZZ, UOA UMZ ZJEVONJX PTJMXTP. NOT PMCVONTS AK NOT KMWEJX UMZ UENO

EAT EAT EAT EA AT TT EAT E T

THE THE THE TH HE EE THE T E

NOTW, AL NOT NOTASX NOMN ZOT UACJP QTTY NOT GEZENASZ ADDCYETP PCSELV

EAT E. EAT TT T E EA

THE T. THE EE E T TH

NOT UMEN. NOT DOEJP UMZ YTSOMYZ ZEI XTMSZ AJP, ZLCR LAZTP, RCDQ NAANOTP MLP

T T E T AT E T TT T EAT E T TT T

E E T E HE T E EE E THE T E EE E

RTZYTNDMDJTP. ZOT WMELNMELTP M PTTY ZEJTLDT MLP NOT NUA JMPETZ YTTSTP

E E AT T EAT EET T E EAT EAT

T T HE E THE TTE E T THE THE

PACRNKCJXX MN OTS. KELMJXX, ALT AK NOTW WCNNTSTP NA NOT ANOTS,

" E T -T-E-E- T T T EAT T AT T EAT

" T E -E-T-T- E E E THE E HE E THE

"LAN GTSX Y-S-T-N-N-X, E KTMS," DMSTKCJXX ZYTJJELV NOT QTX UASP. UOTSTCYAL NOT

6/29/2017 7

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Phase I:

□ Which of this are we likely to have start a sentence, THE or EAT.

Phase II:

1. Line (5) – the 2-letters in English that can stand-alone are – the letters A and I; and A appears more in

English sentence than I. So, we take it that M is A

2. Line (6) (now that THE(PT) is NOT(CT)) – 2-words that start with E are EX or EN or that starts with T are TO. TO appears more in sentence

3. Line (6) – if THE(PT) is NOT(CT) and A(CT) is O(PT) – then ANOTS(CT) from line 6 points to translate as OTHE_(PT) – so that the missing letter seems to be R. Thus, S(CT) is R(PT).

Phase III:

1. Line (1), _ERE(PT) is missing W. So, U(CT) is W(PT). Also, ROO_(PT) is missing M. So W(CT) is M(PT).

2. Line (3) – a 2-word starting with O is ON or OF.

3. Line (4) – the word AN_(PT) is missing D. Replace P(CT) with D(PT), and L(CT) with P(PT).

Phase IV:

1. Line (1) – replace V(CT) with G(PT) and J(PT) with L(PT).

2. Line (2) – replace: (a) X(CT) with Y(PT), (b) C(CT) with U(PT) and (c) K(CT) with F(PT)

3. Line (3) – replace: (a) G(CT) with V(PT), and (b) Q(CT) with K(PT)

4. Line (4) – replace: (a) I(CT) with X, and (b) D(CT) with C(PT)

Phase V: Replace R(CT) with the letter B(PT) – to give our final yield

6/29/2017 8

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Permutation Ciphers RC 1 2 3 4 5

□ In a permutation cipher, we break up the words into equal 1 A B C D E letter chunks, and adding letters where necessary to extend 2 F G H I J and make equal the last chunk so that each chunk encodes

3 L M N O P

itself.

4 Q R S T U

□ After this, one character is substituted for another. Thus, it

5 V W X Y Z

takes a key system E – such that alphabets on the upper divide/line is the encryption key; while, those on the lower part of the divide/line is called the decryption key.

□ For example, we can use the table to encrypt the word (in this case, data from a smartphone) “444211243343441144243433” using a permutation cipher – what do we generate...

□ We can substitute 44 for T, 42 for R, 11 for A, 24 for I, 33 for N, 34 for S, 44 for T, 11 for A, 44 for T, 11 for A, 24 for I, 34 for O and lastly 33 for N.

□ We see that it generates a single phrase “TRAINSTATION”.

□ A look at this informs us that there is no such one word. So it has to be either a 2-, 3 or 4-letter word as the case may be. Since there are no spaces...let us try to tinker and find which word this best suggests...

6/29/2017 9

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Other Ciphers

There are other kinds of ciphers namely:

Block Ciphers:

1. Vigen'ere Ciphers – were we have the alphabet A-Z into A-Z like the Hex-to-Dec table such that each row or column translates into a different cipher. The great merit of this cipher is that any ciphertext can translate into any plaintext – leading to a trapdoors. A trapdoor is when we encode a plaintext such that it cannot be decode – in some case, even by the person who encoded it (unless they go back to the original text)
2. Converting Text to Numbers resulting in: (a) ASCII, (b) Decimal Alphanumerics and (c) Quasi-Pseudo Random Numbers

Public Key Ciphers:

1. RSA ciphers

6/29/2017 10

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 05a – Data objects

These slides are based on textbooks and other materials accompanying this course

6/18/2017 1

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Objective of this Lecture

□ Revision:

- What is the study of computer science? What is a computer?
- Name and describe the parts of a computer?
- What is the role of these parts in a computing system?
- Computer Scientists find answers to questions true : a. ____, b. ____ and c. ____
- Describe the terms: (a) design, (b) analysis, (c) experimentation:
- Name the software development process: A - S - D - I - T/D - M
- Control Strategies used in Programs: 2-way and Multi-way decision systems
- How many Data Types do you know in Python
- How are numbers represented in a computer
- What are libraries, name some and describe what they do?
- How do we use libraries in a Python Program
- Describe the accumulator program and its pattern
- Objective of this Lecture: To understand -
- Objects: Definition, Concept and How Objects simplify programs
- Fundamentals of Computer Graphics, Objects and the graphic library
- Role of coordinate systems and transformation
- Create objects in programs and call appropriate methods to perform graphic tasks
- Write simple interactive graphics program

6/18/2017 2

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Welcome Back...

6/18/2017 3

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Objects - concept & Framework

Traditional programming had always viewed data as passive - an item manipulated and combined with active operations. But, modern programming languages are built with data as king using the object-oriented design approach. What this implies is that objects have the following properties:

1. Objects (codes and data) are packed together with its implementation and internal working hidden

from the user. This is called encapsulation. Most us drive without necessarily knowing how each component part interacts with each other.

2. Objects are grouped into classes. Objects of the same class have similar behaviour. We can build new classes from codes and data declared by old classes or other new class. Thus, feat common to the classes are expressed in its base class. This is called inheritance.

3. Anytime we declare an objects using a certain data type – an instance of that object class is created – though each object has its identity, which depends both on the values it contains, its location in memory and other feats. This is called Polymorphism.

4. Objects know things (contains data) and they can do things (operations local to the data type they

are declared as) because whenever an object is created (declared) as a data type, it is assigned a value immediately based on its data type – even if the value is 0. This is called data binding.

5. Objects interact with each other by sending messages to and amongst themselves – because an object is an active data type that combines and operations. So we know that complex systems are built as an interaction of simpler objects.

6. All objects have properties, events (things that can happen to the object) and methods (actions that the object can perform) – specific to it and its class set.

6/18/2017 4

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Object of Objects with Examples

□ In designing a University data processing system to keep records of students who attend the school, each student represents as an object. The student object may contain: Name, MatNo, Courses taken, Campus Address, Home Address, Grades etc.

□ This student object is meant to do the following: (a) It should respond to requests, (b) It can allow us send out a campus-wide mailing, achieved with the campus address, (c) It should allow us send the printCampusAddress (object variable) to each student object so that when it receives the message – it prints its own address.

□ Objects interact with each other by sending message amongst desired objects. Each course is represented by an object such as: Instructor, Student roaster, c. Prerequisites, d. When and where the class meets etc

□ Some operations of course object can include: add Student, delStudent, changeRoom etc

□ Objects can have one or more objects inside it. The course object will have student objects and instructor object inside it. This will be explained later as we progress with many examples from time to time – so that you can simplify complex problems into interactions between different objects.

6/18/2017 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Object of Objects

□ Objects can have one or more objects inside it. E.g. the course object will have student objects inside it. Similarly, the course object can also have the instructor object inside it. This will be explained later as we progress with many examples from time to time – so that you can simplify complex problems into interactions between different objects.

□ This student object is meant to do the following:

- a. It should respond to requests
- b. It can allow us send out a campus-wide mailing, achieved with the campus address
- c. It should allow us send the printCampusAddress (object variable) to each student object so that when it receives the message – it prints its own address.

□ Sample operation of the “Course” Object can include

- a. add Student
- b. delStudent
- c. changeRoom

Detailed look at objects – before resuming with the University Data processing.

6/18/2017 6

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Simple Graphic Programming

□ Python’s graphic capabilities via Tkinter i.e. your text box comes with graphics.py library. At installation, we put it In python’s library directory with other libraries; Or, place them in the same folder as your graphics program. You call or invoke it using the command line:

```
>>> import graphics
```

□ A graphics window is a place on screen where graphics will appear.

```
>>> win = graphics.GraphWin()
```

□ The command creates a new window titled “Graphics Window”. GraphWin is an object assigned to the variable win. We manipulate the window object via win. Windows can be closed/destroyed via the command:

```
>>> win.close()
```

□ It is tedious to use graphics. notation to access the graphics library routines. We can import specific functions from a library as below. The from statement allows you to load specific functions from a library module; while, the ‘*’ symbol allows us to load all the functions – or we can list the specific ones we wish to load

```
>>> from graphics import *
```

□ The command line below loads just the sqrt function from the math library:

```
>>> from math import sqrt
```

```
>>> sqrt(5)
```

```
6/18/2017 7
```

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Importing functions: Graphics Windows

□ If we import a function directly, we can use it directly also. This eliminates the need to use or write the graphics. notation in our command line. So we can rewrite the command line as below

```
>>> from graphics import *
```

```
>>> win = GraphWin()
```

□ A graphics windows is a collection of points called pixels (the rate at which picture elements are measured). The default GraphWin is 200 pixels tall by 200 pixels wide (i.e. 40,000 pixels in total).

□ One way to get pictures into the window is one pixel at a time, which would also be very tedious. The graphics library has a number of predefined routines to draw geometric shapes.

□ The simplest object is a point whose location is represented via coordinate (x,y) as in geometry. The origin (0,0) in a graphics window is the upper left corner. X values increases from left to right; While, y values from top to bottom. The lower right corner is (199,199)

```
>>> from graphics import *
```

```
6/18/2017 8
```

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

```
>>> p = Point (50,60)
```

Simple graphics window commands

```
>>> p.getX()
```

```
>>> p.getY()
```

```
>>>win = GraphWin()
```

```
>>> p.draw(win)
```

```
>>> p2 = Point(140,100)
```

```
>>> p2.draw(win)
```

We can draw simple geometric shapes

```
>>> win = GraphWin('Shapes') ### Open a graphics window
```

```
>>> center = Point(100,100) ### Draw red circle centered at (100,100) with radius 30
```

```

>>> circ = Circle(center, 30)
>>> circ.setFill('red')
>>> circ.draw(win)
>>> ### Put a textual label in the center of the circle
>>> label = Text(center, "Red Circle")
>>> label.draw(win)
>>> ### Draw a square using a Rectangle object
>>> rect = Rectangle(Point(30, 30), Point(70, 70))
>>> rect.draw(win)
>>> ### Draw a line segment using a Line object
>>> line = Line(Point(20, 30), Point(180, 165))
>>> line.draw(win)
>>> ### Draw an oval using the Oval object
>>> oval = Oval(Point(20, 150), Point(180, 199))
>>> oval.draw(win)

```

6/18/2017 9

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Using Graphics Objects

□ Computation is performed by asking an object to carry out one of its operations. In the previous example we manipulated GraphWin, Point, Circle, Oval, Line, Text and Rectangle. These are examples of classes.

□ Each object is an instance of some class. The class describes the properties of the instance. If we say Barney is a dinosaur is a dog. We imply by this that Barney is a specific individual in the larger class of dinosaurs. Thus, Barney is an instance of the dinosaur class. It has all the feats of the class dinosaur and can also have its own feats that makes it unique, giving it its own identity.

□ To create a new instance of a class, we use a special operation called a constructor: `<class-name>(<param1>, <param2>, ...)`. A `<class-name>` is the name of the class we want to create a new instance of, e.g. Circle or Point. The parameters are required to initialize the object. For example, Point requires two numeric values.

```
p = Point (50, 60)
```

□ The command line above is a constructor for the Point class. It requires two parameters, x and y coordinates for the point. These values are stored as instance variables inside of the object. We can reiterate these concepts below:

□ Class: Think of the class as a “template” or a “blueprint” used to create objects.

□ Instance: A unique copy of a Class representing an object.

□ Object: An Object is an Instance of a Class. It knows stuff and can do stuff.

6/18/2017 10

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Constructor

□ Constructors are a special operation that helps us create a new instance of a class using a special operation called constructor.

`<class-name>(<param1>, <param2>, ...)`

□ `<class-name>` is the name of the class/object we wish to create an instance of. The different class constructors require different types and number of parameters. A class can have many constructors having different type and number of parameters such as `p = Point(20,20)`

□ To perform an operation on an object, we send the object a message. The set of messages an object responds to are called the methods of the object. Methods are functions that live inside the object. They are invoked using dot-notation after object's name such as:

`<object>.<method-name>(<param1>,<param2>, ...)`

□ The methods `p.getX()` and `p.getY()` returns both the x and y values of a point. Such routines are called accessors since they let us access data from the instance variables of the object, which may not be accessible without accessors.

□ Other methods change the state of the object by changing the values of the object's instance variables. The method `move(dx, dy)` moves the object dx units in the x direction and dy in the y direction. The method `Move` erases the old image drawn and redraws it in its new position. Methods that change the state of an object are called mutators.

6/18/2017 11

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Constructing Graphics Object Instances

```
>>> circ = Circle(Point(100,100), 30)
```

```
>>> win = GraphWin()
```

```
circ.draw(win)
```

□ First line draws a circle with radius 30 centered at (100,100). We used the `Point`'s constructor to create a location for the center of the circle and passed it to `Circle`'s constructor. The last line is a request to the `Circle` object `circ` to draw itself into the `GraphWin` object `win`. The `draw` method uses data about the center and radius of the circle from the instance variable.

6/18/2017 12

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Multiple variables: same object

□ It is possible for two different variables to refer to the same object – changes made to the object through one variable will be visible to the other. The idea is to create the left eye and copy that to the right eye which gets moved 20 units. Note that there is one object only. This will not draw two eyes.

```
>>> leftEye = Circle(Point(80,50), 5)
```

```
>>> leftEye.setFill('yellow')
```

```
>>> leftEye.setOutline('red')
```

```
>>> rightEye = leftEye
```

```
>>> rightEye.move(20,0)
```

□ The assignment `rightEye = leftEye` makes `rightEye` and `leftEye` refer to the same circle!

Copying objects is not the same as copying variables where physical copy is created.

The state where two variables refer to the same object is called aliasing.

6/18/2017 13

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Multiple variables: same object

□ Declaring different instances of same class – implies we make multiple objects of same class. There are 2-methods to do this namely:

□ Method 1: Make separate circles, one for each eye:

```
>>> leftEye = Circle(Point(80, 50), 5)
```

```
>>> leftEye.setFill('yellow')
```

```
>>> leftEye.setOutline('red')
```

```
>>> rightEye = Circle(Point(100, 50), 5)
```

```
>>> rightEye.setFill('yellow')
```

```
>>> rightEye.setOutline('red')
```

□ Method 2: Use cloning – objects can have a clone method which makes a copy of the object. E.g. Graphics has a cloning method for its objects as:

```
>>> leftEye = Circle(Point(80, 50), 5)
```

```
>>> leftEye.setFill('yellow')
```

```
>>> leftEye.setOutline('red')
```

```
>>> rightEye = leftEye.clone()
```

`rightEye` is new instance of the `Circle` class and is an exact copy of the left

```
>>> rightEye.move(20, 0)
```

6/18/2017 14

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Graphic Output: example of future Value

□ Recall the future value function in Chapter 2 – if we want to output the future value graphically, we can do so as thus:

1. Graphic programming requires careful planning using pencil
2. Figure out what the program needs to do and what it will do.
3. Choose your: (a) coordinates, and (b) scale
4. Determine the location you wish the graphic to start
5. Method win.setCoords can help set your coordinates

differently as thus:

```
>>> win = GraphWin("window name", 500, 400)
```

```
>>> win.setCoords(x1, y1, x2, y2)
```

//x1,y1 are the lower left corner coordinates

//x2,y2 are the upper right corner coordinates

6/18/2017 15

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Interactive Graphics

□ In a GUI environ, users typically interact with applications by clicking on buttons, choosing items from menus, and typing data via on-screen text boxes – known as Event-driven programming. It draws interface elements (widgets) on the screen and then waits for the user to do something. An event is generated whenever a user moves the mouse, clicks the mouse, or types a key on the keyboard. An event is an object that encapsulates information about what just happened! The event object is sent to the appropriate part of the program to be processed, for example, a button event.

□ Graphics.py module hides its underlying, low-level window management and provides two simple ways to get user input in a GraphWin: (a) mouse click, and (b) textual.

□ Method 1: Mouse Click – Here, we can get graphical information from the user via the getMouse method of the GraphWin class. When getMouse is invoked on a GraphWin, the program pauses and waits for the user to click the mouse somewhere in the window. The spot where the user clicked is returned as a Point.

The code below reports the coordinates of a mouse click:

```
from graphics import *
```

```
win = GraphWin("Click Me!")
```

```
p = win.getMouse()
```

```
print("You clicked", p.getX(), p.getY())
```

In our example, the method win.getMouse() returns an object of type Point from the window “win”.

We can use the accessors like getX and getY or other methods on the point returned.

6/18/2017 16

```

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN
# triangle.pyw: // An Interactive graphics program to draw a triangle
from graphics import *
def main():
    win = GraphWin("Draw a Triangle",400,400)
    win.setCoords(0.0, 0.0, 10.0, 10.0)
    message = Text(Point(5, 0.5), "Click on three points")
    message.draw(win)
    # Get and draw three vertices of triangle
    p1 = win.getMouse()
    p1.draw(win)
    p2 = win.getMouse()
    p2.draw(win)
    p3 = win.getMouse()
    p3.draw(win)
    l1 = Line(p1,p2)
    l1.draw(win)
    l2 = Line(p2,p3)
    l2.draw(win)
    l3 = Line(p3,p1)
    l3.draw(win)
    # You may use Polygon object to draw the triangle
    triangle = Polygon(p1,p2,p3)
    triangle.setFill("peachpuff")
    triangle.setOutline("cyan")
    triangle.draw(win)
    # Wait for another click to exit
    message.setText("Click anywhere to quit.")
    win.getMouse()
    main()
6/18/2017 17

```

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Working with Textual Inputs

□ You can run a Python code by double-clicking on the .py file. This will also open up a command window in addition to any graphics window that you are creating. But, if you save your file as a .pyw extension, the command windows does not appear; Rather, only the graphics window will appear.

□ In the code, we used the polygon class as there is no triangle class. This class takes a number of points and connects them into a closed shape.

We can create a triangle polygon once we have 3-points:

```
triangle = Polygon(p1,p2,p3)
```

□ A single text object is created and drawn near the start of the program:

```
message = Text(Point(5,0.5), "Click on three points")
```

```
message.draw(win)
```

□ Thus, to change the prompt, just change the text to be displayed.

```
message.setText("Click anywhere to quit.")
```

□ Our triangle program's input was done via mouse clicks. We can also get input from users using an Entry object that gets keyboard input. It draws a box on-screen that can contain text. It has methods setText and getText to acquire input with one difference (i.e. that the input can be edited).

6/18/2017 18

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

convert_gui.pyw // Program to convert Celsius to Fahrenheit using a simple GUI

```
from graphics import *
```

```
def main():
```

```
win = GraphWin("Celsius Converter", 300, 200)
```

```
win.setCoords(0.0, 0.0, 3.0, 4.0)
```

```
# Draw the interface
```

```
Text(Point(1,3), " Celsius Temperature:").draw(win)
```

```
Text(Point(1,1), "Fahrenheit Temperature:").draw(win)
```

```
input = Entry(Point(2,3), 5)
```

```
input.setText("0.0")
```

```
input.draw(win)
```

```
output = Text(Point(2,1), "")
```

```
output.draw(win)
```

```
button = Text(Point(1.5,2.0), "Convert It")
```

```
button.draw(win)
```

```
Rectangle(Point(1,1.5), Point(2,2.5)).draw(win)
```

```
# wait for a mouse click and convert input
```

```
win.getMouse()
```

```
celsius = eval(input.getText())
```

```
fahrenheit = 9.0/5.0 * celsius + 32
```

```
# display output and change button
```

```
output.setText(fahrenheit)
```

```
button.setText("Quit")
```

```
# wait for click and then quit
```

```
win.getMouse()  
win.close()  
main()
```

6/18/2017 19

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Its Workings: Input, Processing & Output

- When run, this program produces a window with an entry box for typing in the Celsius temperature and a button to “do” the conversion.
- The button is for show only! We are just waiting for a mouse click anywhere in the window.
- Initially, the input entry box is set to contain “0.0”. The user can delete this value and type in another value. The program pauses until the user clicks the mouse – we don’t care where so we don’t store the point!
- INPUT: The GUI basically invites the user to put in a value and press the button.
- PROCESSING: Done in three steps: (a) The value entered is converted into a number with eval, (b) This number is converted to degrees Fahrenheit, (c) This number is then converted to a string.
- OUTPUT: The final string value is formatted for display in the output text area.

In Summary:

- We learned how to construct objects and how to use their methods.
- We learned how to use the graphics.py library.
- We learned the concepts of coordinate systems and transformations.
- We learned interactive graphics programming using mouse clicks and text-based inputs.

6/18/2017 20

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 03A – introduction to Python

These slides are based on textbooks and other materials accompanying this course

6/18/2017 1

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES

EFFURUN

Let us Begin.....

6/18/2017 2

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Basic Programming

□ The Fetch-Execute Cycle

1. Loads programs into the main memory (R/WM)
2. Fetches the next instruction from memory
3. Decode the next instruction to see what it represents
4. Carry out the appropriate instructions

□ Programming Languages are used to describe algorithms in a precise way that Natural Languages cannot.

- Every structure in a programming language has a precise form called syntax.
- Every structure in a programming language has a precise meaning called its semantics.
- Programming languages is like a code for writing instructions that computers can understand, follow and execute. Thus, they are often or sometimes called computer codes.
- Process of writing an algorithm in a programming language is called coding.

□ There are 2-types of Programming Languages: (a) low-level that is written as mnemonics and understood only by the hardware as machine or target codes, and (b) High-level language – are designed to be understood and written by human using English-like statements.

6/18/2017 3

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Basic Programming

□ High-level Language: In this form, addition of 2-numbers can be addressed as:

$C = a + b;$

□ This instruction needs to be translated to machine language so that the computer can execute it. To achieve this, the computer has 3-types of translators namely:

1. Compilers convert programs from a source language into its equivalent target machine code
2. Interpreters simulate a computer that understands high level codes. The source code is not translated (compiled) into machine code all at once. All interpreters analyze and executes the source code instructions-by-instruction.
3. Assemblers

□ Low-Level code of the statement as above “ $c = a + b$ ” corresponds to the statements below. In reality, these low level instructions are represented in binary (0s and 1s)

- Load the number from memory location 1001 into the CPU
- Load the number from memory location 1002 into the CPU
- Add the two numbers in the CPU
- Store the result into the location 1003

6/18/2017 4

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Compiler vs Interpret

□ Once program is compiled, it can □ The source code and interpreter be executed without the source are needed each time the program code or compiler. is to be executed.

□ Compiled programs run faster □ Interpreted programs run slower since their translation happens only due to the interpretation each time once. it is executed.

□ Program needs to be compiled □ More flexible programming after every minor change in the environment since the programs codes. can be developed and run interactively

□ A compiled program will not run across the different OS platforms □ Interpreted programs are more such as Windows, Mac, Linux etc. portable across different OS Each will be configured to run on platforms such as Windows, Mac, each platform differently. Linux etc

6/18/2017 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Python

□ Python is an interpreted language.

□ If there is a suitable interpreter exists for 2-different machines with very different CPUs, the same program will run on both.

□ When you start Python, you may see something as below:

Python 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22) [GCC 4.2.1 (Apple Inc. build 5666) (dot 3)]

Type "copyright", "credits" or "license()" for more information.

- “>>>” is Python prompt. It tells a user that Python is ready for your command. Any

command Python receives is called statements. Examples are below we can type:

```
>>> print("Hello, world")
```

- Python displays at the prompt, or on your screen - Hello, world. Also when you type the following statement below at the prompt, Python does the following:

```
>>> print(2+3)
```

```
5
```

```
>>> print("2+3=", 2+3)
```

```
2+3= 5
```

```
6/18/2017 6
```

```
FEDERAL UNIVERSITY OF
```

```
PETROLEUM RESOURCES
```

```
EFFURUN
```

Supplementary: Python Programming

□ Next, we go to Python Programming. That chapter is aimed at introducing You to Python basics. Note that every one can program – you may start slow; But, eventually – you will become better and may be, feed yourself from it.

□ Often, becoming an experienced programmer involves code copying from textbooks and other materials available. Change them and play around with other codes to see what result it will yield and so on....

□ No one became great in the art of programming on their own – without help from. They either did so through the help of others or through other materials. However, others as well as textbooks cannot help you with, how creative you can become and how well you learn the art. That is strictly YOU.

□ Programming is a necessity – as a computer scientist. You are almost nothing without it.

□ Learning Python and mastery of it – will help you navigate smoothly other languages such as Java, C-family, Haskell and others.

□ In all as in LIFE – prepare for the worse; But expect the best there is...

□ In this supplementary chapter, we would learn how to define functions and use them as well as learn about chaotic models

```
6/18/2017 7
```

```
FEDERAL UNIVERSITY OF
```

```
PETROLEUM RESOURCES
```

```
EFFURUN
```

Executing single statements

```
>>> 2+3
```

```
5
```

```
>>> 22/7
```

```
3.142857142857143
```

```
>>> 3**2
```

```
9
```

```
>>> print("Hello world")
```

```
Hello world
```

```
>>> print("2+3=", 2+3)
```

```
2+3=5
```

□ All the above are single statement. To solve real world problems, we would often need to generally execute more than one statement. To execute multiple statements, one way to achieve this is through the use of functions:

```
>>> def hello():
```

```
    print("Hello")
```

```
    print("Computers are Fun")
```

```
6/18/2017 8
```

```
FEDERAL UNIVERSITY OF
```

```
PETROLEUM RESOURCES
```

```
EFFURUN
```

```
Define / invoke a Function
```

```
>>> def hello():
```

```
    print("Hello")
```

```
    print("Computers are Fun")
```

□ The first line tells Python we are defining a new function called "hello".

□ The next lines are indented to show they are part of the hello function.

□ The blank line (hit enter/return twice) lets Python know the definition is finished. You also notice that nothing has happened yet! We defined the function; But, we have not told Python to perform the function!

□ A function is invoked or executed by typing its name. So, if we type:

```
>>> hello()
```

```
Hello
```

```
Computers are fun
```

```
6/18/2017 9
```

```
FEDERAL UNIVERSITY OF
```

```
PETROLEUM RESOURCES
```

```
EFFURUN
```

```
FUNCTIONS: Parameter PARSING
```

□ Functions can have changeable parts called parameters, placed between the ()'s. The function "hello" has no parameters. We can define the function with one parameter as:

```
>>> def greet(person):
```

```
    print("Hello ", person)
```

```
    print("How are you?")
```

□ If we try to execute the function "greet()"

```
>>> greet()
```

Traceback (most recent call last):

File "<pyshell#74>", line 1, in <module>

```
greet()
```

TypeError: greet() takes exactly 1 argument (0 given)

□ We get an error (as we did not specify the value of “person”). When we use parameters, we can customize the output of a function.

```
>>> greet("Terry")
```

Hello Terry

How are you?

```
>>> greet("Paula")
```

Hello Paula

How are you?

6/18/2017 10

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Module files

□ When we exit the Python prompt, all functions that we defined will cease to exist.

□ Programs are usually composed of functions, modules or scripts that are saved on disk so that they can be used again.

□ A module file is a text file created in text editing software (saved as “plain text”) that contains function definitions.

□ A programming environment is designed to help programmers write programs and usually includes automatic indenting, highlighting, etc.

File: chaos.py

A simple program illustrating chaotic behaviour

```
def main():
```

```
    print("This program illustrates a chaotic function")
```

```
    x = eval(input("Enter a number between 0 and 1: "))
```

```
    for i in range(10):
```

```
        x = 3.9 * x * (1 - x)
```

```
    print(x)
```

```
main()
```

□ We use filename.py when we save our work to indicate it’s a Python program.

□ Above, we are defining a new function called main(). The main() at the end tells Python to run the code.

6/18/2017 11

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Executing Chaos.py

This program illustrates a chaotic function

Enter a number between 0 and 1: .5

0.975

0.0950625

0.335499922266

0.869464925259

0.442633109113

0.962165255337

0.141972779362

0.4750843862

0.972578927537

0.104009713267

□ Inside the Python Program Chaos.py

□ Line that starts with # are called comments. It is intended that users can read it; while the Python program ignores it. It makes clear the intent of that program section.

□ Thus, Python skips text from # to end of the line.

6/18/2017 12

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Inside the Python Program Chaos.py

def main():

□ We begin with definition of a function called main. Since our program has only one module, it could have been written without the main function. However, the use of main is customary.

□ The next line of statement causes Python to print a message introducing the program.

print("This program illustrates a chaotic function")

□ Next, x is declared as a variable, used to assign a name to a value so that we can refer to it later. The quoted information is displayed, and the number typed in response is stored in x.

x = eval(input("Enter a number between 0 and 1: "))

□ The for loop construct tells Python to repeat the same thing over and over. In this example, the following code will be repeated 10 times.

for i in range(10):

□ The body of the loop is what gets repeated each time through the loop. The body of the loop is identified through indentation. The effect of the loop is the same as repeating this two lines 10 times!

```
x = 3.9 * x * (1 - x)
```

```
Print x
```

6/18/2017 13

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Inside the Python Program Chaos.py

□ Loop is equivalent to writing the same statement multiple times

```
for i in range(10) : x = 3.9 * x * (1 - x)
```

```
x = 3.9 * x * (1 - x) print (x)
```

```
print (x) x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

These two statements $x = 3.9 * x * (1 - x)$

are equivalent print (x)

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

```
x = 3.9 * x * (1 - x)
```

```
print (x)
```

6/18/2017 14

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Inside the Python Program - Chaos.py

□ The first statement (below) is an assignment statement. The part of the statement on the RHS of “=” is the mathematical expression that performs the task. Once the value on that RHS is computed, it is stored back into (or assigned) to x:

```
X = 3.9 * x * (1 - x)
```

□ Next statement below main() tells python to execute the code in the function main()
main()

□ For the chaos.py program as below:

```
# File: chaos.py
def main():
    print("This program illustrates a chaotic function")
    x = eval(input("Enter a number between 0 and 1: "))
    for i in range(10):
        x = 3.9 * x * (1 - x)
    print(x)
main()
```

□ For any given input, return 10 times the seemingly random numbers between 0 and 1. It appears that the value x is chaotic. The function computed has the general form $k(x)(1 - x)$ where $k = 3.9$.

□ This type of function is known as logistic function. It models certain kinds of unstable electronic circuit. A mathematical model is chaotic if very small difference in the initial value can have large difference in the output.

□ From thus, we can model anything we can think about and implement it in Python.

6/18/2017 15

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Execute / import module in Python

□ We can run a program in a module file any time we want using one of the following methods

1. Using IDLE, the easiest way. Select Run □ Run Module from the text editor window (for all operating systems)
2. We can use the shortcut key F5 (windows) and Fn+F5 (Mac OS)
3. In command line (windows) or terminal (Mac OS), enter "python chaos.py" (paths must be specified).
4. You can also double click the .py file in Windows to run it

□ We can also import files into Python. The statements below tells Python interpreter to load the chaos module from the file chaos.py into the main memory. • Since the last statement of chaos.py was "main()" the function main will get executed upon importing chaos.

```
>>> import chaos
```

This program illustrates a chaotic function

Enter a number between 0 and 1: .25

0.73125

6/18/2017 16

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Execute / import module in Python

- When Python imports a module, it executes each line.
- The “def” causes Python to create the “main” function. When Python encounters the last line, it executes the main function.
- Upon first import, Python creates a companion file with .pyc extension. This is an intermediate file containing the byte-code used by the interpreter.
- Modules need to be imported in a session only once. We can execute a function defined in a module by typing
`>>> chaos.main()`

□ We can define multiple functions in a module file. We already know that we can call a function by typing:
`moduleFileName.functionName()`

- To make programming easy, Python will present us with a list of functions in the module when we type the module name and follow it with a dot

6/18/2017 17

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

In Summary

- Python is an interpreted language. We can execute commands directly in a shell or write a module file.
- A Python program is a sequence of commands (statements) for the interpreter to execute. It can take input from the user, print output to the screen and run a set of statements multiple times in a loop.
- A mathematical model is chaotic if very small changes in the input lead to very large changes in the output making it seemingly random. Many real world phenomena exhibit chaotic behaviour and the power of computing is used to model such phenomena.
- Please do not forget the instructions as given in the first introductory pages. Any further questions?
- Please feel free to read ahead as it will make the course easier for you.

6/18/2017 18

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

CSC 122

Introduction to Problem Solving

with Python Programming

Arnold A. OJUGO (PhD)

Lecture 04 – Basic Programming

Processes

These slides are based on textbooks and other materials accompanying this course

6/29/2017 1

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Welcome Back...

6/29/2017 2

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

The Software Development Process

□ There are 6-steps:

1. Analyze Problem – Answer these question: (a) What exactly is the problem to be solved? (b) Have I understood it very well? and (c) Can I solve this problem (you cannot solve a problem unless you fully understand it).
2. Determine Specifications: (a) describe exactly what your program will do (not how to do it for now), (b) what are my input & output, and (c) describe how the outputs relate to its inputs.
3. Create a Design: (a) formulate an overall structure of the program, (b) develop a pseudo code for the design because this is where the how of the program gets worked out, and (c) you choose your own algorithm that solves the problem and meets the specifications.
4. Implement the Design by Writing the Program: (a) translate the design into a computer language, (b) write each step of the design as a program statement, and (c) for the purpose of this course: we will use Python programming language.
5. Test/Debug Program: (a) run your program to see if it works as expected, (b) there may be syntax errors or errors in your algorithm, (c) if there are errors (bugs), they need to be located and fixed. This is called debugging, (d) your goal is to find the errors, and (e) try different input values and see if the results are correct.
6. Maintain the Program: (a) continue developing the program in response to the needs of your users, and (b) in real world – most programs are never completely finished because they simply evolve.

6/29/2017 3

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Example 1: Temperature Converter

□ Analysis: Temperature is in Celsius – and the user wants Fahrenheit.

□ Determine the Specification:

- Input – temperature in Celsius
- Output – temperature in Fahrenheit
- Output = $9/5(\text{input}) + 32$

□ Design: In design, there are two factors: (a) detail the specifications, and (b) Write the pseudocode

- Input, Process, Output (IPO)
- Prompt user for input (Celsius temperature)
- Process it to convert it to Fahrenheit using $F = 9/5(C) + 32$
- Output the result by displaying it on the screen

a. Design (Continued...): Write the Pseudocode First

1. Before writing the program, start by writing the pseudocode
2. Pseudocode is precise English that describes what a program does, step-by-step
3. Using pseudocode, concentrate on algorithm rather than programming language

6/29/2017 4

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Example 1: Temperature Converter (Contd...)

b. Pseudocode

- Prompt the user to input the temperature in degree Celsius (call it Celsius)
- Calculate Fahrenheit as $(9/5)*\text{celsius} + 32$
- Output Fahrenheit

□ Implement: We now we convert it to Python:

""" convert.py

A program to convert Celsius Temps to Fahrenheit by Arnold Ojugo

```
def main():
```

```
    celsius = eval (input ("What is the Celsius temperature? "))
```

```
    fahrenheit = (9/5) * celsius + 32
```

```
    [print ("The temperature is ", fahrenheit, " degree Fahrenheit. ")
```

```
main()]
```

6/29/2017 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Example 1 – Contd...

□ Testing the Program: We then test the Program

What is the Celsius temperature? 0

The temperature is 32.0 degrees Fahrenheit.

```
>>> main()
```

What is the Celsius temperature? 100

The temperature is 212.0 degrees Fahrenheit.

```
>>> main()
```

What is the Celsius temperature? -40

The temperature is -40.0 degrees Fahrenheit.

6/29/2017 6

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Elements in a program

□ Names:

- Names are given to variables (celsius, fahrenheit), modules (main, convert) etc. These names are called

identifiers. Identifiers must begin with a letter or underscore followed by any sequence of letters, digits or

underscore. Names are case sensitive (making python a strongly typed language). These are all different, valid names: X, Celsius, score, _score, eggs, etc. These identifiers below may seem same but

they are different.

1. Boiled_Eggs

2. BoiLed_eggs

3. Boiled_EgGs

- Some other names are part of Python itself called keywords or reserved words. Thus, they are not available for us to use as a name for a variable (identifier) in your program. Example includes: and, del,

for, is, raise, assert, elif, in, print etc

If we run the Python program with these commands:

```
>>> x = 5
```

```
>>> x
```

```
5
```

```
>>> print(x)
```

```
5
```

```
>>> print(spam)
```

Traceback (most recent call last):

File "<pyshell#15>", line 1, in -toplevelprint

spam

NameError: name 'spam' is not defined

NameError is the error that occurs when you try to use a variable without a value assigned to it.

6/29/2017 7

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Elements in a Program

□ Expressions:

- The fragments of codes that produce, calculate or compute new data values are called expression.

$(9/5) * \text{celsius} + 32$

- Expressions are composed of literals, variables and operators. Literals are used to represent a specific value such as 3.9, 1, 1.0 etc.

- Two expressions can be combined with an operator to make another expression

- Simpler expressions can be combined using operators: +, -, *, /, **, && etc

- Spaces are irrelevant within an expression, and the usual mathematical precedence always applies:

$((x1 - x2) / 2 * n) + (egg / k ** 3)$

□ Output Statements

- A Print statement can print any number of expressions

- Successive print statements will display on separate lines

- A bare print statement will print a blank line.

- For example:

`print(3+4)` 7

`print(3, 4, 3+4)` 3 4 7

`print()`

`print(3, 4, end=" ")` 3 4

`print(3 + 4)` 7

`print("The answer is", 3+4)` The answer is 7

6/29/2017 8

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Input data from a user

□ The expressions below will evaluate the input to an "int" or a "float" or a "string" based on what the user enters:

`z = eval(input('type a value ', ))`

□ This statement will cast the input to "int" and throw an error if the user did not type an integer:

`z = int(input('type a value ', ))`

□ In Summary so far:

□ We have learned about the steps of software development process

- We wrote and analyzed a simple Python program
- We learned the concepts of pseudocode
- We learned about the elements of a program

6/29/2017 9

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Assignment statements

- A simple assignment statement yields that the value of a variable is given by value of its corresponding expression: `<variable> = <expression>`
- Thus, the statement on the RHS is evaluated to produce a value which we can then associate with the variable on the LHS.

$\text{Fahrenheit} = (9/5) * \text{celsius} + 32$

- Variables can be assigned and reassigned as many times as possible, and as much as you want!

```
>>> myVar = 10
```

```
>>> myVar Before z = z+ 1 After
```

```
10
```

```
>>> myVar = myVar + 1
```

```
>>> myVar z 10 z 11
```

```
11
```

- Variables are boxes (memory location) we put values in. If the variable changes, the old value is erased, and a new one is written therein as in fig. 1. Technically, this model of assignment is simplistic for Python. Python does not overwrite these memory locations; Instead, it jumps to store the current value of `z` in the new location and sticks the pointer to that current location.

6/29/2017 10

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Assigning Inputs

- An input statement aims to get input from a user and store it into a memory location (as a variable) via the statement: `<variable> = eval (input(<prompt>))`
- First, the prompt is printed.
- The input part waits for the user to enter the value and press `<enter>`.
- The expression that was entered is evaluated to turn it from a string of characters into a Python value (a number).
- This evaluated value is then assigned to the variable.
- Several values can be calculated at the same time

<var>, <var>, <var>, ... = <expr>, <expr>, <expr>, ...

□ It evaluate expressions in the RHS and assign them to the variables on the LHS as: sum, diff, ave = x + y, x - y, (x + y) / 2

□ How can we use this to swap values for x and y; And, why would the example we have below, not work?

- x=y

- y=x

- Assume x = 1, y = 9 initially...the statement x = y will make x = 9 (so the original x is lost).

So that when we now have the next line where y = x, y will retain its previous numbers.

Thus, y = 9.

6/29/2017 11

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Swapping values

□ To achieve this swap, we can use the temp variable:

Temp = x: x = y: y = x

□ In the swap, the value of x is stored in Temp (i.e. Temp = x □ 1). Next, x takes a new value y (i.e. x = y □ 9); Lastly, y takes new value Temp (y = Temp = old x □ 1).

□ We can swap the values of the two variables easily in Python using the statement below:

```
>>> x = 3
```

```
>>> y = 6
```

```
>>> print x, y
```

```
3 6
```

```
>>> x, y = y, x
```

```
>>> print x, y
```

```
6 3
```

□ We can use this idea of multiple assignment to input multiple variables from single input statement. We simply use the commas to separate the inputs:

```
def donutegg():
```

```
donut, eggs = eval(input("Enter number of donuts followed by number of eggs:"))
```

```
print("You ordered ",donuts," donuts and ",eggs," eggs.")
```

```
>>> donutegg()
```

```
Enter # of donuts followed by # of eggs: 3, 2
```

```
You ordered 3 donuts and 2 eggs.
```

6/29/2017 12

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Definite Loops

- A definite loop executes a definite number of times
- As Python starts, it knows exactly how many iterations to do (how many times to execute the statement inside the loop):

For <var> in <sequence>:

<body>

- The beginning and end of the body are indicated by the indentation.
- The variable after “for” is called loop index. It takes on each successive value in the sequence. For example:

```
>>> for i in [0,1,2,3]:
```

```
print(i)
```

```
0
```

```
1
```

```
2
```

```
3
```

```
>>> for odd in [1, 3, 5, 7]:
```

```
print(odd*odd)
```

```
1
```

```
9
```

```
25
```

```
49
```

```
6/29/2017 13
```

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Definite loops...Contd

- In chaos.py, what did range (10) do?

```
>>>list(range(10))
```

```
[0,1,2,3,4,5,6,7,8,9]
```

- range is a built-in Python function that generates a sequence of numbers starting with 0.

- list is also a built-in Python function that turns sequence an explicit list

- The body of the loop executes 10 times.

- for loop alters the flow of the program execution so they are referred to as control structures

More items in

<sequence>

<var> = next item

<body>

```
6/29/2017 14
```

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Example Program: future value

□ Analysis:

1. Money deposited in a bank
2. How much will the account be worth in the next 10years from now?
3. Input: principal amount, interest rate...
4. Output: value of the investment in 10years

□ Determine the Specifications:

1. User inputs the initial amount to invest (i.e. principal amount) named <var = principal>
2. User inputs the annual percentage rate (i.e. interest rate) names <var = interest>
3. The specification is represented as a relationship value expressed mathematically as
New Amount = Principal * (1 + interest). This is done 10 times

□ Design

1. Print an introduction
2. Input the amount of principal
3. Input the annual percentage rate (interest rate)
4. Repeat 10 time: principal = principal * (1 + interest)
5. Output the new value of principal

6/29/2017 15

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Example Program: future value

□ Implementation:

1. Each line translates to one line of Python program (in this case)
2. Print an introduction can be written as
`print("This program computes the future value of a 10year investment")`
3. Input the amount of principal
`principal = eval(input("Enter the initial Principal: "))`
4. Input the annual percentage rate (interest rate)
`interest = eval(input("Enter the interest rate: "))`
5. Repeat 10 times: `for i in range(10):`
6. Calculate `principal = principal * (1 + interest)`
7. Output the value of the principal at then end of 10years

6/29/2017

`print("The value in 10years is: ", principal) 16`

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Example Program: future value

□ The program is thus:

```
# futval.py
# A program to compute the value of an investment
# carried 10 years into the future
def main():
    print("This program calculates the future value of a 10-year investment.")
    principal = eval(input("Enter the initial principal: "))
    interest = eval(input("Enter the annual interest rate: "))
    for i in range(10):
        principal = principal * (1 + apr)
    print("The value in 10 years is:", principal)
main()
```

□ In Summary – we have learnt the concept of assignment in Python

□ We learned how to

1. Assign values to variables
2. Do multiple assignments in one statement
3. Definite simple definite loops
4. Use “for” loop to alter the sequence of the program

6/29/2017 17

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

CSC 122

Introduction to Problem Solving
with Python Programming

Arnold A. OJUGO (PhD)

Lecture 04 – Agent Based Modeling
and Graph as a Model

These slides are based on textbooks and other materials accompanying this course

6/29/2017 1

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Let us Begin.....

6/29/2017 2

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Definition

□ A key challenge in computational thinking is the scale and complexity of the systems we study and build. A major technique to manage this complexity is via abstraction. Abstraction takes many specific forms such as modelling, decomposing, and generalizing. In each case, the complexities are dealt with by hiding the complicated details behind a simple abstraction (or user interface or model) of the situation.

- a. Underground map for train tunnels is a simple model of a complex reality. It is a model that contains precise information necessary to plan a route from one station to another.
- b. A procedure to compute square roots hides the complicated (iterative approximation to the root, handling special cases) implementations behind a simple interface.

□ What is a Model?

1. Computational thinking values elegance, simplicity and modularity over adhoc complexity.
2. Modelling develops a representation of a real world issue, system, or state that captures important aspects of the situation for a particular purpose, while omitting everything else.
E.g: Map of FUPRE; transition diagrams etc.

3. Thus, a model is a mathematical representation of modeling a process. E,g $Y = MC^2$ is a formular for modeling the speed of light in relativity? Show them to work out models

6/29/2017 3

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Agents and self-organizing processes

□ We use agents in modeling. Each item in our mathematical representation can be viewed as an agent. Each of us can also be referred to as an agent.

1. Agents always exist in an environ. The environ has conditions and states under which the agent is expected to occupy and operate. Agents are expected to reach a particular goal as they operate in the environment. The environment is called a search or state space.
2. A State is a place in the environ that an agent can move to/fro. Its start state is the first position an agent finds itself (at the begin of the task). Its final or goal state is an accepted state that all conditions are met and the agent has found or reach a solution to the problem.
3. Conditions are the rules that applies in specific states that agent find themselves. Thus, they are rules that applies to the specific state an agent is occupying or operating under at any point in time. Time is finite and leads an agent to move fro/to states.

□ Thus, in an Environment, agents can only perceive, observe and react.

1. Agents (from start state) tries to understand the environ. Thus, they must observe the environment. This can be full (if the agent sees all in the state it occupies and learns what it observes), or partial (if the agent sees a part of all there is in the state it occupies and learns only

part of what it observes). We use our organs (eyes and ears) to observe the environ.

2. Some conditions in a state can make it difficult for an agent to fully observe. The agent must adopt

other means to adapt and understand. Perception allows an agent to perceive the environ and learn features therein. We use our skin to feel and tongue to taste (fully observe) some environ.

3. Actuators help an agent to react by changing position from one state to another; Or changing its behaviour in reaction as well as in response to inputs that it receives from the environment. We use our legs to move within an environ, and hands to gesture responses to other agents.

6/29/2017 4

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Properties of an agent

1. Rationality: Agents must act right under every conditions they operate and in any state they occupy. As they interact with other agents and react to conditions in the states they occupy – they learn beliefs and gather knowledge. Their belief helps them to predict what happens next; while their knowledge assures them of the inevitable events that will happen.

2. Autonomy/Self-Organize: An agent is solely responsible for its action. Its decisions of its own accord and cannot be influenced without its knowledge and acceptance.

3. Interdependent: Agents use persuasion, sanctions and imitation means to influence other agents (influence the response they receive as input from the environ). Agents may also change their behaviour which will also change some aspects of the environ and in turn affect the behaviour of other agents – so that the consequences of each agent's decision depends in part on the choices of others.

4. Rules: Agents follow simple rules. They abide and live by these norms as conventions, protocols, morals, social habits and traditions. These rules (no matter how simple) produce global patterns that may not be at all obvious; But, are very difficult to understand. Thus, they must adapt to these (inexplicable) behaviour, which is largely a reflection of the complexities of the environment.

5. Adaptive/Backward Looking: As agents interact, they learn and change – by generating a complex adaptive system called behaviour. They adapt by moving, imitating and replicating but not calculating the most efficient action. They fear change (though they easily adapt). They adapt at 2-levels: personal and collective (population-wise). Learning alters the probability distribution of an behaviours competing for attention in an agent (good or bad); while, evolution alters the frequency distribution of agent-types competing for production in a population.

6/29/2017 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Agent-Based Modeling

□ A flock of birds flying in tight formation. Collectively, they form a giant shape that moves

purposefully as if it were a single organism. Yet, the flock has no “group mind” nor is there a “leader bird” choreographing the formation. Instead, each bird reacts to the movement of its immediate neighbors who in turn react to it. It results in a graceful movement of a flock whose rhythm is clearly patterned yet also highly nonlinear.

□ Reynold modeled the flock as agent interaction from relational level as: (a) Separation: Don’t get too close to any object, or other boids, (b) Alignment: try to match the speed and direction of nearby boids, and (c) Cohesion: Head for perceived center of mass of the boids in your immediate neighborhood.

□ At global level, the flock is not governed by a system-wide rule. Thus, such modeling will be immensely difficult due to the extreme complexity of nonlinear systems.

□ But, modeled as interaction between agents on a relational level, the task is quite easy. Reynold modeled the flock as an aggregation of local interactions via the 3-rules as above.

□ ABM focuses on: (a) dynamic social networks and behaviours that shape and are shaped by agents’ interaction, and (b) are used to perform virtual experiments that test theories by manipulating structural factors such as network topology, stratification and mobility.

□ Structures emerge: As agents interact, their behaviors goes through a socio-physical space in response to influences and selection pressures. Agents may start out indifferent about the environ. It will likely change location or behavior to avoid being different, isolated or overcrowded. Rather than produce homogeneity, system yields conformist decisions that aggregates to global pattern of culture, strata and clusters in networks. Some studies start with a heterogeneous population and end with convergence, coordination, diffusion, and sudden collapse of norms, institutions, beliefs, innovations, standards, etc.

□ Emergent Order: Egoistic adaptation can lead to successful collective action without either altruism or global (top-down) imposition of control. A key finding across numerous studies is that viability of trust, cooperation and collective action depends decisively on the social embeddedness of interaction.

6/29/2017 6

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Redefining any Task

Any system can be viewed or decomposed into component (agent) parts interacting with each other to

achieve a common goal. This interaction can lead to social behaviours and norms under which the environ is scheduled and under which the agent must operate.

1. Is interaction global or local, that is, is the population fully connected or is interaction constrained

by the structure of social ties?

2. If interaction is local, are the constraints on tie formation spatial or social?

3. Are ties elective (formed and broken) or is interaction forced?

4. Is adaptation based on learning (which modifies the probability distribution of behaviors in each

agent's repertoire) or evolution (which modifies the frequency distribution of behaviors across the population of agents)?

5. If evolution, does reproduction involve competition for survival or social influence?

6. If influence, is this limited to external states of the agent (e.g., behavior) or do agents copy other agents' internal programming, even though this cannot be directly observed?

7. Is influence based on attainment (success, fitness, payoffs, status) or familiarity (proximity, frequency)?

8. Is the model used as an experiment (parameters are manipulated to test for predicted differences)

or a demonstration (parameters are manipulated to test for predicted robustness)?

9. If used experimentally, are the manipulations mainly of agent-level parameters (to test a micro theory about the global implications of behavioral assumptions) or system-level parameters (to test a macro theory about the dynamical implications of environmental assumptions)?

6/29/2017 7

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Definition

□ What are Graphs? Why study graphs? Which graphs and what commonalities should we study? What tools can be easily adapted?

□ A graph G consist of a set of vertices (nodes) that are joined together by a corresponding set of edges (arcs). It is denoted as $G = (V, E)$. Each vertex x

□ V with each edge $m \in E$.

□ A network (like a graph) can best be described as a representation of interaction structure among units. In social cum economic networks, these units (nodes, agents, actors) are also seen as individuals or firms.

□ At some broad level, the study of graphs (or networks) encompasses the study of all kinds of interactions such as:

a. Information transmission

b. Web links

c. Information exchange through data communications and networks

d. Trade

e. Credit and financial flows

f. Friendships

g. Trust

h. Spread of epidemics

i. Diffusion of ideas and innovations

6/29/2017 8

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Definition

□ A graph G consist of a set of vertices (nodes) that are joined together by a corresponding set of edges (arcs). It is denoted as $G = (V,E)$. Each vertex $x \in V$ with each edge $m \in E$.

- A node or vertex can be viewed as an agent. Thus, the properties of an agent applies to the node. Each node has neighbors it is either linked to, or is isolated from.
- Edges are links that show dataflow direction from a node to another. The links are best seen as relationships (and they are either weak, strong or isolated). Links are measured through dyads D . Links are either undirected (denoted by a line to show data flows in any direction at any time) or directed (denoted by an arrow to show data flows in certain direction). When directed, they are either self-linked (loop), single-link or multi-link.
- Relationship between a parent and their children is seen as directed (due to factors such as dependence, trust, obligation and so on); while, the relationship between siblings of the same parent (i.e. a brother and a sister) is seen as undirected (in this guise).

□ Types of Graphs

- Undirected Graphs has a set of unordered elements joined together by links such that it does not show direction of the relationship from one node to another as fig 1 (overleaf).
- Directed Graphs has a set of ordered elements joined together by arcs such that it shows the direction of the relationship from one node to another as in Fig. 2 (overleaf)
- Weighted Graphs – often, the arcs of a graph can be assigned weights or numbers to indicate the cost or penalty to move from one node to another as fig 3 (overleaf)

6/29/2017 9

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

More Definition f

For a graph G , we have these definitions that:

a e

□ Walk is a sequence of nodal relationship that begin and ends with nodes. b

Closed walk starts and ends with the same node. Walks are unrestricted as c d

they can involve the same node, or same relation multiple times. Length of a

walk is the shortest number of edges a walk uses. Length of walk a-to-b Fig 1: An Undirected $G = (V, E)$

denoted as $(a,b) = 1$; while $(a,b,d) = 2$, $(a,b,c,d) = 3$, $(a,b,c,d,e) = 4$, f

$(a,b,f,b,c) = 4$ (notice here, the node is repeated). A cycle is a restricted walk

such as $(b,a,c) = 2$. cycles beginning and ending with A is (a,b,c) ; Those of a e

B are (b,a,c) , (b,c,d) , (b,a,c,d,e,f) and (b,d,e,f) . b

□ Trail is a walk that includes a given relation at most once. Same node(s) can

be part of a trail multiple times. The length of a trail is number of relation in c d

it. Closed trail begins and ends with the same node. E.g. (a,b,c,b,d) is a walk Fig 2: A Directed $G = (V, E)$

but not a trail. 2 y

□ Path is a walk in which each node (and relation) is used at most once. All x 6
paths list a walk and trails. Thus, all paths are walks and trails; But not all 1
trails and all walks are paths (closed walk). 3 2

z

□ Trees is a connected acyclic (not cyclic) graph. The vertex of degree 1 is Fig 3: Graph $G = (V, E, w)$
called a leaf. The degree of a node is the number of edges incident to it.

Thus, a leaf has only 1-relation (from where it sprout). The in-degree is the x y z
number of incoming relations; while out-degree is the number of outgoing x 0 2 3
relations. E.g. how many persons you consider your friends may not be the y 0 0 2
same number of people that consider you their friend. z 1 6 0

As we walk through a graph, it is a travel or graph traversal. Suppose we have Fig 3b: Weighted
Graph

the weighted graph as in fig 3, the weight data in the graph are represented as
adjacency matrix called an array as in fig 3b.

6/29/2017 10

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Graph Traversal Operations

- The Dictionary as a system can best be a b c ...y z
viewed as a graph, that is why – it allows it
is organized for fast lookup.

- Many strategies exists in solving the graph a b c ...y z abc ...
yz abc

yz

problems. The efficiency and accuracy of
the solution actually boils down to how you ...y z
abc 26-ary “tries”

“search” the graph.

- We search graphs in that in a given task,
we start with an initial state and search for abc ...
yz “CAB”

the goal state. This is called graph travel
or traversal.

- These strategies for searching include: 3 a 1

i. DFS and BFS c

ii. Shortest Path b

iii. Minimal Span Tree (MST) 4 2 6

5 f

- For example, if we consider a travel in the 7 graph G to find a path from node a to d. e

6/29/2017 11

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Graph Traversal: DFS Vs BFS

6/29/2017 12

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Graph Traversal: Minimal Span Tree

MST is a connected G without cycles. Its feats: (a) only a path joins any two nodes, (b) each edge is a bridge, and (c) for a connected tree, there is N-nodes and N-1 edges. Given $G = (V, E)$ with weighted-edges as costs c_e , an MST is a subset of edges $T \subseteq E$ such that T spanning tree (contains all nodes of G) whose sum of edge weights is minimized.

Minimal Spanning tree
(MST)

6/29/2017 13

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Graph Data Representation & Linked-List Implementation

□ Suppose, as a CS specialist, you were consulted by coy that wishes to construct a railroad in Delta State with a connection in Benin also. If you spotted these cities and was asked to decide were to start the construction, how would you go about it.

□ We use the following techniques to resolve graphs: MST, Dijkstra shortest route, network capacity, PERT and others. We represent the graph as below

Graph:

Benin 78 Agbor 48 Asaba

[0] Asb 3 48 6 20

41 36

53 20 [1] Abbi 2 18 2 18

Abraka

Ogwashi [2] Abk 3 48 6 20

19 18 43

Sapele 52 Agb

[3] 2 18

Eku 71 Abbi 54 [4] Ben 3 48

43 23 Ozoro [5] Eku 2 18

21 47

45 [6] Ogw 3 48

Warri 29

Ughelli [7] Ozo 2 18

[8] War 3 48

[9] Sap 2 18

[10] Ugh 3 48

6/29/2017 14

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Array Implementation

Graph:

.numVertices = 11

.vertices [0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12]

[0] 0 0 0 48 0 0 20 0 0 0 0 - -

Asb [1] 0 0 18 0 0 71 43 23 0 0 0 - -

Abbi 0

[2] 18 0 36 41 19 0 0 0 0 47 - -

Abk [3] 48 0 36 0 78 0 0 0 0 0 0 - -

Agb [4] 0 0 41 78 0 0 0 0 0 53 0 - -

Ben

[5] 0 71 19 0 0 0 0 0 21 52 0 - -

Eku

[6] 20 43 0 0 0 0 0 54 0 0 0 - -

Ogw

[7] - 23 0 0 0 0 54 0 0 0 45 - -

Ozo

[8] 0 0 0 0 0 21 0 0 0 43 29 - -

War

[9] 0 0 0 0 53 52 0 0 43 0 0 - -

Sap

[10] 0 0 47 0 0 0 0 45 29 0 0 - -

Ugh

[11] - - - - -

[12] - - - - -

6/29/2017 15

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

More On Relationships: a Practical View

Graph helps us understand data explored and display nodal relations or ties by showing us the complex data and its structural quantities captured. Positive edges between nodes are positive relations (friends, marriage). Negative edge is negative relation (hatred, anger). If $A \square B$ and $B \square C$ are positive relations; And $C \square A$ is negative. The relation is more likely to morph into balanced cycle or remain unbalanced. In every graph, there are 3-kinds of nodes (agents):

1. Alpha – are agents whom the graph is centered around. He makes most of the decisions because his choice eventually emerge as the graph's (global) decision.
2. Confidant – He is the alpha's closest ally. The alpha seeks his opinion in all matters. Thus, he secretly controls the alpha. He is the only person an alpha can die for.
3. Boo – every other friend of an alpha. Some of whom are direct friends, others are friends of a friend. Those with direct relation with the alpha has a pathlength of 1. those who are friends with the alpha's direct friend has a pathlength of 2; and so on.

Other terms we use from a practical view includes:

- Dyads are steady, durable relations between node. It measure features that are local to a node (agent) that eventually emerge from interaction (as a relational structure of strength and multiplexity) that eventually leads to reciprocity and mutuality.
- Diffusion – An innovation is an idea or product introduced to a network. As it spreads with the graph – diffusion takes place. An agent can adopt an idea with respect to the agent (or conditions) influencing his decision. The resistance an agent puts up before he/she adopts the idea is called threshold. The agent who introduces an idea into a graph for it to spread is called a seednode. If a graph has a closely knit relation between a few nodes (such as a caucus) it is called a clique.
- An agent is said to be exposed if he has seen an innovation or idea before.

6/29/2017 16

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Propagation Models

Graphs, helps us to diffuse an innovation. If we have a graph of people in Warri – and the disease Ebola (as innovation) was spreading in the graph. The original person whom the virus came through into Warri is called the seednode. There are two models through which an innovation can spread: (a) Susceptible-Infect-Susceptible (SIS), and (b) Susceptible-Infect-Remove (SIR). A node exposed to an innovation is liable to either accept or reject it. If he accepts – he is said to have adopted the innovation; Else, he has rejected. For these SI-models, a node or an agent can be in any of these states:

- (a) susceptible is when a node is exposed to an innovation, but has not yet adopted the innovation. As in the case above, it implies the node/agent is exposed to someone who has the virus and is likely to be infected any time; But, for the time has not been infected.
- (b) infected is when the node has adopted the innovation. As in the case above, it implies that

the node has been infected by the virus and can thus, pass it to others agents or nodes.

(c) removed is when a node has adopted the innovation and feels he/she made a wrong choice. The node must then be immunized off the innovation. As in case above, the node had been infected by the virus; But, has been immunized or (in some case, virus dies after some time). Thus, the node is permanently immunized and can no longer participate in propagation, and a particular node cannot be infected twice.

□ Thus, in SIS, a node can be cured but not immunized. Thus, it can be infected again. Such node switches between susceptible, infected and cured. But in SIR, a node is immunized; And thus, can never be infected again.

6/29/2017 17

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Propagation/Immunization Models

We know that innovations can be diffused and propagated through SI-models. Some of the challenges in SI propagation model are as follows:

- a. Extent: A seednode is needed to help diffuse an innovation in a graph G. So, with seednode (i.e. specific subset of initially infected nodes) in G and the propagation model used, how many nodes in G are expected to be infected (or activated)after a specific time/period?
- b. Targeting: Which nodes do we target when the seednode introduces the innovation so that we can have the maximum number of adopter in the shortest time possible. An adversary can initiate such target to get best result in shortest time.
- c. Blocking: Which nodes in G, can we also target for immunization to minimize the expected number of infected (activated) nodes in the shortest possible time. Note that immunization is aimed to cancel out whatever targeting has been projected. Thus, its the reverse of targeting aimed to stop propagation.

Why do we use graphs for such? We need to answer these questions...

1. Know these factors: (a) what is the innovation, (b) how does it diffuse (pattern), (c) does it start and end of its own accord...or how?, (d) what is its incubation (time it takes to breed), (e) what other factors do I need to know about this innovation that is not easily visible?
2. For Target/Block: (a) how do I determine the most influential agents/nodes in G, (b) who are the most influential agents, (c) what clusters and cliques exist in this graph, (d) what timeline am I expected to acquire/stop the spread effects, (e) what other factors do I need?
3. Supplementary: (a) what model do I use to accomplish my computations?, (b) how do I present my final report, (c) have I evaluated to seek if this is the best cause of action?

6/29/2017 18

Paper 2 of 3 — COS 102 (Part Two)

Level 100 • First Semester • Handout

Chapter 4

Introduction to

Problem Solving

4.1 Introduction “Computer Science is a

Today, computers are all around us. We use them science of abstraction for doing various tasks in a faster and more accurate -creating the right model for manner. For example, using a computer or smartphone, a problem and devising the we can book train tickets online.

appropriate mechanizable

India is a big country and we have an enormous techniques to solve it.”

railway network. Thus, railway reservation is a complex task. Making reservation involves information about –A. Aho and J. Ullman many aspects, such as details of trains (train type, types of berth and compartments in each train, their schedule, etc.), simultaneous booking of tickets by multiple users and many other related factors.

It is only due to the use of computers that today, the booking of the train tickets has become easy. Online booking of train tickets has added to our comfort by enabling us to book tickets from anywhere, anytime.

We usually use the term computerisation to indicate the use of computer to develop software in order to automate any routine human task efficiently. Computers In this chapter are used for solving various day-to-day problems »» Introduction and thus problem solving is an essential skill that a »» Steps for Problem computer science student should know. It is pertinent Solving to mention that computers themselves cannot solve a »» Algorithm problem. Precise step-by-step instructions should be »» Representation of given by us to solve the problem. Thus, the success of a Algorithms computer in solving a problem depends on how correctly »» Flow of Control and precisely we define the problem, design a solution »» Verifying Algorithms (algorithm) and implement the solution (program) using »» Comparison of a programming language. Thus, problem solving is the Algorithm process of identifying a problem, developing an algorithm »» Coding

for the identified problem and finally implementing the

»» Decomposition

algorithm to develop a computer program.

2021-22

Ch 4.indd 61 21-May-19 11:45:27 AM

62 Computer Science – Class xi

4.2 Steps for Problem Solving

Suppose while driving, a vehicle starts making a strange

GIGO (Garbage In Garbage

Out)

noise. We might not know how to solve the problem

The correctness of the right away. First, we need to identify from where the output that a computer noise is coming? In case the problem cannot be solved gives depends upon the by us, then we need to take the vehicle to a mechanic. correctness of input

provided. The mechanic will analyse the problem to identify the source of the noise, make a plan about the work to be done and finally repair the vehicle in order to remove the noise. From the above example, it is explicit that, finding the solution to a problem might consist of multiple steps.

When problems are straightforward and easy, we can easily find the solution. But a complex problem requires a methodical approach to find the right solution. In other words, we have to apply problem solving techniques. Problem solving begins with the precise identification of the problem and ends with a complete working solution in terms of a program or software.

Key steps required for solving a problem using a computer are shown in Figure 4.1 and are

Figure 4.1: Steps for problem solving discussed in following subsections.

4.2.1 Analysing the problem

It is important to clearly understand a problem before we begin to find the solution for it. If we are not clear as to what is to be solved, we may end up developing a program which may not solve our purpose. Thus, we need to read and analyse the problem statement carefully in order to list the principal components of the problem and decide the core functionalities that

our solution should have. By analysing a problem, we would be able to figure out what are the inputs that our program should accept and the outputs that it should produce.

Algorithm

A set of exact steps 4.2.2 Developing an Algorithm

which when followed,

It is essential to devise a solution before writing a program solve the problem

or accomplish the code for a given problem. The solution is represented in required task. natural language and is called an algorithm. We can imagine an algorithm like a very well-written recipe for

2021-22

Ch 4.indd 62 08-Apr-19 12:34:19 PM

Introduction to Problem Solving 63

a dish, with clearly defined steps that, if followed, one Notes will end up preparing the dish.

We start with a tentative solution plan and keep on refining the algorithm until the algorithm is able to capture all the aspects of the desired solution. For a given problem, more than one algorithm is possible and we have to select the most suitable solution. The algorithm is discussed in section 4.3.

4.2.3 Coding

After finalising the algorithm, we need to convert the algorithm into the format which can be understood by the computer to generate the desired solution. Different high level programming languages can be used for writing a program.

It is equally important to record the details of the coding procedures followed and document the solution. This is helpful when revisiting the programs at a later stage. Coding is explained in detail in section 4.8.

4.2.4 Testing and Debugging

The program created should be tested on various parameters. The program should meet the requirements of the user. It must respond within the expected time. It should generate correct output for all possible inputs. In the presence of syntactical errors, no output will be obtained. In case the output generated is incorrect, then the program should be checked for logical errors, if any.

Software industry follows standardised testing methods like unit or component testing, integration testing, system testing, and acceptance testing while developing complex applications. This is to ensure that the software meets all the business and technical requirements and works as expected. The errors or defects found in the testing phases are debugged or rectified and the program is again tested. This continues till all the errors are removed from the program.

Once the software application has been developed, tested and delivered to the user, still problems in terms of functioning can come up and need to be resolved from time to time. The maintenance of the solution, thus, involves fixing the problems faced by the user,

2021-22

Ch 4.indd 63 08-Apr-19 12:34:19 PM

64 Computer Science – Class xi

Activity 4.1 answering the queries of the user and even serving the request for addition or modification of features.

What sequence of

steps will you follow to

compute the LCM of 4.3 Algorithm

two numbers?

In our day-to-day life we perform activities by following certain sequence of steps. Examples of activities include getting ready for school, making breakfast, riding a bicycle, wearing a tie, solving a puzzle and so on. To complete each activity, we follow a sequence of steps.

Suppose following are the steps required for an activity 'riding a bicycle':

- 1) remove the bicycle from the stand,
- 2) sit on the seat of the bicycle,
- 3) start peddling,
- 4) use breaks whenever needed and
- 5) stop on reaching the destination.

Let us now find Greatest Common Divisor (GCD) of two numbers 45 and 56.

Note: GCD is the largest number that divides both the given numbers.

Step 1: Find the numbers (divisors) which can divide the given numbers

Divisors of 45 are: 1, 3, 5, 9, 15, and 45

Divisors of 54 are: 1, 2, 3, 6, 9, 18, 27,
and 54

Step 2: Then find the largest common number from these
two lists.

The origin of the term Therefore, GCD of 45 and 54 is 9

Algorithm is traced to Hence, it is clear that we need to follow a sequence
Persian astronomer and of steps to accomplish the task. Such a finite sequence
mathematician, Abu of steps required to get the desired output is called
Abdullah Muhammad

ibn Musa Al-Khwarizmi an algorithm. It will lead to the desired result in a finite
(c. 850 AD) as the amount of time, if followed correctly. Algorithm has a
Latin translation of Al- definite beginning and a definite end, and consists of a
Khwarizmi was called finite number of steps.

‘Algorithmi’.

4.3.1 Why do we need an Algorithm?

A programmer writes a program to instruct the computer
to do certain tasks as desired. The computer then follows
the steps written in the program code. Therefore, the
programmer first prepares a roadmap of the program
to be written, before actually writing the code. Without

2021-22

Ch 4.indd 64 08-Apr-19 12:34:19 PM

Introduction to Problem Solving 65

a roadmap, the programmer may not be able to clearly Notes
visualise the instructions to be written and may end up
developing a program which may not work as expected.

Such a roadmap is nothing but the algorithm which is
the building block of a computer program. For example,
searching using a search engine, sending a message,
finding a word in a document, booking a taxi through
an app, performing online banking, playing computer
games, all are based on algorithms.

Writing an algorithm is mostly considered as a first
step to programming. Once we have an algorithm to
solve a problem, we can write the computer program
for giving instructions to the computer in high level
language. If the algorithm is correct, computer will run
the program correctly, every time. So, the purpose of
using an algorithm is to increase the reliability, accuracy
and efficiency of obtaining solutions.

(A) Characteristics of a good algorithm

- Precision — the steps are precisely stated or defined.
- Uniqueness — results of each step are uniquely defined and only depend on the input and the result of the preceding steps.
- Finiteness — the algorithm always stops after a finite number of steps.
- Input — the algorithm receives some input.
- Output — the algorithm produces some output.

(B) While writing an algorithm, it is required to clearly identify the following:

- The input to be taken from the user
- Processing or computation to be performed to get the desired result
- The output desired by the user

4.4 Representation of Algorithms

Using their algorithmic thinking skills, the software designers or programmers analyse the problem and identify the logical steps that need to be followed to reach a solution. Once the steps are identified, the need is to

2021-22

Ch 4.indd 65 08-Apr-19 12:34:19 PM

66 Computer Science – Class xi

write down these steps along with the required input and desired output. There are two common methods of representing an algorithm —flowchart and pseudocode. Either of the methods can be used to represent an algorithm while keeping in mind the following:

- it showcases the logic of the problem solution, excluding any implementational details
- it clearly reveals the flow of control during execution of the program

4.4.1 Flowchart — Visual Representation of Algorithms

A flowchart is a visual representation of an algorithm. A flowchart is a diagram made up of boxes, diamonds and other shapes, connected by arrows. Each shape represents a step of the solution process and the arrow represents the order or link among the steps.

There are standardised symbols to draw flowcharts.

Some are given in Table 4.1.

Table 4.1 Shapes or symbols to draw flow charts

Flowchart symbol Function Description

Start/End Also called “Terminator” symbol. It indicates where the flow starts and ends.

Process Also called “Action Symbol,” it represents a process, action, or a single step.

Decision A decision or branching point, usually a yes/no or true/false question is asked, and based on the answer, the path gets split into two branches.

Input/Output Also called data symbol, this parallelogram shape is used to input or output data

Arrow Connector to show order of flow between shapes.

Example 4.1: Write an algorithm to find the square of a number.

Before developing the algorithm, let us first identify the input, process and output:

- Input: Number whose square is required
- Process: Multiply the number by itself to get its square
- Output: Square of the number

Algorithm to find square of a number.

Step 1: Input a number and store it to num

Step 2: Compute $\text{num} * \text{num}$ and store it in square

Step 3: Print square

2021-22

Ch 4.indd 66 21-May-19 4:35:17 PM

Introduction to Problem Solving 67

The algorithm to find square of a number can be

Think and Reflect

represented pictorially using flowchart as shown in

Figure 4.2. What will happen if an

algorithm does not stop

after a finite number

of steps?

Activity 4.2

Draw a flowchart

that represents the

attainment of your

career goal.

Figure 4.2: Flowchart to calculate square of a

number

Example 4.2: Draw a flowchart to solve the problem of a non-functioning light bulb

Figure 4.3: Flowchart to solve the problem of a non-functioning light bulb

2021-22

Ch 4.indd 67 08-Apr-19 12:34:19 PM

68 Computer Science – Class xi

4.4.2 Pseudocode

A pseudocode (pronounced Soo-doh-kohd) is another way of representing an algorithm. It is considered as a non-formal language that helps programmers to write algorithm. It is a detailed description of instructions that a computer must follow in a particular order. It is intended for human reading and cannot be executed directly by the computer. No specific standard for writing a pseudocode exists. The word “pseudo” means “not real,” so “pseudocode” means “not real code”. Following are some of the frequently used keywords while writing pseudocode:

- INPUT
- COMPUTE
- PRINT
- INCREMENT
- DECREMENT
- IF/ELSE
- WHILE
- TRUE/FALSE

Example 4.3: Write an algorithm to display the sum of two numbers entered by

Activity 4.3 user, using both pseudocode and

Write a pseudocode for flowchart.

creating a scoreboard Pseudocode for the sum of two numbers will be:

for a hockey match. input num1

input num2

COMPUTE Result = num1 + num2

PRINT Result

The flowchart for this algorithms is given in Figure 4.4.

Result

Print Result

Figure 4.4: Flowchart to display sum of two numbers

2021-22

Ch 4.indd 68 08-Apr-19 12:34:19 PM

Introduction to Problem Solving 69

Example 4.4: Write an algorithm to calculate Notes
area and perimeter of a rectangle,
using both pseudocode and flowchart.

Pseudocode for calculating area and perimeter of
a rectangle.

input length

input breadth

compute $\text{Area} = \text{length} * \text{breadth}$

print Area

compute $\text{Perim} = 2 * (\text{length} + \text{breadth})$

print Perim

The flowchart for this algorithm is given in Figure 4.5.

Figure 4.5: Flowchart to

calculate area and perimeter of a rectangle

(A) Benefits of Pseudocode

Before writing codes in a high level language, a
pseudocode of a program helps in representing the
basic functionality of the intended program. By
writing the code first in a human readable language,
the programmer safeguards against leaving out any
important step. Besides, for non-programmers, actual
programs are difficult to read and understand, but
pseudocode helps them to review the steps to confirm
that the proposed implementation is going to achieve
the desire output.

2021-22

Ch 4.indd 69 08-Apr-19 12:34:20 PM

70 Computer Science – Class xi

4.5 Flow of Control

The flow of control depicts the flow of events as
represented in the flow chart. The events can flow in
a sequence, or on branch based on a decision or even
repeat some part for a finite number of times.

4.5.1 Sequence

If we look at the examples 4.3 and 4.4, the statements

are executed one after another, i.e., in a sequence.

Such algorithms where all the steps are executed one after the other are said to execute in sequence. However, statements in an algorithm may not always execute in a sequence. We may sometimes require the algorithm to either do some routine tasks in a repeated manner or behave differently depending on the outcomes of previous steps. In this section, we are going to learn how to write algorithms for such situations.

4.5.2 Selection

Consider the map of a neighbourhood as shown in Figure 4.6. Let us assume that the pink building with Think and Reflect the red roof is the school; the yellow painted house at Can you list some of the the far end of the map is a house. routine activities in your daily life where decision making is involved?

Figure 4.6: Decision making in real life

2021-22

Ch 4.indd 70 08-Apr-19 12:34:20 PM

Introduction to Problem Solving 71

With reference to Figure 4.6, let us answer the following Notes questions :

- Is there a predefined route for walking from home to school?

- Can we have a different route while coming back?

As seen from the map, there can be multiple routes between home and school. We might take the shortest route in the morning. But while coming back home in the afternoon, the shortest route might have heavy traffic. Therefore, we could take another route with less traffic. Hence, the above problem involves some decision-making based on certain conditions.

Let us look at some other examples where decision making is dependent on certain conditions. For example, (i) Checking eligibility for voting.

Depending on their age, a person will either be allowed to vote or not allowed to vote:

- If age is greater than or equal to 18, the person is eligible to vote

- If age is less than 18, the person is not eligible to vote

(ii) Let us consider another example

If a student is 8 years old and the student likes Maths

put the student in Group A

Otherwise

Put the student in Group B

In which group will these students go as per the above condition?

Outcome

- 8-year-old Ravi who does not like Maths: Group B
- 8-year-old Priti who likes Maths: Group A
- 7-year-old Anish who likes Maths: Group B

In these examples, any one of the alternatives is selected based on the outcome of a condition. Conditionals are used to check possibilities. The program checks one or more conditions and perform operations (sequence of actions) depending on true or false value of the condition. These true or false values are called binary values.

2021-22

Ch 4.indd 71 08-Apr-19 12:34:20 PM

72 Computer Science – Class xi

Conditionals are written in the algorithm as follows:

If <condition> then

steps to be taken when the condition is true/fulfilled

There are situations where we also need to

Figure 4.7: Actions depending on true or take action when the condition is not fulfilled false of a condition (Figure 4.7). To represent that, we can write:

If <condition> is true then

steps to be taken when the condition is true/fulfilled

otherwise

steps to be taken when the condition is false/not fulfilled

In programming languages, 'otherwise' is represented using Else keyword. Hence, a true/false conditional is written using if-else block in actual programs.

Example 4.5: Let us write an algorithm to check whether a number is odd or even.

- Input: Any number
- Process: Check whether the number is even or not
- Output: Message “Even” or “Odd”

Pseudocode of the algorithm can be written as follows:

```
PRINT "Enter the Number"
```

```
INPUT number
```

```
IF number MOD 2 == 0 THEN
```

```
PRINT "Number is Even"
```

```
ELSE
```

```
PRINT "Number is Odd"
```

The flowchart representation of the algorithm is shown in Figure 4.8.

Figure 4.8: Flowchart to check whether a number is even or odd
2021-22

Ch 4.indd 72 08-Apr-19 12:34:20 PM

Introduction to Problem Solving 73

Example 4.6: Let us write a pseudocode and draw a flowchart where multiple conditions are checked to categorise a person as either child (<13), teenager (≥13 but <20) or adult (≥20), based on age specified:

- Input: Age
- Process: Check Age as per the given criteria
- Output: Print either “Child”, “Teenager”, “Adult”

Pseudocode is as follows:

```
INPUT Age
```

```
if Age < 13 then
```

```
PRINT "Child"
```

```
else if Age < 20 then
```

```
PRINT "Teenager"
```

```
else
```

```
PRINT "Adult"
```

The flowchart representation of the algorithm is shown in Figure 4.9

Figure 4.9: Flowchart to check multiple conditions

Example 4.7: Algorithm for a card game called “Dragons and Wizards”.

Make two teams DRAGONS and WIZARDS

The rules for the game are as follows:

- If the card drawn is a diamond or a club, Team

DRAGONS gets a point

- If the card drawn is a heart which is a number,

Team WIZARDS gets a point

2021-22

Ch 4.indd 73 08-Apr-19 12:34:20 PM

74 Computer Science – Class xi

Notes • If the card drawn is a heart that is not a number,

Team DRAGONS gets a point

- For any other card, Team WIZARDS gets a point
- The team with highest point is the winner

Let us identify the following for a card:

Input: shape, value

Process: Increment in respective team scores by one based on the outcome of the card drawn, as defined in the rules.

Output: Winning team

Now let us write the conditionals for this game:

IF (shape is diamond) OR (shape is club)

Team DRAGONS gets a point

ELSE IF (shape is heart) AND (value is number)

Team WIZARDS gets a point

ELSE IF (shape is heart) AND (value is not a number)

Team DRAGONS gets a point

ELSE

Team WIZARDS gets a point

The pseudocode for the program can be as follows:

Note: Dpoint (for Dragon) and Wpoint (for Wizard) store points scored by the respective teams.

INPUT shape

INPUT value

SET Dpoint = 0, Wpoint = 0

IF (shape is diamond) OR (shape is club) THEN

INCREMENT Dpoint

ELSE IF (shape is heart) AND (value is number) THEN

INCREMENT Wpoint

ELSE IF (shape is heart) AND (value is not a number) THEN

INCREMENT Dpoint

```

ELSE
INCREMENT Wpoint
END IF
If Dpoint > Wpoint THEN
PRINT "Dragon team is the winner"
ELSE
PRINT "Wizard team is the winner"

```

4.5.3 Repetition

When giving directions to go someplace, we say something like, “walk 50 steps then turn right”. Or “Walk till the next

2021-22

Ch 4.indd 74 08-Apr-19 12:34:20 PM

Introduction to Problem Solving 75

crossing then take a right turn”. Consider some other

Think and Reflect

examples like:

- Clap your hands five times Can you list some of the routine activities
- Walk 10 steps ahead in your daily life where
- Jump on the spot till you get tired repetition or iteration is

These are the kind of statements we use, when involved?

we want something to be done repeatedly, for a given number of times. Likewise, suppose 10 cards need to be withdrawn in the previous card game (example 4.7), then the pseudocode needs to be repeated 10 times to decide the winner. All these are examples of repetitions.

In programming, repetition is also known as iteration or loop. A loop in an algorithm means execution of some program statements repeatedly till some specified condition is satisfied.

Example 4.8: Write pseudocode and draw a flowchart to accept 5 numbers and find their average.

The flowchart representation is shown in Figure 4.10.

Pseudocode will be as follows:

Step 1: Set count = 0, sum = 0

Step 2: While count < 5 , repeat steps 3 to 5

Step 3: Input a number to num

Step 4: sum = sum + num

Step 5: count = count + 1

Step 6: Compute average = sum/5

Step 7: Print average

In example 4.8, a

counter called “count”

keeps track of number

of times the loop

has been repeated.

After every iteration of

the loop, the value

of count is incremented

by 1 until it performs

the set number of

repetitions, given in

the iteration condition.

There are situations

when we are not aware

beforehand about the

number of times a set Figure 4.10: Flowchart to Calculate the Average of 5 Numbers

2021-22

Ch 4.indd 75 08-Apr-19 12:34:21 PM

76 Computer Science – Class xi

of statements need to be repeated. Such requirements

of unknown number of repetitions are handled using

WHILE construct.

Example 4.9: Write pseudocode and draw flowchart to

accept numbers till the user enters 0 and

then find their average.

Pseudocode is as follows:

Step 1: Set count = 0, sum = 0

Step 2: Input num

Step 3: While num is not equal to 0, repeat Steps 4 to 6

Step 4: sum = sum + num

Step 5: count = count + 1

Step 6: Input num

Step 7: Compute average = sum/count

Step 8: Print average

The flowchart representation is shown in Figure 4.11.

Activity 4.4

Let us answer the

following questions

using the pseudocode
given in example 4.9:

1) What will the sum
when the input are 6,
7, 4, 8, 2, 5, 0, 3, 1.

2) What will be the
value of count?

3) Why did we use the

input statement to Figure 4.11: Flowchart to accept numbers till the user enters 0
enter num twice?

4) Why did we divide In this example, we do not know how many numbers
sum by count? a user is going to enter before entering 0. This is handled

5) Can there be any by checking the condition repeatedly till the condition
other approach?
becomes false.

2021-22

Ch 4.indd 76 08-Apr-19 12:34:21 PM

Introduction to Problem Solving 77

4.6 Verifying Algorithms Think and Reflect

Can you imagine what would happen if a banking software Why is verification of
does not work correctly? Suppose functioning of the online algorithm an important
money transfer module is not programmed correctly, and step in problem solving?
it credits into the account only half the amount transacted!

What happens if the account is debited instead of being
credited. Such a faulty software will mess up the working
of the complete system and cause havoc! Today software
are used in even more critical services — like in the
medical field or in space shuttles. Such software needs to
work correctly in every situation. Therefore, the software
designer should make sure that the functioning of all the
components are defined correctly, checked and verified in
every possible way.

When we were told that the formula for the sum of
first N natural numbers is $N(N+1)$, how did we verify it?

2

Well, we can check this for small numbers, for which
we can manually calculate the sum. Let $N = 6$, then the Activity 4.5
sum is $1 + 2 + 3 + 4 + 5 + 6 = 21$ Write an algorithm

Using formula we get sum = $6 \times (6+1)$ 2

to take as input the
measurement of length

We can try with some more numbers this way to and breadth in feet and ensure that the formula works correctly. inches (e.g., 5 ft 6 inch) In the same way, when we have written an algorithm, of a rectangular shape we want to verify that it is working as expected. To verify, and calculate its area and perimeter.

we have to take different input values and go through all the steps of the algorithm to yield the desired output for each input value and may modify or improve as per the need. The method of taking an input and running through the steps of the algorithm is sometimes called dry run. Such a dry run will help us to:

1. Identify any incorrect steps in the algorithm
2. Figure out missing details or specifics in the algorithm

It is important to decide the type of input value to be used for the simulation. In case all possible input values are not tested, then the program will fail. What if there is some other case for which it does not work? Let us look at some examples.

Write an algorithm to calculate the time taken to go from place A to C (T_{total}) via B where time taken to

2021-22

Ch 4.indd 77 21-May-19 11:51:30 AM

78 Computer Science – Class xi

Notes go from A to B (T₁) and B to C (T₂) are given. That is, we want the algorithm to add time given in hours and minutes. One way to write the algorithm is:

PRINT value for T₁

INPUT hh1

INPUT mm1

PRINT value for T₂

INPUT hh2

INPUT mm2

hh_{total} = hh1 + hh2 (Add hours)

mm_{total} = mm1 + mm2 (Add mins)

Print T_{total} as hh_{total}, mm_{total}

Now let us verify. Suppose the first example we take is T₁ = 5 hrs 20 mins and T₂ = 7 hrs 30 mins. On dry run, we get the result 12 hrs and 50 mins. This looks fine.

Now let us take another example where $T1 = 4$ hrs 50 mins and $T2 = 2$ hrs 20 mins, and we end up getting the result as 6 hrs 70 mins which is not how we measure time. The result should have been 7 hrs 10 mins.

With this second example we realise that our algorithm will work only when $mm1 + mm2$ (mm_total) < 60 . For all other cases, it will give us output not the way we want. When $mm_total \geq 60$, the algorithm should increase the sum of hours (hh_total) by 1 and reduce mm_total by 60, i.e., $(mm_total - 60)$. So the modified algorithm will be:

PRINT value for T1

INPUT hh1

INPUT mm1

PRINT value for T2

INPUT hh2

INPUT mm2

$hh_total = hh1 + hh2$ (Add hours)

$mm_total = mm1 + mm2$ (Add mins)

$hh_total = hh1 + hh2$ (Add hours)

$mm_total = mm1 + mm2$ (Add mins)

IF ($mm_total \geq 60$) THEN

$hh_total = hh_total + 1$

$mm_total = mm_total - 60$

PRINT T_total as hh_total , mm_total

Now we can simulate through algorithm for $T1 = 4$ hrs 50 mins and $T2 = 2$ hrs 20 mins, and get $T_total = 7$ hrs and 10 mins, which means the algorithm is working correctly.

2021-22

Ch 4.indd 78 21-May-19 11:55:04 AM

Introduction to Problem Solving 79

Suppose we develop some software without verifying Notes the underlying algorithm and if there are errors in the algorithm, then the software developed will not run. Hence, it is important to verify an algorithm since the effort required to catch and fix a mistake is minimal.

4.7 Comparison of Algorithm

There can be more than one approach to solve a problem

using computer and hence we can have more than one algorithm. Then one may ask which algorithm should be used?

Consider the problem of finding whether a given number is prime or not. Prime numbers are of great importance in computer science as they find application in databases, security, file compression or decompression, modulation or demodulation, etc. There can be four different ways to write algorithms to check whether a given number is prime or not as shown below:

- (i) Starting with divisor 2, divide the given number (dividend) and check if there are any factors. Increase the divisor in each iteration and repeat the previous steps as long as $\text{divisor} < \text{dividend}$. If there is a factor, then the given number is not prime
- (ii) In (i), instead of testing all the numbers till the dividend, only test up to half of the given value (dividend) because the divisor can not be more than half of the dividend
- (iii) In method (i), only test up to the square root of the dividend (numbers)
- (iv) Given a prior list of prime number till 100, divide the given number by each number in the list. If not divisible by any number, then the number is a prime else it is not prime

All these four methods can check if a given number is prime or not. Now the question is which of these methods is better or efficient?

Algorithm (i) requires large number of calculations (means more processing time) as it checks for all the numbers as long as the divisor is less than the number. If the given number is large, this method will take more time to give the output.

2021-22

Ch 4.indd 79 08-Apr-19 12:34:21 PM

80 Computer Science – Class xi

Algorithm (ii) is more efficient than (i) as it checks for divisibility till half the number, and thus it reduces the

The spirit of time for computation of the prime number. Algorithm problem solving by (iii) is even more efficient as it checks for divisibility till

decomposition is to

‘divide and conquer’.

square root of the number, thereby further reducing the

In words of Howard time taken.

Raffa, a famous As algorithm (iv) uses only the prime numbers smaller mathematician:

than the given number for divisibility, it further reduces

“Decompose a complex the calculations. But in this method we require to store problem into simpler

problems get one’s thinking the list of prime numbers first. Thus it takes additional straight in these simpler memory even though it requires lesser calculations.

problems, put these Hence, algorithms can be compared and analysed analyses together with

logical glue”

on the basis of the amount of processing time they need to run and the amount of memory that is needed to execute the algorithm. These are termed as time complexity and space complexity, respectively. The choice of an algorithm over another is done depending on how efficient they are in terms of proecssing time required (time complexity) and the memory they utilise (space complexity).

4.8 Coding

Once an algorithm is finalised, it should be coded in a high-level programming language as selected by the programmer. The ordered set of instructions are written in that programming language by following its syntax. Syntax is the set of rules or grammar that governs the formulation of the statements in the language, such as spellings, order of words, punctuation, etc.

The machine language or low level language consisting of 0s and 1s only is the ideal way to write a computer program. Programs written using binary digits are directly understood by the computer hardware, but they are difficult to deal with and comprehend by humans. This led to the invention of high-level languages which are close to natural languages and are easier to read, write, and maintain, but are not directly understood by the computer hardware. An advantage of using high-level languages is that they are portable, i.e., they can run on different types of computers with little

2021-22

Ch 4.indd 80 08-Apr-19 12:34:21 PM

Introduction to Problem Solving 81

or no modifications. Low-level programs can run on Notes only one kind of computer and have to be rewritten in order to run on another type of system. A wide variety of high-level languages, such as FORTRAN, C, C++, Java, Python, etc., exist.

A program written in a high-level language is called source code. We need to translate the source code into machine language using a compiler or an interpreter, so that it can be understood by the computer. We have learnt about the compiler and interpreter in Chapter 1. There are multiple programming languages available and choosing the one suitable for our requirements requires us to consider many factors. It depends on the platform (OS) where the program will run. We need to decide whether the application would be a desktop application, a mobile application or a web application. Desktop and mobile applications are generally developed for a particular operating system and for certain hardware whereas the web applications are accessed in different devices using web browsers and may use resources available over cloud.

Besides, programs are developed not only to work on a computer, mobile or a web browser, but it may also be written for embedded systems like digital watch, mp3 players, traffic signals or vehicles, medical equipments and other smart devices. In such cases, we have to look for other specialised programming tools or sometimes write programs in assembly languages.

4.9 Decomposition

Sometimes a problem may be complex, that is, its solution is not directly derivable. In such cases, we need to decompose it into simpler parts. Let us look at the Railway reservation system we talked about earlier. The complex task of designing a good railway reservation system is seen as designing the different components of the system and then making them work with each other effectively.

The basic idea of solving a complex problem by

decomposition is to 'decompose' or break down a complex problem into smaller sub problems as shown

2021-22

Ch 4.indd 81 08-Apr-19 12:34:21 PM

82 Computer Science – Class xi

in Figure 4.12. These sub problems are relatively easier to solve than the original problem. Finally, the sub-problems are combined in a logical way to obtain the solution for the bigger, main problem.

Trains' information Reservation information Information about — days, timings, — booking open or close, staff, security, stations, classes and available or waiting List, railway births cancellation and refund infrastructure

Other details about

Food service Billing service

railways

Figure 4.12: Railway reservation system

Breaking down a complex problem into sub problems also means that each sub problem can be examined in detail. Each sub problem can be solved independently and by different persons (or teams). Having different teams working on different sub problems can also be advantageous because specific sub problems can be assigned to teams who are experts in solving such problems.

There are many real life problems which can be solved using decomposition. Examples include solving problems in mathematics and science, events management in school, weather forecasting, delivery management system, etc.

Once the individual sub problems are solved, it is necessary to test them for their correctness and integrate them to get the complete solution.

Summary

- An algorithm is defined as a step-by-step procedure designed to perform an operation which will lead to the desired result, if followed correctly.
- Algorithms have a definite beginning and a definite end, and a finite number of steps.
- A good algorithm, which is precise, unique and

finite, receives input and produces an output.

2021-22

Ch 4.indd 82 08-Apr-19 12:34:21 PM

Introduction to Problem Solving 83

- In order to write effective algorithms we need to identify the input, the process to be followed and the desired output.
- A flowchart is a type of diagram that represents the algorithm graphically using boxes of various kinds, in an order connected by arrows.
- An algorithm where all the steps are executed one after the other is said to execute in sequence.
- Decision making involves selection of one of the alternatives based on outcome of a condition.
- An algorithm may have a certain set of steps, which are repeating for a finite number of times, such an algorithm is said to be iterative.
- There can be more than one approach to solve a problem and hence we can have more than one algorithm for a particular problem.
- The choice of algorithm should be made on the basis of time and space complexity.

Exercise

1. Write pseudocode that reads two numbers and divide one by another and display the quotient.
2. Two friends decide who gets the last slice of a cake by flipping a coin five times. The first person to win three flips wins the cake. An input of 1 means player 1 wins a flip, and a 2 means player 2 wins a flip. Design an algorithm to determine who takes the cake?
3. Write the pseudocode to print all multiples of 5 between 10 and 25 (including both 10 and 25).
4. Give an example of a loop that is to be executed a certain number of times.
5. Suppose you are collecting money for something. You need ` 200 in all. You ask your parents, uncles and aunts as well as grandparents. Different people may give either ` 10, ` 20 or even ` 50. You will collect till the total becomes 200. Write the algorithm.

6. Write the pseudocode to print the bill depending upon the price and quantity of an item. Also print

2021-22

Ch 4.indd 83 08-Apr-19 12:34:21 PM

84 Computer Science – Class xi

Notes Bill GST, which is the bill after adding 5% of tax in the total bill.

7. Write pseudocode that will perform the following:

- a) Read the marks of three subjects: Computer Science, Mathematics and Physics, out of 100
- b) Calculate the aggregate marks
- c) Calculate the percentage of marks

8. Write an algorithm to find the greatest among two different numbers entered by the user.

9. Write an algorithm that performs the following:

Ask a user to enter a number. If the number is between 5 and 15, write the word GREEN. If the number is between 15 and 25, write the word BLUE. if the number is between 25 and 35, write the word ORANGE. If it is any other number, write that ALL COLOURS ARE BEAUTIFUL.

10. Write an algorithm that accepts four numbers as input and find the largest and smallest of them.

11. Write an algorithm to display the total water bill charges of the month depending upon the number of units consumed by the customer as per the following criteria:

- for the first 100 units @ 5 per unit
- for next 150 units @ 10 per unit
- more than 250 units @ 20 per unit

Also add meter charges of 75 per month to calculate the total water bill .

12. What are conditionals? When they are required in a program?

13. Match the pairs

Flowchart Symbol Functions

Flow of Control

Process Step

Start/Stop of the

Process

2021-22

Ch 4.indd 84 08-Apr-19 12:34:21 PM

Introduction to Problem Solving 85

Notes

Data

Decision Making

14. Following is an algorithm for going to school or college. Can you suggest improvements in this to include other options?

Reach_School_Algorithm

- a) Wake up
- b) Get ready
- c) Take lunch box
- d) Take bus
- e) Get off the bus
- f) Reach school or college

15. Write a pseudocode to calculate the factorial of a number (Hint: Factorial of 5, written as $5! = 5 \times 4 \times 3 \times 2 \times 1$).

16. Draw a flowchart to check whether a given number is an Armstrong number. An Armstrong number of three digits is an integer such that the sum of the cubes of its digits is equal to the number itself. For example, 371 is an Armstrong number since $3^3 + 7^3 + 1^3 = 371$.

17. Following is an algorithm to classify numbers as "Single Digit", "Double Digit" or "Big".

Classify_Numbers_Algo

INPUT Number

IF Number < 9

" Single Digit"

Else If Number < 99

" Double Digit"

Else

" Big"

Verify for (5, 9, 47, 99, 100 200) and correct the algorithm if required

18. For some calculations, we want an algorithm that accepts only positive integers upto 100.

2021-22

Ch 4.indd 85 08-Apr-19 12:34:21 PM

86 Computer Science - Class xi

Notes Accept_1to100_Algo

INPUT Number

IF ($0 \leq \text{Number}$) AND ($\text{Number} \leq 100$)

ACCEPT

Else

REJECT

a) On what values will this algorithm fail?

b) Can you improve the algorithm?

2021-22

Ch 4.indd 86 08-Apr-19 12:34:21 PM

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

CSC 122

Introduction to Problem Solving

with Python Programming

Arnold A. OJUGO (PhD)

Lecture 01 - Introduction

These slides are based on textbooks and other materials accompanying this course

4/21/2021 1

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Organization

□ Course Lecturer(s)

- Dr. Arnold Adimabua OJUGO

- Lecture Time: Mondays (12noon – 1pm)

- Laboratory Time: Mondays (1 – 2pm)

- Consultation Time: Mondays (2 – 3pm)

- Website: <https://sites.google.com/sites/aaojugo/home>

□ Unit Website: (In Progress)

□ Two Lectures/Week (1 hour and 1 hour recitation each): The Lectures may be recorded. When so, it will be available (on demand). However, lecture slides will be available on the Unit Website and on Piazza. And, will be updated after lectures. Please, check the Unit website for announcements and other information. When an announcement is made in one place, it is assumed that you are aware of it.

□ One Lab/Week (1 hour): It is ideal to have a Laptop/Desktop to accompany you through the course.

□ One Workshop/One Seminar per Semester (1 Hour)

4/21/2021 2

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Text & Other Materials

□ Python Programming (Materials)

- Introduction to Programming using Python by Daniel Y. Liang
- Learning to Program with python by Richard L. Halterman
- Head-First Programming: A Learner's guide to programming using the Python Language by Paul Barry & David Griffiths
- Python Programming by Mark Lutz

□ Videos Lectures (Available):

- Lectures on Introduction to Computer Science and Programming in Python by Ana Bell
- Lectures on Introduction to Computer Science and Programming by John Guttag

□ Online Materials:

- Freely download other materials online to help you through this course
- We you are confused, ask questions for guidance on how to navigate this course.
- In the course of the lecture, be free to ask the instructor questions at any point; But, please, be sure the question(s) is interestingly intuitive enough that other students can learn as they participate.

□ Development Environment

- Python 2.5 / 3.3 or above with IDLE which you can download from python.org and install on your home computer (as we already have it installed in the Lab Computers)

4/21/2021 3

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Assessment

□ Assessment is based on both

- Understanding of the Fundamental Concepts
- Practical Problem Solving and Programming Skills
- Presentation
- Question formulation and Answer Participation in Classes during the Lecture

□ Mid-Semester Test(s) and Assignments

- Assignments as given will be collected and assessed. But, mid-semester test will be due at Week 8 and it is worth (10%).

□ Projects

- Project 1 due at the end of Week 5 worth (10%)

- Project 2 due at the end of Week 10 worth (10%)

□ Final Exams...as scheduled by the University is worth (70%). It culminates every content of the course as taught.

4/21/2021 4

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Let Us Begin.....

4/21/2021 5

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Introduction / Objectives

□ The course is generally about Problem Solving, and we would view it from both the aspect of a Computer Scientist and a Mathematician. Also,

- Python will be used as the Platform

- I do not assume that you are good or great with programming. Thus, there are no prerequisites for this course as I intend to make scholars out of all of You here.

- However, I would assume that you are undergoing through MTH111 and O-Level Mathematics

□ The course covers:

- Important problem solving techniques used widely in computer science and programming

- Writing basic programs in Python (a modern high level language)

- An introduction to software engineering

□ Lectures, Labs and Projects will complement each other in this course.

□ Objectives of this Chapter 1 – Introduction is to:

1. To help us understand the roles of hardware and software in a computing system

2. To learn what computer scientists study, why they study these things and the techniques they use in the course of studying these things.

3. To understand the basic design of a modern computer

4. To understand the form and function of programming languages

4/21/2021 6

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

The universal Machine

□ From “introduction to computer science” – the computer is an electronic device that can easily accept data as input, processes these data based on some logically-ordered set of instructions (which tells the hardware what to do), outputs our desired result as information – and stores them for later

retrieval.

□ Thus, we can simply put it that: a computer is a machine that stores and manipulates information under the control of a changeable program.

□ The computer touches every area of our lives – from paying bills, driving cars, telephones etc

□ The 2-elements here being:

- Computers are devices for manipulating data, which then yields information
- Computers operate under the control of a changeable program.

□ What is Programming?

1. Human behaviour and thoughts are based on a sequence of actions applied to objects. From infancy, you constantly learn how to act, do things as well as what behaviours to expect from other people as events unfold.
2. On broader scale, mathematics could never have been developed without logical sequences of steps for manipulating symbols to solve problems and prove theorems. Mass production would never have worked without operations taking place on component parts in certain order.
3. Note that our whole civilization is based on the order of things and actions – yielding effective result in the society.

4/21/2021 7

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Programming The universal Machine

□ We create order in the society both consciously and unconsciously – via the art or process called programming. Our concern in this course is to see how we solve problems (via the art programming) and how these translates into our actions and better our society. Note that:

1. Computers manipulate data – just like mathematics manipulates symbols to solve tasks.
2. When you write a program (plan) for a computer – you are simply specifying properties of the data and the operations that can be applied to it.
3. These operations can be combined as necessary to resolve the task.
4. Data is information in a form that computers can use. E.g. numbers, letters etc; While, on a larger scale, information is any knowledge that can be communicated including abstract ideas and concepts such as the “Earth is spherical” and the speed of light is $c = 3 \times 10^8$ m/s.

□ Useful Information: It is thus great to know that

1. The IQ level of the Computer is 0 (zero)
2. Everyone can program – because every one of us here can write the recipe for any food we wish to make
3. Programming often can start slow; But, in time – it will soon overwhelm Us.
4. Virtually any kind of information can be represented as data, or as a combination of data and operations on it.
5. Each kind of data in the computer has a specific type...which we will unfold later.

4/21/2021 8

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Programming: concepts & definitions

□ We have these definitions then:

1. Programming – is the planning and scheduling the performance of a task or an event.
2. Electronic computer – is a programmable device that can accept data as input, process these data to yield information based on some logically ordered set of instructions, output the desired result and stores them for later retrieval.
3. Data is information in a form that the computer can use; Information is any knowledge that can be communicated. Data comes in various forms such as letters (char), words (strings), integer (whole numbers), dates, coordinates on a map etc
4. Data type – is the specification of how information is represented in the computer as data and the set of operations that can be applied to it.
5. Computer programming – is the process of specifying the data types and the operations for a computer to apply to data in order to solve a problem.
6. Computer program – is the data type specification and instructions for carrying out operations that are used by a computer to solve a problem. Thus, when we use the words programming and programs – we actually mean computer programming and computer programs.
7. The computer allows us to perform tasks more efficiently, quickly and accurately. But for it to be a more powerful tool, it requires to be programmed.

4/21/2021 9

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

What is a Computer Program?

- A detailed, step-by-step set of instructions that tells the computer hardware what to do. The human programmer must analyze a problem and come up with solutions – because the computer is not intelligent. The advantage of using the computer is that it can repeat these programmed solution very quickly and consistently. Thus, it frees us from repetitive and boring tasks.
- A programming language is a simplified form of English (with math symbols) that adheres to a set of strict grammatical rules. If we change the program, the computer simply performs a different task or set of actions.
- The machine stays the same; But the program changes. The machine performs the same actions or task so long the program does not change; Else, it performs the task it is programmed to carry out.
- Programs are thus, executed are carried out. Most computers have similar

capabilities. With a suitable program, each computer can do the things other computers can do.

□ The same program will produce similar results on different computers. We then categorically say – software (programs) rules or lords the hardware (physical machine). The process of creating this software is called programming.

4/21/2021 10

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Why learn to Program?

□ A code is product of translating an algorithm into a programming language. Programming is a fundamental part of computer science and mathematics.

□ An understanding of programming helps us to understand the strengths and limitations of the computer.

□ It is expected to be an everyday part of your life that is accompanied by the likes of reading and writing.

□ It is also funtastic...

□ How it all Began

- Alan Turing first proposed the idea of a programmable machine
- He is considered the Father of Computer Science
- He developed a machine in world War II to crack the German ciphers
- Watch the movie “The Imitation Game”which I can give you by the way

4/21/2021 11

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

What is Computer Science?

□ It is NOT the study of computers!

- ‘Computers are to computer science, what chemicals are to chemists and what machines are to mechanical engineers’... Edsger Dijkstra
- If a computer can carry out any process, the question then should be – ‘what process can we describe...?’
- Surely, the true question is – ‘what can be computed...?’

□ Computer Scientists find answers to questions through:

- Design – a way to show that a particular problem can be solved is to actually design a workable and feasible solution.

1. This is done by developing an algorithm. An algorithm is simply a recipe as we did earlier.

2. Algorithm is step-by-step process to achieve desired solution. This course

will teach us how to design an algorithm and write a program for it...

- Analysis – examines the algorithm and problems mathematically because some seemingly simple problems are unsolvable by any algorithm
- Experimentation – aims to implement a system and then study its behaviour.

Thus, it helps: (a) verify the analysis and (b) refine the analysis

4/21/2021 12

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Example algorithm: Recipe to boil an Egg

- Step 1: Keeps your eggs at room temperature before boiling. If the eggs are too cold, the shells may crack during cooking.
- Step 2: Place the eggs in a saucepan of cold water. Place Pan over medium heat oven. Bring to a gentle simmer, gently stirring the eggs constantly in a clockwise direction. The movement of the water helps to center the egg yolks.
- Step 3: Simmer the eggs for 4 minutes for soft-boiled eggs. Simmer for 5-minutes if you wish it to be with semi-firm yolks and hard whites; Else, simmer for 8-minutes for hard-boiled eggs. Use a slotted spoon to remove the egg from the water. Transfer to an egg cup and serve immediately.

Source: <http://www.taste.com.au/how+to/articles/2508/how+to+boil+eggs>

- Step 1: Keeps your eggs at room temperature before boiling. If the eggs are too cold, the shells may crack during cooking.
- Step 2: Place the eggs in a saucepan of cold water. Place Pan over medium heat oven. Bring to a gentle simmer, gently stirring the eggs constantly in a clockwise direction. The movement of the water helps to center the egg yolks.
- Step 3: Simmer the eggs for 4 minutes for soft-boiled eggs. Simmer for 5-minutes if you wish it to be with semi-firm yolks and hard whites; Else, simmer for 8-minutes for hard-boiled eggs. Use a slotted spoon to remove the egg from the water. Transfer to an egg cup and serve immediately.

4/21/2021 13

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Recipe to Fry an Egg

- Step 1: Keeps eggs at room temperature.
- Step 2: Slice onion, tomatoes and pepper into a bowl to form sauce.
- Step 3: Place saucepan over medium heat over, pour oil into pan and allow to heat
- Step 4: Whisk the egg in a separate bowl & add a pinch of salt into the whisked egg
- Step 5: For Omlet: mix sliced sauce with whisked egg; For Scrambled eggs, pour sauce into heated oil, pour egg next and stir gently in clockwise direction.

□ For Bacon: We do step (1), (3) and crack the egg straight into the pan; while the oil has heated to the desired temperature.

Problems

- Step 1: Keeps eggs at room temperature.
- Step 2: Slice onion, tomatoes and pepper into a bowl to form sauce.
- Step 3: Place saucepan over medium heat oven, pour oil into pan and allow to heat
- Step 4: Whisk the egg in a separate bowl & add a pinch of salt into the whisked egg
- Step 5: For Omlet: mix sliced sauce with the whisked egg; For Scrambled eggs, pour the sauce into heated oil, pour the egg next and stir gently in clockwise direction (as this helps the egg to mix properly).
- For Bacon: We do step (1), (3) and crack the egg straight into the pan; while the oil has heated to the desired temperature.

4/21/2021 14

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Hardware Basics

- The CPU as the brain of the computer
 - Performs all basic operations on data...ranging from simple arithmetic to complex comparative tests,
 - And makes possible, direct access to data from the main memory (R/WM)
- Input Devices pass data to the computer. E.gs keyboard, mouse, camera, microphone, stylus, light-pen etc
- Output Devices pass data back to a user. E.g. is visual display unit (VDU), printers, COM, speakers etc
- Memory stores program and data
 - R/WM is fast, volatile etc...all data is lost when power is lost
 - Secondary memory provides more permanent storage (non-volatile) such as (a) Magnetic (hard drives, USB drives), and (b) Optical (CD, DVD, Blue Ray Discs etc)

4/21/2021 15

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Basic Programming

- The Fetch-Execute Cycle
 1. Loads programs into the main memory (R/WM)
 2. Fetches the next instruction from memory
 3. Decode the next instruction to see what it represents
 4. Carry out the appropriate instructions

□ Programming Languages are used to describe algorithms in a precise way that Natural Languages cannot.

- Every structure in a programming language has a precise form called syntax.
- Every structure in a programming language has a precise meaning called its semantics.
- Programming languages is like a code for writing instructions that computers can understand, follow and execute. Thus, they are often or sometimes called computer codes.
- Process of writing an algorithm in a programming language is called coding.

□ There are 2-types of Programming Languages: (a) low-level that is written as mnemonics and understood only by the hardware as machine or target codes, and (b) High-level language – are designed to be understood and written by human using English-like statements.

4/21/2021 16

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Basic Programming

□ High-level Language: In this form, addition of 2-numbers can be addressed as:

$C = a + b;$

□ This instruction needs to be translated to machine language so that the computer can execute it. To achieve this, the computer has 3-types of translators namely:

1. Compilers convert programs from a source language into its equivalent target machine code
2. Interpreters simulate a computer that understands high level codes. The source code is not translated (compiled) into machine code all at once. All interpreters analyze and executes the source code instructions-by-instruction.

3. Assemblers

□ Low-Level code of the statement as above “ $c = a+b$ ” corresponds to the statements below. In reality, these low level instructions are represented in binary (0s and 1s)

- Load the number from memory location 1001 into the CPU
- Load the number from memory location 1002 into the CPU
- Add the two numbers in the CPU
- Store the result into the location 1003

4/21/2021 17

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Compiler vs Interpret

- Once program is compiled, it can be executed without the source code or compiler. □ The source code and interpreter are needed each time the program code or compiler is to be executed.
- Compiled programs run faster □ Interpreted programs run slower since their translation happens only due to the interpretation each time once it is executed.
- Program needs to be compiled □ More flexible programming after every minor change in the environment since the programs codes can be developed and run interactively
- A compiled program will not run across the different OS platforms □ Interpreted programs are more portable across different OS such as Windows, Mac, Linux etc. Each will be configured to run on platforms such as Windows, Mac, each platform differently. Linux etc

4/21/2021 18

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Python

- Python is an interpreted language.
- If there is a suitable interpreter exists for 2-different machines with very different CPUs, the same program will run on both.
- When you start Python, you may see something as below:
Python 3.4.2 (v3.4.2:ab2c023a9432, Oct 5 2014, 20:42:22) [GCC 4.2.1 (Apple Inc. build 5666) (dot 3)]
Type "copyright", "credits" or "license()" for more information.

- ">>>" is Python prompt. It tells a user that Python is ready for your command. Any command Python receives is called statements. Examples are below we can type:

```
>>> print("Hello, world")
```

- Python displays at the prompt, or on your screen - Hello, world. Also when you type the following statement below at the prompt, Python does the following:

```
>>> print(2+3)
```

```
5
```

```
>>> print("2+3=", 2+3)
```

```
2+3= 5
```

4/21/2021 19

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Supplementary: Python Programming

□ Next, we go to Python Programming. That chapter is aimed at introducing You to Python basics. Note that every one can program – you may start slow; But, eventually – you will become better and may be, feed yourself from it.

□ Often, becoming an experienced programmer involves code copying from textbooks and other materials available. Change them and play around with other codes to see what result it will yield and so on....

□ No one became great in the art of programming on their own – without help from. They either did so through the help of others or through other materials. However, others as well as textbooks cannot help you with, how creative you can become and how well you learn the art. That is strictly YOU.

□ Programming is a necessity – as a computer scientist. You are almost nothing without it.

□ Learning Python and mastery of it – will help you navigate smoothly other languages such as Java, C-family, Haskell and others.

□ In all as in LIFE – prepare for the worse; But expect the best there is...

□ In this supplementary chapter, we would learn how to define functions and use them as well as learn about chaotic models

4/21/2021 20

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Executing single statements

```
>>> 2+3
```

```
5
```

```
>>> 22/7
```

```
3.142857142857143
```

```
>>> 3**2
```

```
9
```

```
>>> print("Hello world")
```

```
Hello world
```

```
>>> print("2+3=", 2+3)
```

```
2+3=5
```

□ All the above are single statement. To solve real world problems, we would often need to generally execute more than one statement. To execute multiple statements, one way to achieve this is through the use of functions:

```
>>> def hello():
```

```
print("Hello")
```

```
print("Computers are Fun")
```

4/21/2021 21

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Define / invoke a Function

```
>>> def hello():  
print("Hello")  
print("Computers are Fun")
```

□ The first line tells Python we are defining a new function called “hello”.

□ The next lines are indented to show they are part of the hello function.

□ The blank line (hit enter/return twice) lets Python know the definition is finished. You also notice that nothing has happened yet! We defined the function; But, we have not told Python to perform the function!

□ A function is invoked or executed by typing its name. So, if we type:

```
>>> hello()
```

Hello

Computers are fun

4/21/2021 22

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

FUNCTIONS: Parameter PARSING

□ Functions can have changeable parts called parameters, placed between the ()’s. The function “hello” has no parameters. We can define the function with one parameter as:

```
>>> def greet(person):  
print("Hello ", person)  
print("How are you?")
```

□ If we try to execute the function “greet()”

```
>>> greet()
```

Traceback (most recent call last):

File "<pyshell#74>", line 1, in <module>

greet()

TypeError: greet() takes exactly 1 argument (0 given)

□ We get an error (as we did not specify the value of “person”). When we use parameters, we can customize the output of a function.

```
>>> greet("Terry")
```

Hello Terry

How are you?

```
>>> greet("Paula")
```

Hello Paula

How are you?

4/21/2021 23

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Module files

- When we exit the Python prompt, all functions that we defined will cease to exist.
- Programs are usually composed of functions, modules or scripts that are saved on disk so that they can be used again.
- A module file is a text file created in text editing software (saved as “plain text”) that contains function definitions.
- A programming environment is designed to help programmers write programs and usually includes automatic indenting, highlighting, etc.

File: chaos.py

A simple program illustrating chaotic behaviour

```
def main():  
    print("This program illustrates a chaotic function")  
    x = eval(input("Enter a number between 0 and 1: "))  
    for i in range(10):  
        x = 3.9 * x * (1 - x)  
        print(x)  
    main()
```

- We use filename.py when we save our work to indicate it's a Python program.
- Above, we are defining a new function called main(). The main() at the end tells Python to run the code.

4/21/2021 24

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Executing Chaos.py

This program illustrates a chaotic function

Enter a number between 0 and 1: .5

0.975

0.0950625

0.335499922266

0.869464925259

0.442633109113

0.962165255337

0.141972779362

0.4750843862

0.972578927537

0.104009713267

□ Inside the Python Program Chaos.py

□ Line that starts with # are called comments. It is intended that users can read it; while the Python program ignores it. It makes clear the intent of that program section.

□ Thus, Python skips text from # to end of the line.

4/21/2021 25

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Inside the Python Program Chaos.py

def main():

□ We begin with definition of a function called main. Since our program has only one module, it could have been written without the main function. However, the use of main is customary.

□ The next line of statement causes Python to print a message introducing the program.

print("This program illustrates a chaotic function")

□ Next, x is declared as a variable, used to assign a name to a value so that we can refer to it later. The quoted information is displayed, and the number typed in response is stored in x.

x = eval(input("Enter a number between 0 and 1: "))

□ The for loop construct tells Python to repeat the same thing over and over. In this example, the following code will be repeated 10 times.

for i in range(10):

□ The body of the loop is what gets repeated each time through the loop. The body of the loop is identified through indentation. The effect of the loop is the same as repeating this two lines 10 times!

x = 3.9 * x * (1 - x)

Print x

4/21/2021 26

FEDERAL UNIVERSITY OF

PETROLEUM RESOURCES

EFFURUN

Inside the Python Program Chaos.py

□ Loop is equivalent to writing the same statement multiple times

for i in range(10) : x = 3.9 * x * (1 - x)

x = 3.9 * x * (1 - x) print (x)

print (x) x = 3.9 * x * (1 - x)

```

print (x)
x = 3.9 * x * (1 - x)
print (x)
x = 3.9 * x * (1 - x)
print (x)
These two statements x = 3.9 * x * (1 - x)
are equivalent print (x)
x = 3.9 * x * (1 - x)
print (x)
x = 3.9 * x * (1 - x)
print (x)
x = 3.9 * x * (1 - x)
print (x)
x = 3.9 * x * (1 - x)
print (x)
x = 3.9 * x * (1 - x)
print (x)

```

4/21/2021 27

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Inside the Python Program - Chaos.py

□ The first statement (below) is an assignment statement. The part of the statement on the RHS of “=” is the mathematical expression that performs the task. Once the value on that RHS is computed, it is stored back into (or assigned) to x:

```
X = 3.9 * x * (1 - x)
```

□ Next statement below main() tells python to execute the code in the function main()
main()

□ For the chaos.py program as below:

```

# File: chaos.py
def main():
    print("This program illustrates a chaotic function")
    x = eval(input("Enter a number between 0 and 1: "))
    for i in range(10):
        x = 3.9 * x * (1 - x)
    print(x)
main()

```

□ For any given input, return 10 times the seemingly random numbers between 0 and 1. It appears that the value x is chaotic. The function computed has the general form $k(x)(1 - x)$ where $k = 3.9$.

□ This type of function is known as logistic function. It models certain kinds of unstable

electronic circuit. A mathematical model is chaotic if very small difference in the initial value can have large difference in the output.

□ From thus, we can model anything we can think about and implement it in Python.

4/21/2021 28

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Execute / import module in Python

□ We can run a program in a module file any time we want using one of the following methods

1. Using IDLE, the easiest way. Select Run □ Run Module from the text editor window (for all operating systems)

2. We can use the shortcut key F5 (windows) and Fn+F5 (Mac OS)

3. In command line (windows) or terminal (Mac OS), enter “python chaos.py” (paths must be specified).

4. You can also double click the .py file in Windows to run it

□ We can also import files into Python. The statements below tells Python interpreter to load the chaos module from the file chaos.py into the main memory. • Since the last statement of chaos.py was “main()” the function main will get executed upon importing chaos.

```
>>> import chaos
```

This program illustrates a chaotic function

Enter a number between 0 and 1: .25

0.73125

4/21/2021 29

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

Execute / import module in Python

□ When Python imports a module, it executes each line.

□ The “def” causes Python to create the “main” function. When Python encounters the last line, it executes the main function.

□ Upon first import, Python creates a companion file with .pyc extension. This is an intermediate file containing the byte-code used by the interpreter.

□ Modules need to be imported in a session only once. We can execute a function defined in a module by typing

```
>>> chaos.main()
```

□ We can define multiple functions in a module file. We already know that we can call a function by typing:

moduleFileName.functionName()

□ To make programming easy, Python will present us with a list of functions in the module when we type the module name and follow it with a dot

4/21/2021 30

FEDERAL UNIVERSITY OF
PETROLEUM RESOURCES
EFFURUN

In Summary

□ Python is an interpreted language. We can execute commands directly in a shell or write a module file.

□ A Python program is a sequence of commands (statements) for the interpreter to execute. It can take input from the user, print output to the screen and run a set of statements multiple times in a loop.

□ A mathematical model is chaotic if very small changes in the input lead to very large changes in the output making it seemingly random. Many real world phenomena exhibit chaotic behaviour and the power of computing is used to model such phenomena.

□ Please do not forget the instructions as given in the first introductory pages. Any further questions?

□ Please feel free to read ahead as it will make the course easier for you.

4/21/2021 31

Paper 3 of 3 — COS 102

Level 100 • Second Semester • Handout

Chapter One

LET THERE BE – JUST...PROBLEMS!

INTRODUCTION

Problem solving is both quite difficult to teach and practice. It is both a science and an art. To learn problem solving, you must do it. As in riding a bicycle, explanations are helpful, but eventually you must do it yourself. You may fall, but that is usually necessary in learning to ride a bike. There are no shortcuts or magic formula to problem solving for a particular event, task or scenario. However, there are techniques, tools and procedures or processes that we can all explore in our quest for ground-truth.

Problem Solving is the science, art and process of generating solutions from observed data.

- A problem is characterized by a set of goals that the problem solver must reach.
- A problem is characterized by a set of objects
- A problem is characterized also by a set of operations
- A problem must have a set of states, which must start at some point S (that is, the initial state), and terminates or ends its actions when the conditions of a final point G (that is, the goal state) is reached.
- A problem must have series of action that the problem solver must move with or take to reach the goal state.
- The plausible solution to any problem must have a cost or weight associated with each action taken or explored.
- Each action must be accompanied by a penalty – which is associated to the task or problem if the problem solver takes the wrong move or jumps off to a state that does not lead to a possible solution.

A problem space is an abstract domain that encompasses all valid states that can be generated by the application of any combination or operators on any combination of objects to yield the desired solution. The problem space may also contain one/more solutions. The solution is a combination of operations and objects that achieves the goal state. A search refers to the act of navigating the problem space in quest for ground-truth or solution. The search proceeds with

different types of search control strategies, which will be mentioned in a later section.

Defining a Problem

What then is a Problem? Defining it is not so easy. A problem exists when what is currently happening – often differs from what should be happening. This is so the case because, what is happening is the current state, and what should happen is the desired goal state. A problem can either be a one-time problems or recurring problems. Deciding the total student enrolment into the Department of Computer Science each year can be said to be a recurring problem (since there is an agreed staff-student ratio, which determines the carrying-capacity for each Department). However, to determine why a machine failed or why a device is faulty – can be regarded as a one-time problem.

Many concepts for problem solving apply to both kinds/types of problems; Though, the problems have different situations. Recurring problems require continuing data collection, report, and other infrastructure. Every problem has a lifespan. Some problems must be solved quickly, but others are not urgent. We cannot take too long to develop a schedule for programs run on a computer, because the time to run programs is usually short. Conversely, we have more time to determine the location of departments in a plant to be built next year. Problems have also different impact. Problems solved should be worthy of the resources required to solve them. If it costs more to do the study than it will save, don't do it. The problem impact should determine the amount of effort we put into solving the problem.

Defining A Solution

Problems by nature and typically, do not go away of their own accord unless acted upon, or when we do something to resolve them. This intervention and action is often referred to as the art and science of problem solving. While, many of us often seek to explore and exploit easy and quick solutions; Also, complex problems often require complex and dynamic solutions. Solving a well-structured problem can be found to be quite easy. To find the minimum of a quadratic function, we use first and second derivatives. However, to solve an ill-defined or ill-structured problems is not obvious and in often cases, termed as intractable as such solutions may be out

of, or beyond our reach. For example, how would you reduce world hunger? A large part of problem solving is transforming ill-structured problems into well-structured problems.

There are four (4) approaches to a problem namely:

- a. Absolution, ignores the problem and hopes it gone away, which is seldom a good approach.
- b. Resolution finds an acceptable solution to the problem using common sense; resolution is usually better than absolution, but the answer may not be a good one.
- c. Solution is the third approach. It uses quantitative and experimental methods to get the “best” answer under the current conditions.
- d. Dissolution as the fourth approach seeks to redesign the system to eliminate the cause of the problem. This approach, if possible and not too costly, is the preferred one.

To solve a problem, five (5) conditions should exist namely:

1. What gap does exist between the current state (that is, as we have it now operating), and the goal state (what it actually is that we should be operating)?
2. An awareness of the gap, in which we recognize the problems that creates that distance between what it is now and what it should be?
3. The motivation to close the gap since the problem task is quite important to someone and is of great importance – which will allow them to devote time, resources and man-hours, to resolve the problem task.
4. The ability to measure what the size and performance gap is using performance metrics; Else, while there may be no end to what revisions we can do – we may also never attain/reach a solution. Thus, this measure informs us of the severity of the task, and lets us to know when such improvement has occurred.
5. The ability and resources to close the gap using available tools, processes and techniques that have proven to be successful in similar scenarios; we have the methodology to solve the problem and the resources to carry out the solution.

If one or more of these conditions is found to be missing, our successfully solving the problem becomes quite unlikely. But, if and where these conditions exist and are well-formulated

or defined, we can proceed with the problem-solving stages or approaches. Thus, if/when a problem is ill-formulated, ill-defined and ill-structured, the problem-solving framework can provide useful insights to resolving it.

Thus, every problem/task is defined by its elements and their relations. The right perspective and a formal description of a problem (space and/or domain), every well-formulated, well-defined and well-structured problem must meet all conditions as mentioned above; while also allowing the problem-solver to identify the following:

1. Define the problem state space that contains all possible configurations and the relevant objects including some impossible ones also.
2. Specify all the states that describes all possible situations from which the problem-solving process may start. These states are called the initial state(s).
3. Specify one/more state(s) that will be acceptable solution to the problem called the goal state(s).
4. Specify the rules that describes the actions (operators) available within the problem space (domain).

Thus, a problem can be solved simply by using the rules, in combination with the appropriate search control strategy to move through the problem space (domain) until a path through the series of state(s) from the initial to a goal – is found. This process is called a search; And it is fundamental to problem solving process. Search is the general mechanism that can be used when more direct methods is unknown. A search provides the framework into which more direct methods for solving subparts of a problem can be embedded.

Critical Thinking

The importance of Problem Solving (PS) can never be over-emphasized as it directly and indirectly impacts our daily lives and will continue to play critical roles in our way of living for decades to come. Research have been dedicated to PS, with attempts made by educationists and psychologists to make it accessible to all in various degrees.

No wonder the Nigerian Universities Commission (NUC) has now entrenched this course, as an educational policy at the core of Computing Science(s). Wherever numerical problems exist and is involved, from the simple 2+3 to the complex,

chaotic and numerical analysis – tools have been advanced (from simple calculator to sophisticated computer programs) have been developed to assist the problem-solver to resolve such challenges more effectively and efficiently. Gone, are the days of slide rule and logarithmic tables.

Even graduates today – experience great difficulty in solving real life problems. Somehow, they cannot apply many taught theories into practice; nor can they posit, formulate, reflect or even theorize on practice. In fact, it is the human mind in the end that has to be applied in a problematic situation and solve the problem. Its capacity to solve the problem is directly related to the knowledge stored in the mind. And knowledge is a product of thinking. Thinking can vary from a very simple and mundane thought, to a very sophisticated and complex one. The nature of the problem dictates the level of thinking.

Higher-order thinking can be conceptualized as a complex mode that can generate multiple solutions. Such process can involve abstraction, uncertainty, applying multi-goal/criteria, reflection, and self-regulation. It also facilitates the transfer of knowledge (i.e., use and transformation of already existing knowledge in creating new knowledge). Conversely, lower-order thinking may be considered as one that requires minimum cognitive effort and it is algorithmic. In an attempt by humans to increase the power of the mind, thinking is information-processing; And computers by design accomplish such feats using programs so that they are today – regarded as thinking machines. Thinking can also be viewed as making sense or judgements (and directly relates to critical thinking).

The complexities and dynamics of critical thinking is evident from the fact that there are no globally-accepted definition. But, many scholars have noted that critical thinking skills as identified may include: (a) analysis and synthesis, (b) making judgements, (c) drawing warranted conclusions, (d) decision making, and (e) generalizations. Critical thinking is thus, a prerequisite need to PS. When computers are used in the PS situation, the need for computational thinking (CT) is another prerequisite. CT broadly speaking, describes a set of thinking skills that are integral to solving complex problems using a computer.

Living in a knowledge-based, digital revolution era with the continued advances in digital technology and informatics, such evolution portends the constant progression and ever-increasing trend that seeks to combine as well as leverage on knowledge and technology to solve problems is becoming the mode rather than the exception. This has continued to advance the field of knowledge discovery in database (KDD). Creativity and innovation driven by tacit knowledge, and critical thinking driven by logic, making judgements, analysis and synthesis and so on become the tools for problem thinking and PS. If technology is added as another tool to the process, then CT is a pre-requisite (and must transcend its mere performing computations). CT frees a problem-solver's memory so that s(he) can concentrate on the essence of the problem.

Discovering Knowledge Data

Knowledge Discovery in Data (KDD) is a systematic process that identifies valid, potentially useful, and understandable patterns from large amounts of data. It's about transforming raw data into valuable knowledge.

The KDD process typically consists of the following steps:

1. Data Cleaning – One truth is that real-life data are quite often, messy and unstructured. So, the process or step seeks to remove any inconsistencies, errors, or outliers that might skew the results. Simply put, it removes noise (unwanted data) and inconsistent data.
2. Data Integration – Data is often collected from multiple sources, and each has its own format and structure. This step merges all these data into a unified set, and ensures there is consistency to also reduce redundancy or data replication. Thus, it combines data from different sources.
3. Data Selection – not all data are relevant for analysis. This step helps us to select the data-subset that pertains to our specific objective and choose, only data relevant for our analysis.
4. Data Transformation – Data may need to be summarized, aggregated, or transformed to make the dataset, suitable for mining.
5. Data Mining – It uses various algorithms and techniques, patterns, trends, and relationships are extracted from the data.

6. Pattern Evaluation – not all recognized data patterns are useful, meaningful or interesting. This step helps filter out the noise, ensuring only valuable insights are considered.

7. Knowledge Presenting – This step helps visualize, report, or make the knowledge accessible and understandable in a proper format.

Figure 1.1. Knowledge Discovery in Database (Data Mining)

The Many Definitions of PS

1. Polya as the pioneer in PS – defines solving a problem means finding a way out of a difficulty, a way around an obstacle, attaining an aim that was not immediately understandable.
2. Schoenfeld [14] notes that a problem is only a problem, if you do not know how to go about solving it. A problem that has no surprises in store, and can be solved comfortably by routine or familiar procedures (no matter how difficult!) it is an exercise.
3. Green and Gilhooly [5] PS in all its manifestations is an activity that structures everyday life in a meaningful way. This activity draws together the different components of cognition. Thus, the kind of problem will dictate the type of cognitive skill necessary to solve the problem: linguistic skills are used to read about a certain task and debate about it, memory skills to recall prior knowledge and so on. Depending on the knowledge and thinking skills possessed by a problem solver, what could be a problem for one might not be a problem for somebody else.
4. Martinez"s [4] PS can be defined simply as the pursuit of a goal when the path to that goal is uncertain. In other words, it is what you do when you do not know what you are doing. In concluding, we adopt the following working definition for this paper: PS is an activity that makes use of cognitive or cognitive and physical means to overcome an obstacle (problem) and develop a better idea of the world that surrounds us.
5. Ojugo et al. (Ojugo et al., 2016) Many authors and scholar agree that PS is a cognitive process, and findings continue to suggests that PS is a product of a number of cognitive processes and actions; Though, it is not actually the processes that are of value; But, the successful action

that yields or leads to a solution of the problem.

Engaging in PS implies conscious and subconscious thinking. And the type of problem will dictate the type of thinking: The more complex a task, the higher the level of thinking required.

6. Voskoglou et al. [21] Mathematics by its nature explores and exploits PS at its core and essence. It does examine the role of PS in learning and attempts a review of the evolution in the emergence of a solution as a self-sufficient science at the 1960"s until today.

A rough chronology of this progress is as thus:

a. 1950"s – 1960"s: Polya"s ideas on the use of heuristic strategies in PS ([13], [16], etc). 1970"s: Emergency of mathematics education as a self – sufficient science (research methods were almost exclusively statistical).

Research on PS was still based on Polya"s ideas.

b. 1980"s: A framework describing the PS process, and reasons for success or failure in PS: „Expert performance model" ([17], [18], etc). 1990"s: Models of teaching using PS, e.g. constructivist view of learning [19], mathematical modelling and applications (e.g. see [20] and its references), etc.

c. 2000"s: While early work on PS focused mainly on analyzing the PS process and on describing the proper heuristic strategies to be used in each of its stages, more recent investigations have focused mainly on solvers" behaviour and required attributes during the PS process; e.g. „Multidimensional PS Framework" (MPSF) of Carlson & Bloom [21]. More comprehensive models for the PS process in general (not only for mathematics) were developed by Sternberg & Ben-Zeev [22], by Schoenfeld [23] („PS as a goal-oriented behaviour"), etc.

What has been agreed by many authors ([5], [11], [12], etc) is that a problem basically consists of three states: the starting state, the goal state and the obstacles or a set of available actions or strategies to move from the starting state to the goal state. Matlin [12] states that the initial state describes the situation at the beginning of the problem; the goal state is reached when we solved the problem; and the obstacles describe the restrictions that make it difficult to

proceed from the initial state to the goal state. The greatest difference among problem solvers of the same problem tends to lie in the third state, which might have infinite" possibilities, if the goal and the starting points are the same for everyone.

Who is a Problem-Solver?

The person who has the problem or someone paid to solve problems could be the problem solver. In the production arena, a problem solver can/may be a manager, analyst, or industrial engineer. Often, there are several problem solvers working together to achieve or reach an optimal solution to a case. And because problem solvers are people, they are not infallible. Their personal beliefs, values, bias, and norms can impact their judgment and disposition toward solving a certain problem; And thus, affect the whole problem-solving process. Whether a problem exists or not, is affected by a person's point of view, but recognizing that bias exists should minimize its impact.

The knowledge and experience of a problem solver also comes into play. If a person has more tools, that person has a wider range of options for solving problems. Experience teaches which tools to use in certain case and even helps in inventing new ones or adopting old tools to new situations. If a particular tool is not in a kit, you will not use it.

Problem Representation

A problem space or domain is best viewed or represented as a directed graph. First, a graph is a structure that consists of a series of nodes (vertices) existing in space such that each node has relations with other nodes in the same space. The relations details what binds or connects a node to another. Thus, we have that the graph G consists of a set of vertices V (or nodes) that are interconnected with a set of edges e (arcs, links and relations) that allow these vertices to interact and learn from each other. Thus, the graph is denoted as:

$$G = (v, e) \quad (1)$$

In some cases, the relations are weighted. For example, the relationship between a mother (Nkem) and her son (Eric) as well as between a father (Arnold) and daughter (Elena), which is based on dependence – cannot obviously not have the same weight as that relationship between the son (Eric)

and his friend (Emma) as well as between the daughter (Elena) and her friend (Elizabeth), which is based on respect. Thus, such a graph G is denoted as:

$$G = (v, e, w) \quad (2)$$

A graph is either be directed (noting that it depicts the flow of data from one node as source to another as destination), or undirected (when the flow can be in any direction). The problem space is represented as a directed graph where vertices represent search states, and the path represents the operators that are applied to change the state(s). To simplify the search – algorithms are modeled as logically and programmatically convenient representation of the problem space as a tree. This is often so, because the tree decreases the complexity of a search at a cost through its capability of duplicating some nodes on it that are linked numerous times in the graph as in Figure 1.2(a). A tree is best be viewed as a graph in which any two nodes are connected by exactly one path such that there are no connection ends up as a circle as in Figure 1.2(b).

Figure 1.1. (a) Graph (b) Trees

Formal Description to Problem Solving

Problem-solving is often characterized as a systemic search through a range of possible actions to reach a predefined goal or solution. The problem-solving methods can either be:

1. Special purpose are tailor-made solutions for a particular task or problem as it often exploits very specific features of the situation in which the problem is embedded.
2. General purpose is applicable to a wide range of tasks and/or problems. An example is the means-end analysis – a step-by-step, incremental reduction of the difference between the current state and the final/goal state.

For Example: The Tower of Hanoi puzzle/game has the story that there exist 1000-disks and three (3) pegs denoted as A, B and C. The goal of which, is to move the rings/disks from the pegs A-to-C such that no bigger disk-ring is placed at any time on top of a smaller one as in Figure 1.3 below.

Solution:

- The puzzle involves a set of rings/disks of different sizes that can be placed on 3-different pegs (A, B, C)
- Initial State: The puzzle starts with the rings arranged

as in Figure 1.3.

- Goal State: The goal is to move all the disk rings from figure 1.3(a) to Figure 1.2(b)
- Condition: (a) Only the top-most ring on a peg can be moved, (b) once moved, it can be placed on an empty peg, and (c) no smaller ring shall be under a bigger one, or no bigger ring shall be placed on a smaller one.
- States are all the possible situations encountered while solving the task of moving the disks between the pegs.
- Problem Space are all the possible configurations of the disk or ring on the pegs

Problem Description

A problem must consist of the following description as thus:

(a) its current state, (b) the actions that can transform one state into another, and (c) the desired (goal) state that acts as an acceptable final state of the problem. Thus, a problem solver must be able to identify the following:

1. State space (can either be explicit or implicit) and should describe everything that is needed to solve a problem.
2. Initial state is the state from which the problem starts.
3. Goal state is description of a desired, final state as either be partial or complete conditions for which data/values are assigned to states based on a variety of conditions. Thus, the goal state has assigned to its variables, a set of data values that fulfill the conditions of the desired state.
4. Operators are changes caused to the states. They are simply actions that can transform one state to another. Operators often consist: (a) pre-conditions that provide or renders a partial description of the current state of the task/problem that must be true in order to perform an action, and (b) instructions tells us what to accomplish as next state, when to act, and how to create that next state. It is often advised that the operators should be as general as possible to reduce their number(s).
5. Elements shows what is relevant to a task and it details previous knowledge of the task as the starting point.
6. Restrictions are the quality of the solution(s) or ground-truth as reached, which is either be optimal or complete. This can thus, results to the following options: (a) finding the shortest sequence, (b) finding the least expensive sequence defining cost, and (c) finding any sequence as

quickly as possible.

7. Problem-Solving is finding the ordered sequence of operators that transforms the current (start) state into a goal (final) state that acts as an acceptable solution.

Problem Description of the 8-Queen Problem

It is a game of 8-queen puzzle as in Figure 1.4.

Figure 1.4. One solution of the 8-Queen Puzzle Problem

Solution:

- The puzzle involves placing 8-Queens on the chess-board such that no Queen is found attacking another
- Problem/State Space: configuration $N(\text{Queens}) = 8$ on chess board with only one queen per row and per column
- Initial State: configuration without queens on the board
- Goal State: configuration with $N = 8$ queens such that no queen attacks any other on the chess board
- Operators or Actions
 - a. Condition: The new queen is not attacked by any other queen that is already placed on the board.
 - b. Transformation: place a new queen in a particular cell or square on the board
- Solution: one solution is as above in Figure 1.4 without cost consideration.

The Tree Structure

A tree, simply put is a way of organizing related objects in a hierarchical form. Objects are data, and data is anything that we can manipulate. Thus, a tree is a type of data structure where each element is attached to one or more elements directly beneath it. The connections between these various elements (or nodes) on a tree to describe the relationships between the various elements is called branches. They are often called inverted trees since their root is often expressed at the top. The elements that have no other elements below them are called leaves. A binary tree is also a special tree for which each element has two-branches below it. A simple tree is as expressed in Figure 1.5.

Properties of a tree is as thus:

- a. A tree is a special case graph. Its topmost node is called a root node, from which all operations on the tree begins.
- b. A node has at most one parent. However, the topmost root node has no parent.

- c. Each node has zero or more child nodes below it
- d. The node at the bottom-most level of the tree is called a leaf node. The leaf node thus, has no child node.
- e. A node that has a child node is called a parent node.
- f. The depth of a node n is the length of the path from the root node to the node. A root node is at a depth of zero.
- g. All nodes have relationship with other nodes such that the total number of connections going out from a node is called an out-degree. Conversely, the total number of connections coming into a node is called in-degree.

Figure 1.5. A simple unordered tree structure

The Stack Structure

Stacks are also a type of data structure in which the data or objects are decked in a linear form. Thus, accessing such data requires that we explore and maintain either the order of last-in-first-out (LIFO) or the first-in-last-out (FILO) respectively. In Computing Science, the orders are known as linear linked list. The stack implements or maintains a LIFO order with its object/data piled up in a sequence of one atop the other as in Figure 1.6(a).

Figure 1.6(a) Stack

Properties of the stack are:

- a. Insert and delete are made at only one end called the top
- b. Any intermediate element $a[i]$ is on the top of element $a[i+1]$ where $1 < i \leq n$.
- c. All operations takes place at the top of the stack.
- d. The Pop operation removes a data from the top of the stack; while, the Push operation inserts or adds a data item to the top of the stack as in Figure 1.6(b).

Figure 1.6(b) Insert / Delete

The Queue Structure

Queues just like stacks, are also a type of data in linear form, and accessing it means to maintain the first-in-first-out (FIFO) order as in Figure 1.7(a). A queue restricts how data are added to and removed from the list as in Figure 1.7(b).

Figure 1.7(a) Queue

Properties of the queue are:

- a. A queue has two-ends.
- b. All insert (enqueue) takes place from one end called the rear or back; While, all delete (dequeue) takes place from

the other end called the front

c. If a queue has $a[n]$ as the rear element; then, $a[i+1]$ is behind $a[i]$ where $1 < i \leq n$.

d. All operations takes place at one end of the queue or the other end. A dequeue operation removes the data at the front of the queue. An enqueue operation adds data to the rear of the queue.

Figure 1.6(b) Dequeue / Enqueue

Searching Techniques

Searching is a process of locating a given data value, item or object position in a linked list (or an array) or tree from a list of known value(s). searching is the process of finding a value in a list of values. A search is successful if the object value being sought for or searched is found; else, it is said to be unsuccessful. Typically, a search answers the True or False as to whether the value search is present or not. Also, there as scenarios where the search conditions may be modified to return the position where the item or value can be found.

Thus, there are basically two (2) search techniques explored namely linear search and binary search.

The Linear Search – is a simple search algorithm, that is sequentially run to search over all available elements on a list, one by one. It linearly searches through a tree or array of elements till the desired element is not found in sequential order. Each and every element of the list is checked against the goal object; And, where it is found – the particular value item is returned; Else, the search continues till the end of the data collection. Its search begins from first element on the tree/list, and continues until the desired element is found or the end of the file is reached. Linear Search is often applied to unordered list when there are fewer elements in a list.

Basic Working Idea of the Linear Search:

- a. Read the goal (search) element to be searched
- b. Initially compare, the search (goal) element with the first element in the list. If both values are matched, then display "element is found"; And, terminate function.
- c. If both are unmatched, then compare search element with the next element in the list.
- d. Repeat steps until the search element is compared with all the elements in the list.
- e. Where the last element in the list has been searched and

no element in the list is matched with the search element, then display "element is not found" And, stop or terminate the function.

Algorithm Listing for the Linear Search Algorithm

Linear Search (A, x)

// Given an array A[i, x] and x represents the number of elements

Set found to false

Set position to -1 && Set index to 0

While found is false and index < x

if A[index] is equal to search value then

found = true

position = index

endif

Add 1 to index

end while

return position

Pseudocode Listing for the Linear Search Algorithm

Procedure linear_search (list, value, x)

int index = 0;

// Position for search

int position = -1;

// Indicate whether the value is found

bool found = 0;

while (index < x && !found)

if (list[index] == value

//The value is found

Position = index;

Index++;

return position;

end procedure

Example: Suppose in a list of 10-elements, we are to search for the value 39. Our goal element is X = 39 (index A[4]).

Step 1: It compares the x with first element located at index 0. Element is not matched then it moves to the next element.

Step 2: It compares x with next element located at index 1. Element is not matched then it moves to the next element.

Step 3: It compares x with next element located at index 2. Element is not matched then it moves to the next element.

Step 4: It compares x with first element located at index 3. Element is not matched then it moves to the next element.

Step 5: It compares x with first element located at index 4. Element is matched – and the desired goal (search) element is found at location 4. The process is then stopped, and the function is terminated. Then, the function returns the 'element is found' information.

Note: If the function had gone through to the last element or reached the end-of-file – and the search (goal) element was not found or does not match any value in the array list, then the function will display 'element not found' message. And thereafter, it will terminate the function.

Complexity analysis: Time complexity of linear search in best case is $O(1)$ and in average and worst case is $O(n)$.

Its merits are: (a) they are simple, easy to understand and implement, (b) they are very resource efficient, (c) does not require copying/partition of arrays, (d) they are memory efficient, (e) can be used on both unsorted and sorted data, (f) does not require the data in the array to be stored in any particular order. Conversely, its demerits are: (a) has a very poor $O(n)$ general efficiency. That is, the performance of the algorithm scales linearly with the size of the input, and (b) they are very slow and time consuming.

Binary Search – is the most popular search as it is based on the divide-and-conquer model technique. It first sorts the array list either in ascending or descending order. In this method firstly find the middle element and to search an element compare the value with the elements in the middle position of the array. If the value is matched, then search is successful otherwise list is divided into two-halves (one from the first element to the middle element i.e. first half or lower half and second half or upper half from middle to last element). If the value is less than the middle element, then it must lie in the lower half of the array and if it's greater than the element then it must lie in the upper half of the array. We repeat this procedure on the lower (or upper) half of the array. Binary Search is useful when there are large numbers of elements in an array.

Basic Working Idea of the Linear Search:

Let X be the element to be searched in an array $A[0 - (n-1)]$.

Thus, it proceeds as follows:

a. It selects the element to be searched

- b. It finds the mid-value of an array list using the formula:
 $\text{mid} = (\text{low} + \text{high})/2$. Here, low refers to lower bounds of an array (i.e. 0) and high refers to upper bounds of the array (i.e. n-1).
- c. If the element to be searched, is the middle element then the search terminates' Else, it continues.
- d. If the element to be searched is less than the middle element then the search is performed in left sub array and high is set to mid-1.
- e. Otherwise, if the element to be searched is greater than the middle element then the search is performed in right sub array and low is set to mid+1. In this way the problem size is reduced to half after every comparison.
- f. It repeats same process until we find the search element in the list, or until the sub-list contains only one element.
- g. If that element also does not match with search element, then display 'element not found in the list!' and terminate the function.

Pseudocode Listing for the Binary Search Algorithm

Procedure Binary_Search (a, i, n, x)

A $\square$ sorted array: n $\square$ size of array: x $\square$ value to be search

Set lowerBound = 1

Set upperBound = n

while x not found

if upperBound < lowerBound

exit: x does not exists.

set midpoint = lowerBound + (upperBound - lowerBound) / 2

if A[midpoint] < x

set lowerBound = midPoint + 1

if A[midpoint] > x

set lowerBound = midPoint - 1

if A[midpoint] = x

exit: x is found at location midPoint

end while

end Procedure

Algorithm Listing for the Binary Search Algorithm

BinarySearch (a, i, n, x)

//Given array a[i: n] of elements in nondecreasing order, $1 \leq i \leq J$, determine whether

//x is present, and if so - return j such that $x = a[j]$; else return 0.

if (n = i) then // if small(P)

if (x = a[j]) then return n;

```

else return 0;
else
{ //reduce P into a smaller sub-problem
mid = [(i + n)/2];
if (x == a[mid]) then return mid;
else if (x < a[mid]) then
return BinarySearch(a, i, mid - 1, x);
else return BinarySearch(a, mid+1, n, x);

```

Complexity analysis: Time complexity in best case is $O(1)$, and in the average and/or the worst case is $O(\log(n))$.

Its merits are: (a) they are more efficient than the linear search, (b) every time it makes a comparison and fails to find the desired item, it eliminates half of the remaining portion of the array that must be searched, and (c) they use random access to the data but linear search only requires sequential access. Conversely, their major demerit is the binary search process must only be performed on sorted list.

Example: Suppose in a list of 10-elements, we are to search for the value 72. Our goal element is $X = 72$

Step 1: With X as 72 and because the list has been sorted - it performs midpoint value computation. Note: the indexes are $low=0$, and $high=9$. Thus, $midpoint = (0+9)/2 = 4$.

Step 2: As $X > mid$, so low become $mid+1$; and $high$ remain the same. So, next computation indexes $low=5$, and $high=9$. Thus, $midpoint = (5+9)/2 = 7$.

Step 3: As $X > mid$, so low becomes $mid+1$ and $high$ remain the same. Thus, we have $low=8$ and $high=9$, $mid=(8+9)/2 = 8$. Element is matched, and search terminated at 3-steps.

QUESTIONS

Multi-Choice:

- Which of the following is not a required condition for binary search algorithm?
 - The list must be sorted,
 - Direct access to the middle element in any sub list,
 - A mechanism to delete and/or insert elements in list.
 - Number values should only be present
- The Average case occurs in linear search algorithm.
 - when item is somewhere in the middle of the array
 - when item is not the array at all
 - when item is the last element in the array
 - Item is the last element in the array or is not there

3. Binary search algorithm cannot be applied to.
a) Sorted list b) Sorted Binary tree c) Sorted Linear Array d) Pointer Array
4. Complexity of linear search algorithm is:
a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$ d) $O(n \log n)$
5. To find the location of an item in a set of items is ____
a) Discovering b) Finding c) Searching d) Mining
6. Worst-case time complexity of linear search algorithm is?
a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$

True/False Choice

1. Finding the location of a given data value in a set of items is called searching.
2. The binary search is much more efficient than the linear search.
3. A Linear search algorithm or function can be used for large array list elements.
4. Complexity of binary search algorithm in worst and average case is $O(n)$.
5. Comparisons are necessary and sufficient for computing both the minimum and the maximum is $3n-3/2$.

Exercises

1. Given the array (24, 13, 56, 35, 46, 27, 78, 48)
a. Use the binary search algorithm
Write the BinarySearch program for the answer to Q1(a)
2. Given the array (24, 13, 56, 35, 46, 27, 78, 48)
a. Use the linear search algorithm
b. Write the LinearSearch program to answer to Q2(a)
3. Define searching. Explain different types of searching.

Chapter Two

LOGIC GATES

2.0 DATA REPRESENTATION

Digital computers recognize data as electrical pulses or signals, represented as two-state conditions using 1 and 0 respectively. They are arranged in various combinations to represent numbers, alphabets and symbols in the system. The smallest unit of data that can be manipulated by the computer is called a bit, seen as a single binary value 1 or 0. Computers manipulate data as a group of eight bits called byte. Gates allows digital systems to perform complex tasks with the interconnection between its components designed to carryout logic laws based on mathematical rules known as Boolean Algebra, which has led an increased productions of digital systems due to their reliability in the use of 1/0 states.

Data and instructions as manipulated by the computer can

either be numeric or non-numeric and are represented as binary values of 0 and 1 called bits making the computer a two-state machine. A good knowledge of number systems helps us to understand how the computer is programmed. Also, to know also how the system is made from two-state components and used to manipulate other number schemes. For example, if we have:

$$7 * 1000 + 3 * 10 + 4 * 0.1 + 3 * 0.001$$

Represented as: $A_n * b^n + A_{n-1} * b^{n-1} + \dots$ where b is base and A is the number system digit. The system's advantage is that only few different symbols are required to represent any number no matter how large or small. Symbols 0, 1, 2, 3 are represented by quantities zero, one, two, three etc.

NUMBER SYSTEMS: OVERVIEW

Number systems use a finite number of symbols, with each symbol called a digit and is used to denote a quantity. The numbers of these distinct symbols are known as the base or Radix. To denote quantities larger than those associated with the individual symbols, the digits are juxtaposed to form a number. Hence the relative position of a digit within the number is associated with a weighting factor. There are four major number base systems namely Decimal, Binary, Octal and Hexadecimal number systems.

Decimal system requires that data be represented with symbols (0 – 9). It evolved from natural counting ability via the use of fingers in which the symbols 0, 1, 2, 3 and so on are represented with quantities one, two, three etc. Digital systems convert these symbols into its binary equivalence to aid data manipulation and storage. Its major advantage is that it makes arithmetic tasks simple; while its disadvantage is that systems constructed from it will find it hard to differentiate correctly between ten levels of switch used to represent the various bits, making the system unreliable.

Binary System is used in digital equipments for circuit design to improve reliability – because it uses two-states “0 and 1” to represent data as voltage pulses of either a “low = 0” and “high = 1”. Its highest value is 1 and the lowest is 0; hence, it is called a two-state system. A two-state system represents characters, numbers and alphabets using logic states 0 and 1 in that each state will be used to represent a

different digit and the system must have as many states as there are digits in the number system.

Octal System is used in computer microprocessor and with some computer-based controllers. One of the particularly useful properties of this system is the simple conversion technique from binary to octal system and vice versa, because it has a base of 8, which is $2^3 = 8$. The system has its lowest character as 0 and highest as 7.

Hexadecimal System is used by microprocessors as their design and internal construct today relies on this system. Just like the octal system, their useful property is their conversion from binary to hexadecimal and vice versa as they are used in computers to represent alphanumeric symbols and they have a base of 16, which is $2^4 = 16$.

Table 1.1 shows the different number systems

Decimal Binary Octal Hexadecimal

1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14

For interaction between the number schemes, the computer performs conversion between the various schemes as thus:

Decimal to Binary Conversion They are converted using either to the power of 2 or divide by the number 2. For example if we are to convert the number 6, which is in base

10 (written as 610) to binary, our solution is:

Divisor Number Remainder

2 6

2 3 0

2 1 1

0 1

Binary To Decimal Conversion is as thus:

Position 1st 2nd 3rd 4th 5th 6th 7th 8th 9th

Weight 20 21 22 23 24 25 26 27 28

Answer 1 2 4 8 16 32 64 128 256

Converting the value 11011012 to decimal is as follows

1. Space out the numbers indicating their positional weight
2. Place the answer to the various weights
3. Sum of all the weights together and write it out as follows

7th 6th 5th 4th 3rd 2nd 1st

1 1 0 1 1 0 1

$1*2^7 + 1*2^6 + 0*2^5 + 1*2^4 + 1*2^3 + 0*2^2 + 1*2^1$

6 5 4 3 2 1

$64 + 32 + 0 + 8 + 4 + 0 + 1 = 10910$

Binary To Octal is the reverse of the former and step for its conversion. Example $1011101 = 1,011,101 \rightarrow 1, 3, 5 = 1358$.

Octal To Binary It is easier to convert because we know that octal is 23. Hence, we take the long sequence of binary given to us and group them in bits of 3 such that each digit is converted to its binary equivalent. The binary characters then form the binary equivalent. For example:

Octal: 1: 5: 7: 6

Binary: 001: 101: 111: 110

Binary equivalent of 15768 is 11011111102

Hexadecimal to Binary Hexadecimal is 24 – so we take long sequence of binary and group them in bits of 4 such that each digit is converted to its binary equivalent. The binary characters then form the binary equivalent.

Binary Hexadecimal

1: A 1A

01: 1010

8: 3 83

1011: 0011

Binary Addition There are four rules to binary addition:

$0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$ and $1 + 1 = 2$ but written as 0 carry 1

For example $109 + 92 = 201$

$11011012 = 10910$

$+10111002 = 9210$

$100010012 = 20110$

Binary Subtraction For example, just like addition also starts from the least significant bit (LSB) for example,

$11011012 = 10910$

$-10111002 = 9210$

$00100012 = 1710$

DIGITAL SYSTEMS

Digital systems are more reliable due to the ON-OFF states and operation, accomplished via the aid of semiconductor materials such as diode and transistor packages. These elements are connected together to help perform diverse tasks using logic laws known as Boolean Algebra. This algebra was formulated in 1847 by George Boole based on two-states "True/False" to aid mathematical method used for systems containing two-state elements, based on two possible values 0 and 1. It was later used for the design and implementation of modern digital equipments. The algebra allows any symbols such as A, B or C to have one of two-allowable states namely "True/1 or False/0".

Logic - is the science of arguments or is regarded as the science of reasoning by formal method. Any statement in a logical context is a declaration, verbal or written that is either true or false, but never both.

Arguments - Any statement is regarded as an argument.

Example: (a) All girls are soft minded (b) All airplanes fly

Arguments are meant to help the thinker build reasonable ways of arriving at conclusions, drawn from intelligent point of view, devoid of rancour and prejudice. There are two types of arguments namely inductive and deductive.

1. Inductive Reasoning - are methods of arriving at conclusions that extracts and discovers general laws from facts, allowing the thinker to start from a particular case and extend the argument to a more general case. This

we refer to as forward chaining. For example, we can state these arguments as follows:

- a. Dupe is dirty. Hence, all girls called dupe are dirty.
- b. Arnold is tall. Hence, those called Arnold are tall.

2. Deductive Reasoning – are methods of arriving at conclusions drawn from the facts that the thinker starts from the general laws to a more specific case. This is a direct opposite of the inductive principle and can also be referred to as backward chaining. For example: -

- a. All express roads are dual carriage roads. Hence, the Benin – Lagos road is a dual carriage road.
- b. American countries are rich. Mexico is a rich country.

Propositions – any simple statement, which can either be true or false. For example: -

- a. Benin city is Edo State of Nigeria – True
- b. Samuel's father is pregnant – False

We must note that statements that propose questions and those used to signify exclamations are not propositions because they can neither return a true or false value. E.g. (a) What is your name? (b) Are you crazy!

Compound Statements – Two or more statements can be combined to form a compound statement. They are very useful especially when we need to make the proper and accurate decisions. These statements are combined using negation, disjunction and conjunction.

- a. Disjunction – is when two proposition are joined by the OR operation (Union or addition) of the two statements.

If two statements Y and Z are as thus:

Y = The sun is too hot today and Z = It will rain today

Their disjunction/OR is given by the table 2.1:

Y Z G

True True True

True False True

False True True

False False False

Hence, the disjunction of Y and Z produces statement G:

“The sun is too hot or it will rain” such that G is True

when any of the statement is true – and False only when both statements are false. Sometimes, the context in which the OR statement differs. For example, a pregnant

woman can give birth to a single, twin etc. It can be a girl, boy or both. This is called inclusive OR. But, if it is used so that only one statement is correct, it is called exclusive OR. For example, you are either a man or a woman. You can never be both.

b. Conjunction – is when two statements are joined by an AND operation. Like the OR, if we have two statements:

Y = The sun is too hot today and Z = It will rain today

Y Z G

True True True

True False False

False True False

False False False

Conjunction of Y and Z produces G, which states that:

“The sun is too hot and it will rain today”. G is True, only when both statements are true; otherwise False when any of the statement is false as in table 3.2:

c. Negation – If a statement Z is negated, it produces another statement that is false when Z is True; or True when Z is false and denoted by NOT Z as in table 3.3

Z = “I am singing and dancing right now”. Hence, NOT Z = “I am not singing and dancing right now”

Z NOT Z

True False

False True

A A B

B

Disjunction Conjunction

Fig 2.1 (a) and (b) shows the disjunction and conjunction

2.1 BOOLEAN ALGEBRA THEOREM

Boolean postulates and its fundamental properties are seen in table 3.4 below with its dual form.

Minterm Maxterm Postulate

$A+B=B+A$ $A.B = B.A$ COMMUTATIVE

$A + (B+C) = (A+B) + C$ $(A.B) . C = A. (B .C)$ ASSOCIATIVE

$A+(B.C) = (A+B).(A+C)$ $A.(B+C) = (A.B) + (A.C)$ DISTRIBUTIVE

$A+0 = A$ $A.1 = A$ IDENTITY

$A + (A') = 1$ $A . A' = 0$ COMPLEMENT

$(A.B)' = A' + B'$ $(A + B)' = A' . B'$ DE MORGAN

$A+1=1$ $A+0=0$ IDEMPOTENCE

Truth Tables provide a list of all possible input-states and its corresponding output that can result from each state of a logic gate. It specifies the circuit's behaviour via the use of Boolean theorems, applied to the gate since there are only two possibilities about a statement: true or false. This is because a statement can never be partly true or partly false. If true, it has a 1 value, else 0 if false.

2.2 SEMICONDUCTOR GATES

Gates are two-state systems whose input and output takes only one of two-allowable states representing binary voltages 0/Off and 1/On states. Systems constructed from such two-state elements are known as logic circuits. These include switches, decoders amongst others. Gates directly employ a set of mathematical rules used to describe the logic circuit's behaviour, which either allows or not allow the flow of electrical signals along the wire paths referred to as opening or closing of logic gate paths. Its advantages are:

1. They are exceedingly small and require little power to operate
2. They are extremely cheap
3. They are much reliability because of their binary operability.
4. They are constructed from two-state materials and are combined into a network to form systems such as flip-flops, registers etc
5. Operate at higher speed for a very long time without failure

The behaviour of a circuit is determined by the gates implemented on that circuit, usually constructed from a truth table that describes the output produced by the circuit. Each circuit has its own unique truth table and the gates used for their implementation are as follows:

OR GATE can have any number of inputs with just one output, which produces a 1 output when any of the input is a 1; otherwise it produces a 0 output when all the inputs are 0 as seen in table below and figure 2.2.

Input A Input B Output

0 0 0

0 1 1

1 0 1

1 1 1

A C=A+B Figure 2.2

B

AND GATE just like the OR gate can have any number of

inputs with just one output. It produces a 0 output unless all its inputs are 1. Hence, it gives an output of 0 unless all inputs are 1 in which case the output is 1 as in figure 2.3.

$A \cdot B = AB$

Figure 2.3

B

Input A Input B Output

0 0 0

0 1 0

1 0 0

1 1 1

NOT GATE has a single input and output as its function is to produce an output directly opposite of whatever input was sent through it. Hence, there are only two states

Input Output

1 0

0 1

A A' Figure 2.4

NAND GATE is same as an AND gate plus an Inverter so that the gate performs an AND-NOT task on its inputs - giving an output that is directly opposite to an AND output.

Figure 2.5

Input A Input B AND NOT = OUTPUT

0 0 0 1

0 1 0 1

1 0 0 1

1 1 1 0

NOR GATES performs a NOT-OR operation on any number of inputs - giving an output opposite to that of the OR gate as seen in figure 2.6 below.

Figure 2.6

Input A Input B AND NOT = OUTPUT

0 0 0 1

0 1 0 1

1 0 0 1

1 1 1 0

EXCLUSIVE-OR GATE (XOR) produces an output of 1 when either of the input is a 1. Such a circuit excludes the case that both inputs are 1, as it is a two input circuit. The

XOR function is only defined by two inputs and its results are same for whatever combinations as in figure 2.7.

$$C = A \oplus B$$

Figure 2.7

Input A	Input B	Output C
0	0	0
0	1	1
1	0	1
1	1	0

These gates are used for designing and manufacturing of circuits and also in the development of diodes, transistors and integrated circuits (IC) supplied with two or more input gates. Circuits produced from such these are usually classified into Combinational and Sequential.

A combinational logic is any logic circuit whose steady state output values depends solely on its steady state input so that the specific outputs obtained depends on the given input. E.g. adders, decoder, encoder, multiplexers, etc. The major characteristics for combinational Logic includes:

- No output state depends on another output state.
- Input states that have occurred have no influence on its behaviour.
- The order of the input does not affect the final output.
- Actual level of the particular output at any time depends on the Logic State present at all of the inputs at that instant or time.
- As input changes, output changes also immediately to the level which corresponds to the new input.

Sequential circuit consists of the combinational logic and storage elements. The state of the storage elements depends on the present and its internal state. Hence, its output state depends on current inputs and previous output. Examples include flip-flops, multivibrators etc.

2.3 DESIGN AND IMPLEMENTATION

There are several ways employed in solving problems associated with circuit design and implementation. Hence, it becomes apparent that a logic designer must define some parameters when faced with such design task. Hence, he must determine the circuit's specification or the output as expected of the circuit. The best way to present this specification is via truth table that attempts to reproduce the detailed circuit design and the exact description of the

circuit's behaviour.

To implement a circuit, we need basic steps such as:

- Development of a truth table
- Determine the circuit specifications
- Equivalent Circuits and design of the Logic circuit

Truth Table Development Gives designer a pictorial representation of what is expected of the circuit behaviour.

Truth tables are the first systematic approach in designing a circuit, with initial specifications represented, general rules are formulated for the construction of the truth table to give the designer an insight to the number of inputs and outputs that must be obtained by the logic circuit clearly stating all possible combination of the input states for the circuit.

Determine Specifications Suppose we have three simple two-state switches connected to supply inputs to a logic circuit with a single output, which controls a lamp and the circuit operation is such that changing the position of any of these switches changes the condition of the lamp by going off and on, and vice versa. To design a truth table for this logic circuit behaviour, we note that because the number of input and output for the circuit have already been stated and determined, many other features must be considered to produce the desired outcome and they are:

- ✓ Input specifies current, properly labeled or the states of all inputs conditions represented by a 0 or 1.
- ✓ Output specifies the outcome of the input states (as combined under the stipulated gate, which having an output of 1 – causes the lamp to be lit or an output of 0 – causing the lamp not to be lit.
- ✓ Starting Positions specification does not indicate the output of any condition, although it states how the circuit must change when any input changes are made. For this reason, it is best for the designer to choose the output for a single set of input. The output results from the gate in use.

Equivalent Circuit/Implementation It is appropriate for the designer to know which gates to use in order to produce the required circuit's behaviour. Most integrated circuits are constructed using NAND gate – because they are easier and less costly to manufacture than other gates. Hence, any circuit can be constructed from NAND gates if only we know the equivalent gates that make up the circuit.

Fig 2.8a has that NAND gate gives same output as an Inverted OR gate

Figure 2.8b has that NOR gate gives same output as an Inverted AND gate

Implementation means to design a circuit as expressed in the Boolean form. If we have: $L = A + B'C$ as in Figure 2.9.

$$L = A + B'C$$

A

B B'C

C

$L = B'C + ABC' + A'C$ is shown figure 2.10.

B B'C

$$C L = B'C + ABC' + A'C$$

ABC'

A

A'C

2.5 DUALITY

A network represents the interconnection of various gates, which closely corresponds to the hardware components with lines that connects the gates in the logic diagram seen as the physical wires connecting the devices on the IC board. Each Boolean expression is a corresponding operation with a gate symbol. But most combinational networks are implemented and designed using NAND and NOR gates, since they contain more than one variable, combined together based on laws. Thus, each expression there is a logic diagram that causes a one-to-one relationship.

Duality simply means obtaining the same values for a reverse operation. Take the AND operation as an example, if we take all the 0s and change them into 1s and take the 1s and change them into 0s, we get the output of the OR gate. The same for the NOR and NAND operations. The postulates that have so far been presented are both in dual forms (that is, they are in pairs). They reflect the summitry of the "OR and the AND" operations. Hence, expressions in dual form are derived by simply replacing the occurrence of "." by "+", "OR by AND" and 0 by 1 as in the postulate table.

With duality, we see that the AND circuit with a value logic 1, becomes an OR circuit with logic 0 – and vice versa. Rules of duality are often listed in the order that illustrates the duality of the algebra, which allows the circuit designer to work with

two sets of values by listing the various states that the input to the logical network can take, as well as the desired output for each input condition. The logical expression is thus divided up into two sets of standard values (minterm and maxterm) known as the canonical forms, used to express any combinational logic.

Minterms are obtained by performing the AND of several OR operation based on the different variables given to obtain a 1 output. By complementing those variable whose values in the truth table is 0 so that it leaves the variable with a values 1 and uncomplementing those whose values are 1 such that its logical product produces a 1 instead of a 0. For example, such truth table for a 3-variable is shown below using Z:

$$Z = A'.B'.C + A'.B.C' + A.B'.C' + A.B.C$$

A, B and C are the variables. The fourth column represents the minterm, which lists AND terms of the input variable to give a 1 output. Variables with 0 values are complemented to give 1; while those with a 1 are left uncomplemented. For example, in row 1 – A, B and C are complemented so that their AND gives us a 1 value. In row 2 – we have that A and B are complemented with C not complemented to give 1. The last column Z is the output, to indicate those minterms that appear in the expression. Hence, the circuit behaves in such a way that at places where the minterm is 1, Z is 1 and the circuit is high; while at other places where the output is 0 – the circuit is low. This is best understood as with the help of Karnaugh's map. We can also write the minterms as:

Value A B C ABC Minterm Output Z

0 0 0 0 000 A'.B'.C' 0

1 0 0 1 001 A'.B'.C 1

2 0 1 0 010 A'.B.C' 1

3 0 1 1 011 A'. B.C 0

4 1 0 0 100 A. B'. C' 1

5 1 0 1 101 A. B'. C 0

6 1 1 0 110 A. B. C' 0

7 1 1 1 111 A.B.C 1

Maxterm is same as product of all sum expressions. Since an OR is the opposite of the AND, we simply change all the 0 outputs to 1 and changing the 1s to 0. Hence, using the same table in our previous example as in the table below:

Value A B C ABC Maxterm Output Z

0 0 0 0 000 A+B+C 0
 1 0 0 1 001 A+B+C' 1
 2 0 1 0 010 A+B'+C 1
 3 0 1 1 011 A+B'+C' 0
 4 1 0 0 100 A'+B+C 1
 5 1 0 1 101 A'+B+C' 0
 6 1 1 0 110 A'+B'+C 0
 7 1 1 1 111 A'+B'+C' 1

From the table, the Maxterm is as thus:

Maxterm (0, 3, 5, 6) = $(A+B+C)(A+B'+C')(A'+B+C')(A'+B'+C)$

2.6 KARNAUGH'S MAP

Algebraic simplification of logic expression can sometimes be difficult and expertise is needed to effect the changes properly in circuit design. Hence, Karnaugh's map provides a graphical method for simplifying Boolean expressions so that it is arranged in boxes, each of which represents a Minterm combination of the variables given by the truth table. The number of boxes for each map depends on the number of variables in the given expression. A two-variable expression map will require four (2^n), where $n = 2$, to give us four boxes or cells. The steps are as follows:

- Convert all terms into their Minterm forms and plot table.
- Draw the map with appropriate number of boxes or cells and place 1 in the cells that correspond to each minterm in the expression and leave the cells with no corresponding values.

For example, construct the map for $L = A'B + A'B' + AB$

The truth table below shows Minterms obtained.

Value A B Minterm

0 0 0 $A'B' = 1*1 = 1$
 1 0 1 $A'B = 1*1 = 1$
 2 1 0 $AB' = 1*1 = 1$
 3 1 1 $AB = 1*1 = 1$

A 0 1 A 0 1

B B

0 00 10 0 A'B' AB'

1 01 11 1 A'B AB

Place 1 in boxes of $L = A'B + A'B' + AB$ as thus:

A 0 1 A 0 1

B B

0 A'B' AB' 0 1

1 A'. B AB 1 1 1

Adjacency: Karnaugh's map has that the cells in each map are assigned a minterm value. Each cell is arranged such that their geographical location faces or is adjacent to other cells. It helps with grouping minterms for finding a minimal circuit as given by the expression.

A 0 1

B

Minterm A B Adjacent to

0 0 0 1, 2 0 0 2

00 10

1 0 1 0, 3

2 1 0 0, 3 1 1 3

3 1 1 1, 2 01 11

For a 3-variable, the table holds as thus:

Minterm A B C ABC together Adjacent to

0 0 0 0 000 1, 2, 4

1 0 0 1 001 0, 3, 5

2 0 1 0 010 0, 3, 6

3 0 1 1 011 1, 2, 7

4 1 0 0 100 0, 5, 6

5 1 0 1 101 1, 4, 7

6 1 1 0 110 2, 4, 7

7 1 1 1 111 3, 5, 6

AB

C 00 01 11 10

0 0 2 6 4

000 010 110 100

1 1 3 7 5

001 011 111 101

For a 4-variable, the table below holds:

Minterm A B C D ABCD Adjacent to

0 0 0 0 0 0000 1, 2, 4, 8

1 0 0 0 1 0001 0, 3, 5, 9

2 0 0 1 0 0010 0, 3, 6, 10

3 0 0 1 1 0011 1, 2, 7, 11

4 0 1 0 0 0100 0, 5, 6, 12

5 0 1 0 1 0101 1, 4, 7, 13

6 0 1 1 0 0110 2, 4, 7, 14

7 0 1 1 1 0111 3, 5, 6, 15

8 1 0 0 0 1000 0, 9, 10, 12
 9 1 0 0 1 1001 1, 8, 11, 13
 10 1 0 1 0 1010 2, 8, 11, 14
 11 1 0 1 1 1011 3, 9, 10, 15
 12 1 1 0 0 1100 5, 9, 12, 15
 13 1 1 0 1 1101 5, 9, 12, 15
 14 1 1 1 0 1110 6, 10, 12, 15
 15 1 1 1 1 1111 7, 11, 13, 14

AB 00 01 11 10

CD

00 0 4 12 8
 0000 0100 1100 1000
 01 1 5 13 9
 0001 0101 1101 1001
 11 3 7 15 11
 0011 0111 1111 1011
 10 2 6 14 10
 0010 0110 1110 1010

Example 1: Plot a map for $L = ABC + A'B'C + A'B'C'$

Value A B C Minterm

0 0 0 0 $A'B'C' = 1*1*1 = 1$
 1 0 0 1 $A'B'C = 1*1*1 = 1$
 2 0 1 0 $A'B'C' = 1*1*1 = 1$
 3 0 1 1 $A'BC = 1*1*1 = 1$
 4 1 0 0 $AB'C' = 1*1*1 = 1$
 5 1 0 1 $AB'C = 1*1*1 = 1$
 6 1 1 0 $ABC' = 1*1*1 = 1$
 7 1 1 1 $ABC = 1*1*1 = 1$

A

B 0 0 1 1

C 0 1 1 0

0 $A'B'C'$ $A'BC'$ ABC' $AB'C'$

0 2 6 4

1 $A'B'C$ $A'BC$ ABC $AB'C$

1 3 7 5

A

B 0 0 1 1

C 0 1 1 0

0 1

1 1 1

Example 2: Plot $L = ABC' + AB'C' + A'BC + AB'C$

A A

B 0 0 1 1 0 0 1 1

C 0 1 1 0

B 0 1 1 0

C

0 A'B'C' A'BC' ABC' AB'C' 0 1 1

0 2 6 4

1 A'B'C A'BC ABC AB'C 1 1 1

1 3 7 5

Plot $L = ABC'D + ABCD + A'BC'D' + AB'C'D' + A'B'C'D'$

No A B C D Minterm L

0 0 0 0 0 A' B' C' D' = $1*1*1*1 = 1$ 1

1 0 0 0 1 A' B' C' D = $1*1*1*1 = 1$ 0

2 0 0 1 0 A' B' C D' = $1*1*1*1 = 1$ 0

3 0 0 1 1 A' B' C D = $1*1*1*1 = 1$ 0

4 0 1 0 0 A' B C' D' = $1*1*1*1 = 1$ 1

5 0 1 0 1 A' B C' D = $1*1*1*1 = 1$ 0

6 0 1 1 0 A' B C D' = $1*1*1*1 = 1$ 0

7 0 1 1 1 A' B C D = $1*1*1*1 = 1$ 0

8 1 0 0 0 A B' C' D' = $1*1*1*1 = 1$ 1

9 1 0 0 1 A B' C' D = $1*1*1*1 = 1$ 0

10 1 0 1 0 A B' C D' = $1*1*1*1 = 1$ 0

11 1 0 1 1 A B' C D = $1*1*1*1 = 1$ 0

12 1 1 0 0 A B C' D' = $1*1*1*1 = 1$ 0

13 1 1 0 1 A B C' D = $1*1*1*1 = 1$ 1

14 1 1 1 0 A B C D' = $1*1*1*1 = 1$ 0

15 1 1 1 1 A B C D = $1*1*1*1 = 1$ 1

AB 00 01 11 10

CD

00 0 4 12 8

0000 0100 1100 1000

01 1 5 13 9

0001 0101 1101 1001

11 3 7 15 11

0011 0111 1111 1011

10 2 6 14 10

0010 0110 1110 1010

AB 00 01 11 10

CD

00 1 1 1

01 1

11 1

10

Example 2: $L = A'BCD + ABC'D + ABCD' + AB'C'D' + ABCD$

As with the table above, we plot the map as thus:

AB 00 01 11 10

CD

00 1

01 1 1

11 1

10 1

2.7 CIRCUIT-MINIMIZATION

Every Boolean expression can be transformed into minterm has important practical effects on the processing speed of any combinational circuit network. As the circuit become larger with more than 5 variables, it become important to simplify the circuit to obtain a minimal circuit that behave in just the same way as the initial circuit expression given. Due to the string of gates, when there is a change in the input, output does not respond to it immediately because of time delay. This is the time it takes for the signal to propagate the change made via the components that make up the network. The time delay it takes for the output to respond to the input change is known as gate delay and different manufacturer use different gate delays because it is more expensive to implement shorter gate delays than longer gate delays because shorter gate delays require more power to operate. These delays becomes noticed as the network contains more gates that will require it to loop the signal in the circuit.

Typical gate delay is 10ns so that if we have three-level net, an input signal change will require 10ns to propagate via the first gate, another 10ns via the second gate and another 10ns to the last gate – a total of 30ns. In a two-level net, it will require 10ns to propagate via the first gate and another 10ns via the second gate, total of 20ns for the input change to be propagated through to its output. Hence, a two-level network grants a reduced time propagation delay from 30ns

to 20ns, a 33% improvement in speed performance. Most circuit networks are implemented as two-level networks. Networks with more than two-levels require fewer gates than their equivalent two-level network – since a gate requires same amount of physical space in an IC board. Hence, a two-level network achieves faster speed at the expense of the extra space required for the additional gates (a tradeoff between time and space).

The minimization methods are as follows:

- Boolean Postulates or Algebraic simplification
- Karnaugh's map
- Tabulation and charting (Quine McCluskey method)

The Objectives of Minimization includes

- a. It helps logic designers and engineers to produce a design that meets the specification of the system required.
- b. It allows lesser number of gates as well as lesser number of input elements because the smaller the logic designs, the less complicated and cheaper the production cost.
- c. Circuits are produced at reduced production cost.
- d. The smaller the number of gates used, the lower the gates chances of malfunctioning when in use.

Minimization simply helps produce a minimal circuit form of the Boolean expression giving, to minimize cost. However, the use of Boolean algebra when minimizing is somehow unreliable – because most times the minimum form of the circuit achieved via it does not appear as the cheapest cost of implementation. Boolean algebra minimization usually produces a more expensive form of the circuit, at which juncture, Karnaugh's map and McCluskey's tabulation and charting method is employed. Tabulation method is most appropriate for design involving more than five variable inputs, where Karnaugh's maps are not effective.

Karnaugh's Map Minimization: The steps for obtaining the minimal form of an expression using map method are:

- a. Complete the table and map as required by entering a 1 value into the boxes that have their minterm in the Boolean expression given
- b. Find minterms that differ from each others by a 1 variable and construct the smallest, possible number of groups by grouping those minterms that have 1 variable separating them.

- c. All squares containing a 1 must be in atleast one group as well.
- d. A minterms should not appear twice in the expression. But iif it should, its inclusion must enable a smaller group to be obtained from the larger group.

Minimize $L = AB + AB'$

Step 1: We construct the truth table and plot the map:

Value A B Minterm

$$0\ 0\ 0\ A'B' = 1*1 = 1$$

$$1\ 0\ 1\ A'B = 1*1 = 1$$

$$2\ 1\ 0\ AB' = 1*1 = 1$$

$$3\ 1\ 1\ AB = 1*1 = 1$$

A 0 1 A 0 1

B B

0 A'B' AB' 0 1

1 A' B AB 1 1

Step 2: We group the minterms and minimize algebraically:

$$L = AB + AB'$$

$$L = A (B' + B) \text{ ----- Distributive Law}$$

$$L = A (B' + B) \text{ ----- } B' + B = 1 \text{ (Complement Law)}$$

$$L = A.1 \rightarrow L = A$$

Example 1: $L = A'B'C' + A'BC' + A'BC + ABC + AB'C$

Construct table and plot the map gives us:

Value A B C L output Minterm

$$0\ 0\ 0\ 0\ 1\ A'B'C' = 1*1*1 = 1$$

$$1\ 0\ 0\ 1\ 0\ A'B'C = 1*1*1 = 1$$

$$2\ 0\ 1\ 0\ 1\ A'B C' = 1*1*1 = 1$$

$$3\ 0\ 1\ 1\ 1\ A'BC = 1*1*1 = 1$$

$$4\ 1\ 0\ 0\ 0\ AB'C' = 1*1*1 = 1$$

$$5\ 1\ 0\ 1\ 1\ AB'C = 1*1*1 = 1$$

$$6\ 1\ 1\ 0\ 0\ ABC' = 1*1*1 = 1$$

$$7\ 1\ 1\ 1\ 1\ ABC = 1*1*1 = 1$$

AB

C 00 01 11 10

0 1 1

1 1 1 1

$$L = A'B'C' + A'BC' + A'BC' + A'BC + ABC + AB'C$$

$$L = (B'+B) A'C' + (C'+C) A'B + (B'+B) AC \text{ -----Identity Law}$$

$$L = A'C' + A'B + AC \text{ ---- Minimal circuit}$$

Crosscheck if $L = F$:

No A B C A' B' C' A'C' A'B AC F

```
0 0 0 0 1 1 1 1 0 0 1
1 0 0 1 1 1 0 0 0 0 0
2 0 1 0 1 0 1 1 1 0 1
3 0 1 1 1 0 0 0 1 0 1
4 1 0 0 0 1 1 0 0 0 0
5 1 0 1 0 1 0 0 0 1 1
6 1 1 0 0 0 1 0 0 0 0
7 1 1 1 0 0 0 0 0 1 1
```

We crosscheck if the circuit given is same as the initial circuit as sometimes, we can have two solutions the best of which depends on the circuit. Hence, $L = F$. We also minimize as:

A

B 0 0 1 1

C 0 1 1 0

0 1 1

1 1 1 1

The second grouping is minimized as thus:

$$L = A'B'C' + A'B'C' + A'BC + ABC + ABC + AB'C$$

$$L = (B'+B) A'C' + (A'+A) BC + (B'+B) AC \text{ -----Identity Law}$$

$$L = A'C' + BC + AC \text{ ----- Minimal form}$$

Crosschecking with the truth table shows that $L = F$.

A B C A' B' C' A'C' BC AC F

```
0 0 0 0 1 1 1 1 0 0 1
1 0 0 1 1 1 0 0 0 0 0
2 0 1 0 1 0 1 1 0 0 1
3 0 1 1 1 0 0 0 1 0 1
4 1 0 0 0 1 1 0 0 0 0
5 1 0 1 0 1 0 0 0 1 1
6 1 1 0 0 0 1 0 0 0 0
7 1 1 1 0 0 0 0 1 1 1
```

$$\text{If } L = A'BCD + ABCD + ABC'D' + ABC'D + ABCD'$$

Solution: Construct table, plot map and group as thus:

AB

00 01 11 10

CD

00 1

01 1

11 1 1

10 1

$$L = A'BCD + ABCD + ABC'D' + ABCD + ABC'D + ABCD'$$

$$L = BCD + AB(C'+C)(D'+D) + AB(C'+C)(D'+D) \text{ -----Identity}$$

$$L = BCD + AB(1).(1) + AB(1).(1) \text{ ----- } (AB + AB = 1)$$

$$L = BCD + AB \rightarrow L = AB + BCD$$

Crosschecking with original truth table plotted will gives:

$$N0 \ A \ B \ C \ D \ AB \ BCD \ F = AB + BCD$$

0 0 0 0 0 0 0

1 0 0 0 1 0 0 0

2 0 0 1 0 0 0 0

3 0 0 1 1 0 0 0

4 0 1 0 0 0 0 0

5 0 1 0 1 0 0 0

6 0 1 1 0 0 0 0

7 0 1 1 1 0 1 1

8 1 0 0 0 0 0 0

9 1 0 0 1 0 0 0

10 1 0 1 0 0 0 0

11 1 0 1 1 0 0 0

12 1 1 0 0 1 0 1

13 1 1 0 1 1 0 1

14 1 1 1 0 1 0 1

15 1 1 1 1 1 1 1

Our minimal circuit yields same behaviour as the initial circuit.

Example 2: Plot the map and minimize $L = A'B'CD' +$

$$AB'CD' + AB'CD + A'BC'D' + ABC'D' + A'BC'D + ABC'D$$

No A B C D Minterm L

$$0 \ 0 \ 0 \ 0 \ 0 \ A' \ B' \ C' \ D' = 1*1*1*1 = 1 \ 0$$

$$1 \ 0 \ 0 \ 0 \ 1 \ A' \ B' \ C' \ D = 1*1*1*1 = 1 \ 0$$

$$2 \ 0 \ 0 \ 1 \ 0 \ A' \ B' \ C \ D' = 1*1*1*1 = 1 \ 1$$

$$3 \ 0 \ 0 \ 1 \ 1 \ A' \ B' \ C \ D = 1*1*1*1 = 1 \ 0$$

$$4 \ 0 \ 1 \ 0 \ 0 \ A' \ B \ C' \ D' = 1*1*1*1 = 1 \ 1$$

$$5 \ 0 \ 1 \ 0 \ 1 \ A' \ B \ C' \ D = 1*1*1*1 = 1 \ 1$$

$$6 \ 0 \ 1 \ 1 \ 0 \ A' \ B \ C \ D' = 1*1*1*1 = 1 \ 0$$

$$7 \ 0 \ 1 \ 1 \ 1 \ A' \ B \ C \ D = 1*1*1*1 = 1 \ 0$$

$$8 \ 1 \ 0 \ 0 \ 0 \ A \ B' \ C' \ D' = 1*1*1*1 = 1 \ 0$$

$$9 \ 1 \ 0 \ 0 \ 1 \ A \ B' \ C' \ D = 1*1*1*1 = 1 \ 0$$

$$10 \ 1 \ 0 \ 1 \ 0 \ A \ B' \ C \ D' = 1*1*1*1 = 1 \ 1$$

$$11 \ 1 \ 0 \ 1 \ 1 \ A \ B' \ C \ D = 1*1*1*1 = 1 \ 1$$

$$12 \ 1 \ 1 \ 0 \ 0 \ A \ B \ C' \ D' = 1*1*1*1 = 1 \ 1$$

$$13 \ 1 \ 1 \ 0 \ 1 \ A \ B \ C' \ D = 1*1*1*1 = 1 \ 1$$

$$14 \ 1 \ 1 \ 1 \ 0 \ A \ B \ C \ D' = 1*1*1*1 = 1 \ 0$$

$$15 \ 1 \ 1 \ 1 \ 1 \ A \ B \ C \ D = 1*1*1*1 = 1 \ 0$$

AB 00 01 11 10

CD

00 1 1

First group

01 1 1

11 1

Second

10 1 1 Third

$$L = A'BC'D' + ABC'D' + A'BC'D + ABC'D + A'B'CD' + AB'CD + AB'CD'$$

First Quadruple - written in this order Minterms 4, 5, 12, 13 as

$$L = A'BC'D' + ABC'D' + A'BC'D + ABC'D$$

$$L = BC' (A' + A)(D' + D) + BC' (A' + A)(D' + D)$$

$$L = BC' + BC' \rightarrow L = BC'$$

Second group is written in this order Minterms 10, 11 as

$$L = AB'CD' + AB'CD \rightarrow L = AB'C (D' + D) \rightarrow L = AB'C$$

Third grouping, we have the Minterms 2 and 10

$$L = A'B'CD' + AB'CD' \rightarrow L = B'CD' (A' + A) \rightarrow L = B'CD'$$

$$\text{Hence, } F = BC' + B'CD' + AB'C$$

Crosschecking to see if the output is same as thus

$$A \ B \ C \ D \ BC' \ B'CD' \ AB'C \ F = BC' + B'CD' + ABC'$$

0 0 0 0 0 0 0

0 0 0 1 0 0 0

0 0 1 0 0 1 0

0 0 1 1 0 0 0

0 1 0 0 1 0 0

0 1 0 1 1 0 0

0 1 1 0 0 0 0

0 1 1 1 0 0 0

1 0 0 0 0 0 0

1 0 0 1 0 0 0

1 0 1 0 0 1 1

1 0 1 1 0 0 1

1 1 0 0 1 0 0

1 1 0 1 1 0 0

1 1 1 0 0 0 0

1 1 1 1 0 0 0

Again $L = F$. Minimizing 4-variable is confusing and tricky.

Hence, it requires practice, reasoning and experimenting

using several minterm groups to find the true minimal circuit.
Hence, adjacency will help us know which Minterm groupings to consider when we have complex maps because there can be more than one true or valid minimal circuits.

Consider $L = \sum (0, 1, 2, 3, 5, 6, 7, 8, 9, 12, 13, 14)$ soled as:

Step 1 and 2 ends us up with the map below:

AB 00 01 11 10

CD

00 1 1 1 First

Second

01 1 1 1 1

11 1 1 Third

10 Fourth

1 1 1

Fifth

First Quadruple - written in this order Minterms 8, 9, 13, 12 as

$$L = AB'C'D' + AB'C'D + ABC'D + ABC'D'$$

$$L = AC' (B'+B)(D'+D) + AC' (B'+B)(D'+D)$$

$$L = AC' + AC' \rightarrow L = AC'$$

Second Quadruple written as 0, 3, 1, 2

$$L = A'B'C'D' + A'B'CD + A'B'C'D + A'B'CD'$$

$$L = A'B' \text{ (common terms)}$$

Third Quadruple written as 3, 6, 2, 7

$$L = A'B'CD + A'BCD' + A'B'CD' + A'BCD$$

$$L = A'C(B'+B)(D'+D) + A'C(B'+B)(D'+D)$$

$$L = A'C + A'C \rightarrow L = A'C$$

Fourth Quadruple written as Minterm 1, 5, 13, 9

$$L = A'B'C'D + A'BC'D + ABC'D + AB'C'D$$

$$L = C'D(A'+A)(B'+B) + C'D(A'+A)(B'+B)$$

$$L = C'D + C'D \rightarrow L = C'D$$

Fifth couple written as Minterm 4, 6

$$L = A'BCD' + ABCD'$$

$$L = BCD'(A'+A) \rightarrow L = BCD'$$

$$\text{Hence: } F = AC' + A'C + A'B' + C'D + BCD'$$

This is a plausible solution, but is not correct since it does not proffer the minimal solution. But again, consider this:

AB 00 01 11 10

CD

00 1 1 1 First

Second

01 1 1 1 1

11 1 1 Third

10 Fourth

1 1 1

The problem of the first solution was with our grouping of the Minterms 13. In the second, it is grouped in the third quadruple as Minterm 1, 5, 13 and 9. This second grouping does not contain this third quadruple.

The solution is as:

First Quadruple - written in this order Minterms 8, 9, 13, 12 as

$$L = AB'C'D' + AB'C'D + ABC'D + ABC'D'$$

$$L = AC' (B'+B)(D'+D) + AC' (B'+B)(D'+D)$$

$$L = AC' + AC' \rightarrow L = AC'$$

Second Quadruple written as 0, 3, 1, 2

$$L = A'B'C'D' + A'B'CD + A'B'C'D + A'B'CD'$$

$$L = A'B' \text{ (common terms)}$$

Third Quadruple written as 1, 3, 7, 5

$$L = A'B'C'D + A'BCD + A'B'CD + A'BC'D$$

$$L = A'D (B'+B)(C'+C) + A'D (B'+B)(C'+C)$$

$$L = A'D + A'D \rightarrow L = A'D$$

Fourth Quadruple written as Minterm 6, 14

$$L = A'BCD' + ABCD'$$

$$L = BCD' (A'+A) \rightarrow L = BCD'$$

$$F = AC' + A'B' + A'D + BCD'$$

Don't Care Circuits: Any situation in which the circuit is designed to process its inputs, not minding that some input conditions may erupt as the circuit is in use, that were initially not cared for. As these inputs occur, it means that not all inputs states are represented by such an expression - since some states were not expected. But if they occur - we don't care about the signal change. Hence, a circuit with such ability that takes care of such inputs are called Don't care circuits because the circuit does not also care about what output is obtained, if those inputs ever appear.

This feature gives designers extra flexibility in minimization such that designers can design a circuit to give a 0 or 1 when don't care condition is present. By carefully selecting don't care states, to produce a 1 and 0 at some points, we improve minimization. When representing don't care conditions, we

are free to cover the cell with the symbol x, which acts as a wildcard that it can be treated either as a 0 or a 1 value.

Consider the example below:

AB

C

Minimized as follows:

$$1 \ x(a,b,c) = \sum(1, 4, 5)$$

$$= A'B'C + AB'C + AB'C + AB'C'$$

$$1 \ 1 = B'C (A'+A) + AB' (C + C')$$

$$= B'C + AB'$$

Consider adding don't care and minimize the function as:

AB

C

Minimized as follows:

$$X \ 1 \ x(a,b,c) = \sum(1, 4, 5) + d(0, 7)$$

$$= A'B'C + AB'C' + AB'C + ABC$$

$$1 \ X \ 1 = (A'+A)(B'+B)(C'+C) + AC (B'+B)$$

$$= AC$$

The function without a don't care requires two AND gate and one OR gate; while that with don't care requires just one AND gate - by simply allowing the minterm 0 to be grouped and the minterm 7 to be a 0.

2.8 COMBINATIONAL DEVICES

Combinational circuits are most commonly used in the design of computers, because even in sequential circuit designs, combinational circuits are used but in addition to memory elements that store bits sent through or to the device. Each device can be specified as a black-box with its corresponding truth table to help us know and define how the device's output depends on its input. Most of the devices as discussed below are implemented using a two-level network, which trades off less space for processing time.

These devices have an input line called enable, which acts like an on/off switch for the device such that when the enable is off or 0, the output lines all become 0 regardless of what the input values are. Hence, the device is disabled or turned-off. But when the enable line is 1/off, the output lines at this instance will then depend according to what the input lines specifies for the device and so, the device is turned-on and enabled for the specified activity. Hence, an AND gate

with two inputs has one input as x and the other is the enable alongside a corresponding output Y as in figure 2.20.

X Y Figure 2.20 shows the logic diagram of an enable gate

Enable

The truth table is shown below when the enable is 1:

Enable = 1

Enable X AND Output

1 0 1.0 = 0

1 1 1.1 = 1

Also, when the enable is 0, we have the table as thus:

Enable = 0

Enable X AND Output

0 0 0.0 = 0

0 0 0.1 = 1

The enable gate does not require a new “enable” input but requires that the AND gate be viewed differently such that we see input X as data line; while the enable is the control line. Hence, the enable controls the data either letting it to pass through unchanged or prevents its passage to output. Another useful gate used is the selective inverter having inputs (a data line and an inverter). When the invert line is 1, its output becomes the complement of the data line; while if the invert line is 0, the data passes via to output unchanged as seen in figure 2.21 with XOR gate viewed differently.

X Y

Figure 2.21 shows the logic diagram for the selective Invert Line inverter

When the invert line is 1, then we have that

$$Y = X \oplus (\text{invert})$$

$$= X' \cdot (\text{Invert}) + X \cdot (\text{Invert})'$$

$$= X' \cdot 1 + X \cdot 1' \rightarrow X' \cdot 1 + X \cdot 0 = X'$$

Its output is complement of the data input. If invert line = 0:

$$Y = X \oplus (\text{invert})$$

$$= X' \cdot (\text{Invert}) + X \cdot (\text{Invert})'$$

$$= X' \cdot 0 + X \cdot 0' \rightarrow X' \cdot 0 + X \cdot 1 = X$$

The data passes via unchanged to output. Some of these devices are discussed below as thus:

Multiplexers – is a device that simply selects one of the several data inputs to be routed to a single data output, such that a set of control lines determine which particular data input is sent via to output as in figure 2.22 below.

D0

D0

S1

S0 D1

F

D1 D2

S1 D3

S0 S0

S1

D2

S1

S0

Fig 2.22 (a) abd (b) shows the D3 block implementation and block diagram of a four-input S1 multiplexer

S0

S1 S0 F = Output

0 0 D0

0 1 D1

1 0 D2

1 1 D3

F is the single data output with the device having 6 inputs namely S1, S2 and D0 to D3. Its complete table requires $2^6 = 64$ entries as column 3 shows that D2 is selected when the select lines are 10. Hence with D2 = 1 with F = 1. If D2 is 0, then F is 0 regardless of other values of D0 via D3. With n selects lines, which can choose one from 2^n data lines. The number of data inputs of a multiplexer is the power of 2.

Demultiplexer – This device routes a single input to several output lines. Hence, it does nothing more than a decoder with an enable such that the data input line D is connected to the enable. If D = 0, the decoder is disabled and the data output selected by S1 and S0 is 0. If D = 1, the decoder is enabled and the data output line selected is 1. In either case – the selected output line has same value as the data input

line as in table as thus:

S1 S0 D0 D1 D2 D3

0 0 D 0 0 0

0 1 0 D 0 0

1 0 0 0 D 0

1 1 0 0 0 D

D0

D1 Figure 2.23 shows

the four-output

D D2 demultiplexer

D3

S1 S0

Binary Adders – have two sets of input terminals such that in parallel addition, series of bits corresponding to the two binary numbers appear at these terminals while the output terminals gives the sum of the two numbers. Adders are grouped into half and full adder. The basic building block for the half adder is an XOR gate with two inputs and two outputs as in figure 2.24. The table shows that the S output of a half-adder gives the sum of each digit while output C gives the carryover of the digit represented as thus:

$$S = a \oplus b \rightarrow a'b' + a'b' \text{ and } C = ab$$

X Y Sum Carry

0 0 0 0

0 1 1 0

1 0 1 0

1 1 0 1

X S (Sum)

X S

Half

Adder

C (Carry) Y C

Y

Figure 2.24 shows the logic diagram for the binary adder

The half adder shows that the sum is identical to the XOR function and the carry is identical to the AND gate function as in the truth table and its implementation above. The full adder is similar to half adder except for additional inputs that receives carry-overs from the lower significant bit (LSB) as in table below. Adding two numbers of such will exceed to the

next stage and requires a combinational net with three inputs instead of two as with the half adder namely the Cin, X and Y. Cin is carry input that comes with the carry made from the addition of the LSB. X and Y are the binary numbers; while the outputs are Sum and Cout is the carry output, which goes to Cin of the full adder for the next column.

Cin Sn

X S

Xn

Y C

X S Cout

Yn

Y C

Figure 2.26 shows implementation of a full adder

X Y Cin Sum Cout

0 0 0 0 0

0 0 1 1 0

0 1 0 1 0

1 0 0 1 0

0 1 1 0 1

1 0 1 0 1

1 1 0 0 1

1 1 1 1 1

The full adder sums from the second half adder such that if any of the half adders have a carry, the full adder has a carry because it is implemented on two half adder with an OR gate. As with the truth table, to add two 4-bit numbers will require an eight-input network as in figure 2.27.

X3

C Figure 2.27 shows

X2 the block diagram

of full adder.

X1

FOUR-

X0 BIT S3

ADDER S2

Y3

S1

Y2

S0

Y1

Y0

Xn defines 4-bit of the first LSB as Yn defines the second 4-bits number; Sn is the 4-bit sum; while C is the carry.

Implementing a 4-bit adder uses one-half adder for the LSB and three full adders, one for each of the remaining columns in the addition – it becomes a ripple-carry adder, because a carry from the LSB is propagated or rippled via the columns to the left.

Binary Decoder – A decoder is a device that takes a binary number as input and sets one of the data output lines to 1 while the rest becomes set to 0. The data line that is set depends on what value of the binary number is sent as input. As with the previous example, if we have an n-bit number – it will have 2^n values such that the number of data outputs of the decoder is a power of 2.

S1 D0 D0

S0 S0 D1

S1 D1

D2

S1

S0

D3

S1 D2

S0

Enable

S1 D3

Fig 2.28 shows implementation and S0 block diagram of a 2 x 4 binary decoder.

Truth table for a 2 X 4 binary decoder is as below:

S1 S0 D0 D1 D2 D3 F = output

0 0 1 0 0 0 D0

0 1 0 1 0 0 D1

1 0 0 0 1 0 D2

1 1 0 0 0 1 D3

Arithmetic Logic Unit – Devices above perform operations such as AND, ADD and OR. Unary addition is an arithmetic operation, while AND and OR are logical operators and the CPU perform these via the ALU. Table shows a 1-bit ALU.

S1 S0 F Cout

0 0 X.Y 0

0 1 X+Y 0

1 0 1 0

1 1 X plus C

Y plus Cin

Cin Cout

X Figure 2.29

ALU shows the block

Y

F diagram of a 1-

bit arithmetic

and logic unit

S1 S0

The truth table shows that if S1-S0 is 00, the output F is the AND of X and Y. If S1-S0 is 01, it means that F is the OR of X and Y. When it is 10 – F is simply X regardless of the value of Y. In these three cases, F does not depend on the value of Cin except when S1-S0 is 11. F is the sum of X, Y, Cin with Cout being the carryover. The full adder contains an enable input such that when E is 0, Cout and Sum are 0 regardless of Cin, X and Y; Otherwise, if E is 1, the full adder operates normally. D0, D1 and D2 are connected to the enable lines for X.Y, X+Y and Y in the logic unit.

2.9 INTEGRATED CIRCUITS (ICs)

These are semiconductor materials that brought about third generation computers allowing devices to be made from logic gates, which are designed specifically for computer logical and arithmetic operations. These transistors are packaged in different scales to fit the purpose of the circuitry and help to determine how many transistors make up one chip. Hence, digital systems are constructed from such two-state element in form of silicon chips known as ICs. Their arrangements are usually as Dual In-Line Package, Straight Line and Circular Line package as they are grouped into families based on semiconductors that makeup their circuit design as thus: Bipolar Logic uses bipolar junction transistor throughout the circuit for a gate, which is the combination of the p-type and n-type materials for the construction of the circuitry and logic gate. The voltage 0v and 5v represent the logic gate

values 0 and 1 respectively. Examples are RTL, TTL, ECL.

MOS is based on Metal Oxide Semiconductor Field

Effect Transistor, devices with its resistance between the two (2) terminals and is controlled by the voltage on the third terminal. They are further subdivided into NMOS, PMOS and CMOS

Logic family is described by characteristics as explained:

- ✓ Logic Level are voltage values assigned to the binary 0 and 1 states for a given family. It is known as minimum voltage level at which the circuit switches from one state to the other. The switching between logic 1 and 0 values takes place only if the input level is above 2.5v or below 0.8v. Hence, Logic 1 is between 2.5v and +5v while Logic 0 is between 0v and 0.8v.

- ✓ Noise Margin and Immunity Any undesired voltage at the input of a circuit is called noise. Generally gate circuits have the ability to tolerate disturbances at their input – so that there is a margin between its input and output voltage levels corresponding to each logic value due to the noise disturbances. Hence, the degree to which the gate suppresses the effects of undesired pulses is known as Noise Immunity, because large noise pulses cause the circuit to switch suddenly and give false information. Noise margin is the difference between the circuits' operating voltage and logic voltage; and the probability of false switching due to noise is reduced if the noise margin is high.

- ✓ Propagation Delay/Transition Time is the measure of the speed of operation of the logic circuit or it can be simply viewed as the total time delay between the time the input is applied and the time the output state gets switched. That is the total time taken for the output to respond and get switched due to the input signal applied. It is the time taken for the circuit to respond to an input. A major determinant of this feature is the load on the output line of the gate, particularly the capacitive components.

- ✓ Fan-Out In sequential circuits, the output of a circuit can be used as input for another circuit so that a logic gate acts as input to another gate. This property describes the maximum number of allowable output, each rated at a certain mA that a logic circuit may have and still operate properly. Hence, the maximum number of logic gates that can be connected to the output of another gate, while the logic gate still maintains a nominal value describes.

- ✓ Fan-In describes the maximum number of allowable input, each rated at a certain mA that an IC logic circuit may have and still

operate properly.

✓ Power Dissipation is one half the power that an integrated logic circuit draws from the power supply. It is the amount of power consumed by a logic gate while operating. Power consumption should be minimized with applications that use batteries. Power consumed by an integrated circuit is physically removed in form of heat – even if the rate at which the heat is removed is limited, a limitation to attainable density and number of the gates in an IC. Because power consumption is different for the binary states – power dissipation becomes the total power dissipation plus its average.

Table 2.3: Comparative analyses of IC families: Med for Medium

ICs Cost Propagati- Noise Fan Fan Power

on Delay Margin In Out Dissipate

DTL Med High Low High Med Med

RTL Med Med Low Low Low Low

TTL Med Low High High Med Med

ECL High Very low Med Med High High

CMOS Med High High High High Very Low

Med – Medium

CHAPTER QUESTION

2-1 Explain the concept of Data representation in your own term.

2-2 Convert 98, 543 and 2176 to their octal equivalence and convert these binary numbers to decimal 10010 and 1000111

2-3 Express these in their hexadecimal equivalent 2E, 83D, and DF7

2-4 List and describe the features associated with the powers of a CPU based on the Von Neumann's stored program concept.

2-5 Convert these decimal numbers to binary: 37, 56, 89, and 1002

2-6 Perform the following binary additions

10100111 10101010 11111111

+ 1011101 +10111011 +11111111

2-7 Carry out the following binary additions

10100111 10101010 11111111

- 1011101 - 10111011 - 1111001

2-8 Define the following in your own term

(a) Logic, Argument, Proposition and negation

(b) Inductive and deductive arguments or reasoning

2-9 Define Karnaugh's map? Draw the circuits and their maps for:

$$a. K = A' B' C' D' + A C' D' + A B' C' D' + A' B' C D + A B C'$$

$$b. K = A' B' C + A C' + A B' C' + A' B' C' + A B C + AC$$

2-10 Define what you understand by the following with diagrams:

Multiplexer and Demultiplexer, Binary Decoder, Full and Half Adders, Arithmetic and Logic Unit (ALU)

2-11 Given the map below, deduce the expressions

AB 00 01 11 10

CD

00

01 1 1

11 1 1 1

10 1 1

AB

C 00 01 11 10

0 1

1 1 1 1

2-12 Minimize the circuits given the map in 2-4(a) and (b) respectively.

2-13 In your own terms, define the following:

- a. Logic level b. Propagation delay c. Fan in
- d. Fan out e. Power dissipation f. Noise Margin

Chapter Three

THINKING PROBLEM – AIDING COMPUTERS!

INTRODUCTION

Computers are incredible devices. They extend what we can traditionally, do with our brains. And with them as computing devices, we can do things faster, explore their use in almost every sphere of life, we can keep track of a vast amount of data, we can effectively and efficiently use such processed information as evidence to support our decisions, and we can share our ideas and knowledge with other people no matter where they are, when they need it and how they wish to get it. Getting computers to help us to solve problems is a 2-step process: (a) first, we think about all requisite steps needed to solve a problem, and (b) then, we explore our technical skills to get the computer working on the problem.

Even when it is something as simple as using a calculator to solve a word problem in mathematics. First, it is a requisite that we have to understand and interpret the problem before the calculator can help out with the arithmetic bit. Similarly,

if we wish to make an animation, we start by planning the story and on how to shoot it. Afterwards, we can then use computer hardware and software to help you get the work done. In both of these scenarios and examples, the thinking that is undertaken before starting work on a computer is known as computational thinking.

Computational thinking describes the science, processes and approaches we draw on when thinking about problems or systems in such a way that a computer can help us with these. Computational thinking is not the same as thinking about computers, or like computers. This is also true because computers cannot or do not think for themselves, or of their own accord (atleast, not yet) without the aid of programs to help them as processing units accomplish such feat and task. Computational thinking is simply, about looking at a task or problem in a way that a computer can conveniently be used to help us solve the task before us. To accomplish this type of thinking (i.e. computational thinking), we use a variety of computational thinking processes to tackle a problem. These processes (concepts and approaches) include:

1. Logical Reasoning – predicting and analyzing
2. Algorithms – making steps and rules
3. Decomposition – breaking down the problem into smaller parts
4. Abstraction – removing all unnecessary details
5. Patterns Generalization – spot similar data/objects, and using these similarities as evidence to make meaning of the process.
6. Evaluation – means making the right judgements based on the conclusions reached that are backed up with evidence.

What to do with Computational Thinking

Computational thinking describes the type or sort of thinking that computer scientists and software developers engage in. Many people think this way too. This thinking processes and approaches help with computing and informatics, and can be useful in many other domains. A team of software engineers to create a new game, team of instructors can plan together to put up school play, or to organize an educational visit.

Whatever the case and task, we should be able to use some tact and explore the following: (a) first, take a complex task and break it down into smaller problems, (b) secondly, work out the steps to get it done, (c) thirdly, we must manage the complexity of the task by focusing on the key details, and

(d) fourthly, we must explore methods previously used for similar projects, and see how they be of help here?

Concepts to Computational Thinking

Computational thinking is at the crux, pivot and epicenter of computing education and ambition. This paradigm posits that a high-quality computing education equips her graduates to explore and exploit computational thinking and creativity to understand and change the world. Whilst, programming is an important and crucial feat to computing science, it is wrong to see this as an end in itself. Rather, computational thinking provides the requisite platform to propel practical experience with programming with insights that can best be developed and explored as thus. We should not just see computational thinking as a new name for 'problem-solving skill'. Rather, we should see computational thinking as skills needed to solve problems over a wide range of applications, and across other disciplines. Though, it is most effectively learned via rigorous, creative processes of writing codes as seen in Figure 3.1

Figure 3.1. Concept and Approach to Computational Thinking

Logical Reasoning

If you set up two computers in the same way, give them the same instructions (the program) and the same input, you can pretty much guarantee the same output. Computers do not make things up as they go along or work differently depending on how they happen to be feeling at the time. This means that they are predictable. Because of this we can use logical reasoning to work out exactly what a program or computer system will do. Children quickly pick this up for themselves: that is, the experience of watching others and experimenting for themselves allows even very young children to develop a mental model of how technology works. A child learns that clicking the big round button pops up a list of different games to play, or that tapping here or stroking there on the screen produces a reliably predictable response. The process of using existing knowledge of a system to make reliable predictions about its future behaviour is one part of logical reasoning. At its crux, logical reasoning is being able to explain why something is the way it is – while, seeking to work out if there is a better way as to how that something should be. Logic is the fundamental to how computers work: deep inside a computer's central processing unit (CPU), every

operation the computer performs is reduced to logical operations carried out using electrical signals. It is because everything a computer does is controlled by logic, that is why we can use logic to reason about program behaviour.

Software engineers use logical reasoning all the time in their work. They draw on their internal mental models of how computer hardware, the operating system (like Windows, Macintosh, X etc) and the programming language they are using all work, in order to develop new code that will work as they intend. They will also rely on logical reasoning when testing new software and when searching for and fixing the 'bugs' (mistakes) in their thinking (known as debugging) or their coding when these tests fail.

There are many ways that we already use logical reasoning across various curriculum as we proceed in life namely:

1. In English, students may explain what they think about any character in a novel they just read, or to predict what they feel and think a character will do next based on the character's action so far.
2. In Mathematics or other Science subjects, a learner or student should explain how they have arrived at their conclusions from the results of their experiments.
3. In history, a student should discuss those logical connections between cause-and-effect overtime in history; And, they should be able to understand how historical knowledge is constructed from a variety of sources.
4. In Computing – a stage-1 learner is expected to use logical reasoning to predict the behaviour of simple programs. It may also include those they themselves have written, and may also include them predicting what happens when they play a game; while, a stage-2 learner is expected to use logical reasoning to explain how some simple algorithms work and to detect and correct errors in algorithms and programs.

Classroom Activity

1. Provide students with a robot. They should make predictions of where the robot will end up when the go button is pressed. And they should explain why they think of that. Being able to give a reason for their thinking is what using logical reasoning is all about.

2. In their own coding, logical reasoning is key to debugging (finding and fixing the mistakes in their programs). Ask them to look at each other's codes/programs and spot bugs. Encourage them to test the programs to see if they can isolate exactly which bit of code is causing a problem. If their codes fail to work, get them to explain their code to a friend or even an inanimate object.
3. Give them a program of your own samples and ask them to work backwards from the code to work out what it will do.
4. Ask pupils to think carefully about some school rules, for example those in the school's computer Acceptable Use Policy. Can they use logical reasoning to explain why the rules are as they are?
5. There are many games, both computer-based and more traditional, that draw directly on the ability to make logical predictions. Organize for the pupils to play noughts and crosses using pencil and paper. As they are playing, ask them to predict their opponent's next move. Let them play computer games such as Minesweeper, SimCity, etc. Ask them to pause at certain points and tell you what they think will happen when they move next. Consider starting a chess club if your school does not already have one.

Algorithm

An algorithm is a finite sequence of instructions or rule to get something done. Your fastest route to home from school may be → turn left, walk for 2-miles, turn right, etc). That, is an 'algorithm'. A finite, logically ordered number of instructions (or steps) that effectively yields a solution to a problem.

There are many algorithms (i.e. routes) that will accomplish the same goal as getting you to your destination from any one point. Even the Google-Map algorithm for working out the shortest/fastest route. Search engines like Google and Bing use algorithms to put a set of search results into order, so that more often than not, the result we are looking for is at the top of the front page.

Your Facebook news-feed is derived from your friends' status updates and other activity. It only shows that activity which the algorithm (EdgeRank) thinks you will be most interested in seeing. So also, the recommended views you get on eBay Netflix, Twitter, and other social networking sites. All these are algorithmically generated, based in part on what other people are interested in. Given the extent to which so much of their lives is affected by algorithms, students should grasp and understand what an algorithm is. A simple algorithm can

be seen in Figure 3.2 to help student get an idea of what an algorithm is.

Figure 3.2. Schematic diagram of an Algorithm

Students have been tasked to and will already use algorithms in many different ways across the school as thus: (a) lesson plan or course compact of curriculum is regarded as an algorithm for teaching a course, (b) a cooking recipe can be thought of, as an algorithm, (c) we can think of instructional writing as a form of algorithm in English; while for science – we may talk about method of an experiment as an algorithm, and (d) for Mathematics, your approach to mental arithmetic (or many computer-based educational games) might be an implementation of a simple algorithm.

We expect students to have: (a) first, an understanding of what algorithms are, and how they are used in programs of digital devices. There can be many algorithms to solve the same problem, and each of these can be implemented using different programming languages of choice on the different computer systems, and (b) secondly, students must build on this to design programs with particular goals in mind, which will draw on their being able to think algorithmically as well as using logical reasoning to explain algorithms and to detect and correct errors in them.

To practice this, encourage pupils to carry out the steps for an algorithm: to follow the instructions themselves rather than writing these as code for the computer. Errors and inconsistencies should become apparent with time. While, some programming languages like BASIC, Java, Python can make it seem unnecessary to go through the planning stage of writing a program, it is good practice for pupils to write down the algorithm for a program, perhaps as rough jottings, a storyboard, pseudocode (a written description of how a program will operate) or even as a flow chart. This makes it far easier for them to get feedback from you or their peers on their algorithms before implementing these as code on their computers.

Classroom Activity

1. A GSM company just arrived and wishes to make investment in Delta State by citing their Base-Transceiver Stations (BTS) around the State.
- Predict how, where and which places (urban, semi-urban and rural)

areas that will be connected to the network the GSM company intends to develop in the State.

- What components and factors should be considered when citing these connection provider site(s).

- Explain why and where will be your initial cum pivot connection point (if you are Head of Operations for the said GSM company)

- Draw the network showing all sites for Delta State of towns and villages to be connected, and state the reasons for your choice.

2. Generate a list of numbers by asking the students to guess as many numbers between 1-100. Recall the BinarySearch and LinearSearch.

The students are to perform both search types and generate their answers. They are to generate the algorithm(s) (in steps 1-to-N) to resolving the task for the BinarySearch and LinearSearch

Decomposition

The process of breaking down a problem into smaller manageable parts is known as decomposition. Decomposition helps us solve complex problems and manage large projects.

This approach has many advantages. It makes the process a manageable and achievable one – large problems are daunting, but a set of smaller, related tasks are much easier to take on. It also means that the task can be tackled by a team working together, each bringing their own insights, experience and skills to the task. Decomposing problems into their smaller parts is not unique to computing: it's pretty standard in engineering, design and project management. Software development is a complex process, and so being able to break down a large project into its component parts is essential – think of all the different elements that need to be combined to produce a program, like PowerPoint.

The same is true of computer hardware: a smartphone or a laptop computer is itself composed of many components, often produced independently by specialist manufacturers and assembled to make the finished product, each under the control of the operating system and applications. You must use decomposition to tackle big projects at school, just as programmers do in the software industry. Our Computer Science curriculum is typically decomposed into a 4-Years programme with courses at the various Years (called Levels). Courses are further decomposed into topics with units of work and individual lessons/activities. Notice how the project is tackled by a team working together (your colleagues), and

how important it is for the parts to integrate properly.

Figure 3.3. Decomposition of a School Trip

We already use decomposition in many different ways across the curriculum. In science, we label diagrams to show the different parts of a plant, or the different nations which make up Africa. In English, we plan the different parts of a story. In general project planning, planning a research project for any subject or working collaboratively to deliver a group presentation. Technology can help with collaborative group work, or can even be a focus for it, and great collaborative tools are available in Office and other 'cloud'-based software. In Mathematics, we break-down a problem to solve it.

Students are expected to learn to 'decompose problems into smaller parts first, in a bid to solve them, through design and creation of a range of systems (and sub-systems) that have a particular goal in mind (a system is something with a number of interconnected components).

Students must plan to execute programs or systems using the features of decomposition to achieve it as thus: (a) first, work out what the different parts of the program or system must do, and (b) second, we must think about how these inter-relate. For example, a simple educational game is going to need some way of generating questions, a way to check if the answer is right, some mechanism for recording progress such as a score and some sort of user interface, which in turn might include graphics, animation, interactivity and sound effects.

Classroom Activity

1. Plan out opportunities for students to experience team-work through their collaboration on a software development project to build small apps or even challenging programming project such as making a computer game or even a mobile phone app.
2. The process of even having them develop a term-paper for submission requires the task of writing and filling up the agreed number of pages with words that can be further decomposed via planning, research, drafting, reviewing and publishing (typing) phases.

Patterns and Generalization

The method of looking for a general approach to a class of problems is called generalization. By identifying patterns, we can make predictions, create rules and solve more general

problems. To learn about the area of a rectangle, students can be asked to find the area of a particular rectangle by counting the centimeter squares on the grid on which it is drawn. But a better solution would be to multiply the length by the width. This method is both quicker and is a method that will work on all rectangles, including really small ones and really large ones. Although it takes a while for students to understand this formula, once they do it's so much faster than counting squares. Students are likely to encounter the idea of generalizing patterns in many disciplines and areas of their primary focus and endeavors as in Figure 3.4.

Figure 3.4. Patterns Matched

From an early age, we must become familiar with repeated phrases in nursery rhymes and stories. We should later then notice the repeated narrative structures in traditional tales. In music, we must recognize repeating the melodies or bass lines in many musical formats. In Maths, we must typically aim to undertake investigations in which we spot patterns in a problem as well as deduce generalized results. In English, we might notice and identify common rules for spellings and grammar, and also note their exceptions.

Abstraction

The process of deciding those details to be highlighted and those details we can ignore. Abstraction is about simplifying things; identifying what is important without worrying too much about the detail. Abstraction allows us to manage complexity. We use abstractions to manage the complexity of life in schools. School timetable is also an abstraction of what typically happens weekly in a citadel of learning as it aims to capture key information and knowledge of: (a) who teaches what course, (b) what courses is to be taught or undertaken for a specific time-slot, (c) to whom is the course taught, and (d) which venue and time is the said course to be taught. We must also realize that it leaves out that angle of further layer of complexity, such as the learning objectives and activities planned in any individual lesson as in Figure 3.5.

Figure 3.5. Schematic diagram of point A to B abstracted

Abstraction is such a powerful way of thinking about systems and problems that it seems worth introducing pupils to this whilst they're still at primary school. Word problem in Maths

involves identifying key data, and establish means on how to represent the task (in arithmetic, algebra or geometry). Maps are an abstract complexity of the environ with various scales providing a layered nature of the chosen site. Histories are an abstraction of detail present in local histories or individual biographies as abstractions of actual events. The piano score of a pop song is also an abstraction for that piece of music.

Our curriculum hopes that students (although, as part of the overarching aims of this course/subject) – must understand and apply the fundamental principles and concepts of computer science, including abstraction, logic, algorithms and data representation. Students must learn about the process of abstraction from playing interactive games that involves real-world simulations. We encourage students to be curious about how things work, and to think of what happens inside the computer, or online as they surf the web. Students must put together a presentation on a topic they know about, they will need to focus on the key information, and think about how this can be represented, whilst leaving to one side much of the detail of the subject: this too involves abstraction.

Classroom Activity

Encourage students learning to program – to create their own games that can either be based on real world systems or otherwise. They will need to use some abstraction to manage the complexity of their game system. They can simulate a table tennis game (i.e. ping-pong) such that the simulation can include the ball's motion in 2D, and how it bounces off the bat. Ask them to think carefully about what detail they need to include, and what can be left out when programming a similar game.

Evaluation

This is often the most underemphasized feat – even when it is the most crucial step in problem solving. It is the reason and crux of why researches are conducted. The reason why people do not embark on evaluations includes time, cost, the unease with evaluation, and the fear of being challenged in their decisions reached with evidence. Evaluation is essential because it seeks to determine how effective the proposed solution is. Evaluation can take many forms, and it can be as simple cum brief as a conversation with key individuals and members; Or, it can be as elaborate as sophisticated surveys and in-depth record analysis. Evaluation seek to determine how well the solution works/fits or why the solution

does not work. We must or can decide how comprehensive evaluation should be, based on the complexity of the task or problem and the decided solution.

Evaluation must be poised towards reaching the cost-benefit analysis. It must be capable to address 4-critical features in the search for solution. It must seek to address the following strengths, weaknesses, opportunities, and threats, which must also become and form the basis to ensure that a proposed solution complies with. Measuring a solution based on these – will help ensure that whatever solution is put forward will always outperform the existing/current system as seen in Figure 3.6 that details difference in method.

Figure 3.6. Difference in Evaluation of the Used Methods

First, to decide how to execute a solution, we must uncover possible weaknesses. What often sounds and looks as a good decision in our mind or on paper, may in practice prove to be inoperable and non-functioning. We often end up here when we try to initiate an action of the plan. Where the solution is unworkable, it can be abandoned in favor of one that is more likely to work. Depending on the severity of the problem and the quality of solution desired, objectives and approaches for problem solving will vary. For minor tasks, we can focus on quickly reaching a solution as the main goal. But, where we seek or are primarily concerned with finding quality solution as a main objective – we will always factor in more time for the problem-solving process.

For effectiveness, problem solving must reach or arrive at a solution that both gets the job done, efficiently use available resources, promote cooperation among team members, and fosters competence. Problem solving is an ongoing process that is an integral part of everyday life either at home or at work. A problem must first be felt as clear, understood, and alternate choices created; Before, we consider its solution, implement the decision, and evaluate how well the solution works. Evaluation should also call for a back-trace and review of each step advanced where they seem unclear.

Classroom Activity

Plan a traditional 'design, code and evaluate task such as plan the process for building a house. Break this complex problem down into smaller stages, such as: (a) planning the design (abstraction to capture key elements of

building), (b) sourcing of the materials (decomposition to identify the different components), (c) assemble the materials to create the instrument (algorithm – systematic, step-by-step approach to put the house together), and evaluate and test how the solution works.

Approaches to Computational Thinking

As with the processes, there are a number of approaches that characterize computational thinking. If students are to start thinking computationally, then they must develop these approaches to their work, so they can be more effective in putting their thoughts into action.

Tinkering is that willingness to experiment and explore with work. An element of learning a new programming language or exploring a new system look quite similar to this sort of purposeful play. It is often an effective approach to learning. Open-source software makes it easy to take someone else's code, look at how it is written, see all the comments on how the codes fit together, and then adapt it to your own task or particular purpose. Many platforms like TouchDevelop and Scratch can positively encourage users to practice this by looking at other programmers' work and use it as a basis for their own creative coding. We must encourage pupils to play with a new piece of software, sharing what they discover about it with one another; Rather, than just teach or explain to them exactly how it works. Students must look for ways to use other experienced coders' code as a pivot/pilot starting point for their own programming projects.

Figure

Creating: Programming is a creative process, which involves both originality, making something of value and something that typically is useful or fits a specific purpose. Students must approach tasks with a creative spirit, and look for those programming tasks that allow them express such creativity also; Rather, than merely arriving at a right answer. Students must reflect on the quality of the work they produce – simply by critiquing their own against others. The process of always looking for ways to improve on a task is becoming common practice in software development. Look for projects in which artistic creativity is emphasized such as working with images, animation, virtual environments or even 3D printing.

Figure

Debugging: Due to the nature and complexity of coding, many programs often do not work at first impulse. And even when it does, it does not often work as intended. So, getting users to take responsibility of thinking through the algorithms and code, to identify and fix errors is an important part of learning to think, and work like a scientist and programmer. Get students to check through their Maths work, to proofread their English writings and debug another programmer's code by looking for mistakes and suggesting improvements. There is always evidence that learning from mistakes – is continued to particularly be an effective approach. Allowing students to embark therein this process of debugging their own and/or other users' code is one way to achieve this. Teachers must learn also to keep an eye on the bugs his/her students often encounter, as this can sometimes reveal misconceptions that s(he) may need to address.

Figure

Persevering: Coding can be quite hard, tedious and often mind-bending. It requires patience as a crucial part and facet to its appeal of writing elegant, effective code often comes with its intellectual challenge. This requires not only the apt understanding of the ideas of the algorithms being coded; But, also of the programming language being used and the willingness to persevere with something that is quite difficult and frankly, very frustrating. The 'growth mind-sets' suggests that hard work and a willingness to persevere in the face of difficulties are key factors to achieving the set educational outcomes. Students must have commitment to the cause of coding, and be consistent in their resolve never to give up. These are great strategies they can imbue and imbibe when they encounter difficulties with their programming work, such as working out exactly what the problem is, searching for the solution on Bing or Google, or asking a friend for help.

Collaborating: Software has/is often developed by teams of programmers working together on project. Many see pair programming and collaboration as an effective development method, with two/more programmers sharing a screen and a keyboard, working together to write software. Typically one programmer acts as the driver, dealing with the detail of the programming, whilst the other takes on a navigator role,

looking at the bigger picture. The two programmers regularly swap roles, so as to grasp both granular details of the program and the big picture. Working in a larger group can help us develop a number of additional skills where each student contributes their own particular talents to the shared project. It is important to remember that all students should develop their understanding of each part of the process; Thus, they must share and exchange roles and peer-tutoring should be incorporated into such activities.

Question

Classroom Activities

1. Ask your pupils to write a recipe for a sandwich, thinking carefully about each step that needs to be carried out.
 - Please, detail step-by-step instruction sequence (i.e. algorithm).
 - Ask them to share each other's recipes – so they can efficiently spot patterns in them (i.e. generalization).
 - Read out a range of other related recipes, and discuss the layers of simplification (abstraction) present relatively simple recipes.
2. Break them into groups and also challenge them to work individually or collaboratively on more complex projects, for example researching and writing up aspects of a curriculum topic such as the Viking invasion, or putting together an assembly or a class play. In each case ask them to note down the individual steps needed for the task and to think about what they have left out to make the subject fit their brief.

Term Paper 1

A GSM company just arrived and wishes to make investment in Delta State by citing their Base-Transceiver Stations (BTS) around the State. Using the various computational thinking concepts of: logical reasoning, algorithm, decomposition, abstraction, patterns/generalization and evaluation – write up a report with a detailed map of the area where you are assigned of the road network and why you feel optimistic about the GSM network provision and tele-penetration Project in Nigeria. (15-pages)

REFERENCES

- Ojugo, A. A., Oyemade, D. A., & Allenotor, D. (2016). Solving For Computational Intelligence the Timetable-Problem Advances in. *Advances in Multidisciplinary and Scientific Research Journal*, 2(2), 67–84.
1. Barefoot Computing, available at [web]: barefootcas.org.uk/barefoot-primarycomputing-resources/concepts/abstraction/
 2. BBC Bitesize, 'Abstraction', available at: www.bbc.com/bitesize

bbc.co.uk/education/guides/zttrcdm/revision.

3. BBC Cracking the Code, 'Simulating the Experience of F1 Racing Through Realistic Computer Models', available at: www.bbc.co.uk/programmes/p016612j.

4. Google for Education, 'Solving Problems at Google Using Computational Thinking', available at: www.youtube.com/watch?v=SVVB5RQfYxk.

5. The Art of Abstraction – Computerphile', available at: www.youtube.com/watch?v=p7nGcY73epw.

All Correspondence:

Ojugo Arnold P.A.C

College of Sports and Science Education

Department of Computer Science

Mosogar, Delta State

Tel: 08050331178, 08034072248

We value your opinion – Please share it with us to help us improve.

1. Please Rate this text by ticking the ones that apply S for Superior, BTM for Better than Most, A for Average and P for Poor.

Items Stated S BTM A P

Author's Writing Style

Readability and Organisation

Layout And Design

Understandability And Clarity

Illustration And Photos

Examples, Problems & Exercises

Topic Selection

Explanation Of Difficult Concepts

Application To Real Life

How Current is Text Coverage

2. List the chapter(s) you were not taught properly due to textbook

3. State the chapters you do not liked to be in this textbook_____

4. State the chapters you liked least _____

5. State the chapters that you liked best_____

6. What topics do you think should be covered, make it concise.____

7. Do you plan to sell the text after usage? ____ Yes ____ No

8. Why? _____

9.